

Cálculo Numérico

Fundamentos, métodos e investigações
computacionais em Python

Notas de aula e projetos organizados a partir da disciplina de
Cálculo Numérico

Compilados e organizados por

Jeann Victor

Prefácio

Este livro organiza os principais conceitos da disciplina de Cálculo Numérico e os conecta às implementações e investigações computacionais presentes no acervo de 2026/1. O texto central foi escrito como guia de estudo e consulta. Ao final, quatro relatórios acadêmicos completos preservam os experimentos, tabelas, gráficos e análises produzidos durante a disciplina.

O material distingue resultados teóricos, decisões de implementação e evidências experimentais. Explicações complementares foram incluídas para formar uma narrativa contínua, mas os resultados numéricos específicos permanecem nos relatórios originais, sem alteração.

Sumário

Prefácio	i
1 Pensamento numérico e qualidade da solução	1
1.1 Do problema real ao resultado computacional	1
1.2 Erro, resíduo e condicionamento	1
1.3 Verificação em três níveis	2
2 Representação numérica e propagação de erros	3
2.1 Aritmética de ponto flutuante	3
2.2 Cancelamento catastrófico	3
2.3 Critérios de parada	3
3 Equações não lineares	5
3.1 Isolamento da raiz	5
3.2 Bisseção e falsa posição	5
3.3 Iteração de ponto fixo	5
3.4 Newton e secante	6
3.5 Escolha do método	6
4 Sistemas lineares: métodos diretos	7
4.1 Eliminação de Gauss e pivoteamento	7
4.2 Fatorações LU e PLU	7
4.3 Cholesky	7
4.4 Sistemas tridiagonais	8
4.5 Condicionamento e matriz de Hilbert	8
4.6 PageRank como sistema linear	8
5 Sistemas lineares: métodos iterativos	9
5.1 Decomposição e ideia básica	9
5.2 Gauss-Jacobi e Gauss-Seidel	9
5.3 SOR	9
5.4 Gradiente conjugado	10
5.5 Equação do calor como projeto integrador	10
6 Interpolação polinomial e splines	11

6.1	O problema de interpolação	11
6.2	Diferenças divididas de Newton	11
6.3	Fenômeno de Runge	11
6.4	Splines cúbicas	12
7	Ajuste de curvas e mínimos quadrados	13
7.1	Aproximação na presença de ruído	13
7.2	Resíduos e escolha do modelo	13
7.3	Linearização de modelos	13
7.4	Interpolação ou regressão?	14
8	Diferenciação numérica	15
8.1	Diferenças finitas	15
8.2	Escolha do passo	15
8.3	Dados tabelados	15
8.4	Segunda derivada e extrapolação	15
9	Integração numérica	17
9.1	Newton-Cotes compostas	17
9.2	Quadratura de Gauss	17
9.3	Integração de dados experimentais	17
9.4	Aplicações integradoras	18
10	Projetos reprodutíveis em Python	19
10.1	Separação de responsabilidades	19
10.2	Testes numéricos	19
10.3	Gráficos que respondem perguntas	19
10.4	Reprodutibilidade	19
11	Guia de escolha de métodos	20
11.1	Perguntas antes de programar	20
11.2	Mapa de decisão	20
11.3	O que relatar	20
12	Roteiro de revisão e exercícios	21
12.1	Revisão conceitual	21
12.2	Exercícios computacionais	21
12.3	Critério de conclusão	21
	Cadernos de investigação computacional	22

Capítulo 1

Pensamento numérico e qualidade da solução

1.1 Do problema real ao resultado computacional

Cálculo Numérico estuda como obter aproximações confiáveis para problemas matemáticos que não admitem uma solução fechada conveniente, que envolvem dados medidos ou cuja dimensão torna inviável uma solução simbólica. O objetivo não é apenas produzir um número. É construir um processo no qual modelo, algoritmo, implementação e interpretação possam ser auditados.

Um fluxo numérico completo possui quatro camadas. A primeira é a modelagem: quais variáveis, hipóteses e equações representam o fenômeno? A segunda é a discretização ou reformulação: como transformar o problema contínuo ou implícito em operações executáveis? A terceira é o algoritmo: quais propriedades de convergência, estabilidade e custo ele possui? A quarta é a validação: como distinguir uma solução plausível de uma resposta apenas numericamente produzida?

Princípio central.

Uma aproximação só é útil quando vem acompanhada de um critério de erro, de um teste de consistência e de informação suficiente para que o experimento seja repetido.

1.2 Erro, resíduo e condicionamento

Se x^* é a solução exata e $\hat{x}$ é a aproximação, o erro absoluto é $|x^* - \hat{x}|$. Quando a escala do problema importa, utiliza-se o erro relativo

$$e_r = \frac{|x^* - \hat{x}|}{|x^*|},$$

desde que $x^* \neq 0$. Em muitos problemas a solução exata é desconhecida. Nesse caso, mede-se uma condição que deveria ser satisfeita. Para um sistema $Ax = b$, por exemplo, o resíduo é

$$r = b - A\hat{x}.$$

Um resíduo pequeno mostra que a aproximação quase satisfaz as equações, mas não garante sozinho que $\hat{x}$ esteja perto de x^* . Se o problema for mal-condicionado, pequenas perturbações em A ou b podem provocar grandes mudanças na solução.

O número de condição de uma matriz não singular é

$$\kappa(A) = \|A\| \|A^{-1}\|.$$

Valores próximos de 1 indicam baixa sensibilidade. Valores muito grandes indicam que erros de dados e arredondamento podem ser amplificados. Condicionamento é propriedade do problema; estabilidade é propriedade do algoritmo. Um bom método evita acrescentar uma amplificação desnecessária à sensibilidade que já existe.

1.3 Verificação em três níveis

- verificar casos pequenos cuja resposta é conhecida;
- medir resíduos, erros relativos ou conservação de grandezas;
- variar tolerância, passo, malha e precisão para observar se o comportamento acompanha a teoria.

Nos relatórios anexos, essa estratégia aparece repetidamente: compara-se a solução com rotinas de referência, mede-se o resíduo e depois se altera um parâmetro para observar estabilidade, custo ou convergência.

Capítulo 2

Representação numérica e propagação de erros

2.1 Aritmética de ponto flutuante

Computadores representam apenas um subconjunto finito dos números reais. Em notação simplificada, um número de ponto flutuante tem sinal, mantissa e expoente. O resultado de uma operação precisa ser arredondado para o número representável mais próximo. Por isso, em geral,

$$\text{fl}(a \circ b) = (a \circ b)(1 + \delta), \quad |\delta| \leq u,$$

onde u é a unidade de arredondamento e $\circ$ representa uma operação elementar.

As identidades algébricas usuais podem perder validade exata. A adição não é plenamente associativa, comparações por igualdade podem falhar e subtrair números quase iguais pode destruir dígitos significativos.

2.2 Cancelamento catastrófico

Considere a e b próximos. Embora ambos possuam vários algarismos corretos, a diferença $a - b$ pode preservar poucos deles. Esse cancelamento aparece em fórmulas de derivação numérica, raízes de equações quadráticas e avaliações polinomiais mal organizadas.

Uma defesa é reformular a expressão. Outra é escolher uma escala adequada e evitar passos excessivamente pequenos. Na diferenciação por diferenças finitas, diminuir h reduz o erro de truncamento até certo ponto; depois, a divisão por h amplifica arredondamentos e o erro volta a crescer.

2.3 Critérios de parada

Um algoritmo iterativo não deve parar apenas porque atingiu um número fixo de iterações. Critérios comuns são

$$|x_{k+1} - x_k| \leq \varepsilon_a + \varepsilon_r |x_{k+1}|$$

e, quando existe uma equação $f(x) = 0$,

$$|f(x_{k+1})| \leq \varepsilon_f.$$

É recomendável combinar um teste de variação, um teste de resíduo e um limite máximo de iterações. Assim, o programa diferencia convergência, estagnação e divergência.

Checklist de implementação.

Registre tolerância, norma, máximo de iterações, motivo de parada e quantidade de passos. Sem esses dados, duas execuções aparentemente iguais podem representar critérios muito diferentes.

Capítulo 3

Equações não lineares

3.1 Isolamento da raiz

Antes de iterar, é preciso entender onde uma raiz pode estar. Se f é contínua em $[a, b]$ e $f(a)f(b) < 0$, o teorema do valor intermediário garante pelo menos uma raiz no intervalo. Essa condição sustenta os métodos de encaixamento.

O gráfico ajuda a formular hipóteses, mas não substitui o teste numérico. Um intervalo largo pode conter várias raízes; uma tangência pode não produzir mudança de sinal; uma descontinuidade pode imitar um cruzamento.

3.2 Bisseção e falsa posição

A bisseção escolhe o ponto médio $m = (a + b)/2$ e conserva a metade que mantém a mudança de sinal. Depois de k iterações, o comprimento do intervalo é

$$\frac{b - a}{2^k}.$$

O método é lento, mas previsível e robusto. A falsa posição substitui o ponto médio pela interseção da secante com o eixo:

$$x = \frac{af(b) - bf(a)}{f(b) - f(a)}.$$

Ela pode avançar mais rapidamente, mas em funções muito assimétricas um extremo pode permanecer praticamente fixo. O histórico dos intervalos revela esse comportamento.

3.3 Iteração de ponto fixo

Reescreve-se $f(x) = 0$ como $x = g(x)$ e aplica-se

$$x_{k+1} = g(x_k).$$

A reformulação não é neutra. Perto do ponto fixo x^* , a condição $|g'(x^*)| < 1$ favorece convergência local; se $|g'(x^*)| > 1$, pequenas perturbações tendem a crescer.

Duas formas algébricas equivalentes podem gerar processos com comportamentos opostos.

3.4 Newton e secante

O método de Newton lineariza a função em x_k :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Quando o ponto inicial é adequado e a raiz é simples, a convergência local costuma ser quadrática. A derivada pequena, uma aproximação inicial ruim ou uma função não regular podem causar saltos e divergência.

O método da secante aproxima a derivada com dois pontos:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Ele dispensa a derivada analítica e frequentemente oferece um equilíbrio interessante entre custo e velocidade.

3.5 Escolha do método

- use bisseção quando houver um intervalo com mudança de sinal e robustez for prioritária;
- use Newton quando a derivada estiver disponível e uma boa aproximação inicial puder ser obtida;
- use secante quando derivar for caro ou inconveniente;
- use ponto fixo somente depois de analisar a função de iteração;
- em aplicações críticas, combine um método rápido com salvaguarda por intervalo.

Capítulo 4

Sistemas lineares: métodos diretos

4.1 Eliminação de Gauss e pivoteamento

A eliminação de Gauss transforma $Ax = b$ em um sistema triangular superior $Ux = c$. A retro-substituição resolve então as incógnitas da última equação para a primeira. O custo dominante para uma matriz densa $n \times n$ cresce como $O(n^3)$.

Dividir por um pivô muito pequeno amplifica erros. No pivoteamento parcial, a linha do maior elemento em módulo da coluna ativa é trocada com a linha do pivô. O relatório de métodos diretos mostra experimentalmente que essa troca reduz drasticamente o erro em aritmética de precisão limitada.

Teste mínimo.

Depois de resolver o sistema, calcule $\|A\hat{x} - b\|$. Para comparar sistemas de escalas diferentes, use também um resíduo relativo, por exemplo $\|A\hat{x} - b\|/(\|A\|\|\hat{x}\| + \|b\|)$.

4.2 Fatorações LU e PLU

Na fatoração LU, escreve-se

$$A = LU,$$

com L triangular inferior e U triangular superior. Uma vez fatorada a matriz, cada novo vetor b exige apenas duas substituições triangulares. Isso é vantajoso quando a mesma matriz aparece com muitos lados direitos.

Com pivoteamento, a forma apropriada é $PA = LU$, em que P registra as trocas de linha. Ignorar P ao verificar a fatoração é um erro frequente.

4.3 Cholesky

Se A é simétrica definida positiva, existe uma fatoração

$$A = LL^T.$$

Cholesky aproveita a simetria, requer menos armazenamento e aproximadamente metade do trabalho de uma LU genérica. Antes de utilizá-la, deve-se verificar simetria e positividade. Tentar esconder uma falha adicionando um pequeno valor à diagonal muda o problema e precisa ser explicitamente justificado.

4.4 Sistemas tridiagonais

Matrizes tridiagonais aparecem em discretizações de equações diferenciais. O algoritmo de Thomas explora a estrutura e resolve o sistema em $O(n)$, enquanto uma eliminação densa desperdiçaria trabalho e memória. Esse é um exemplo de princípio geral: conhecer a estrutura do problema pode ser mais importante que escolher a implementação genérica mais sofisticada.

4.5 Condicionamento e matriz de Hilbert

Matrizes de Hilbert são exemplos clássicos de sistemas mal-condicionados. Mesmo que o resíduo calculado seja pequeno, os coeficientes da solução podem apresentar erro relevante. O experimento deve comparar precisão, número de condição e erro na solução, não apenas o resíduo.

4.6 PageRank como sistema linear

Em uma rede de páginas, o PageRank pode ser formulado como um sistema que combina a matriz de transição com um fator de amortecimento. O exemplo conecta álgebra linear, grafos e métodos numéricos. Também mostra que modelagem influencia condicionamento: alterar amortecimento e conectividade muda a sensibilidade e a velocidade dos métodos.

Capítulo 5

Sistemas lineares: métodos iterativos

5.1 Decomposição e ideia básica

Escrevendo $A = D - L - U$, os métodos estacionários constroem sequências da forma

$$x^{(k+1)} = Tx^{(k)} + c.$$

A convergência para qualquer ponto inicial ocorre quando o raio espectral satisfaz $\rho(T) < 1$. Critérios como dominância diagonal e o critério de Sassenfeld são condições suficientes úteis, mas não equivalem à condição espectral.

5.2 Gauss-Jacobi e Gauss-Seidel

Jacobi calcula todas as componentes novas usando somente valores da iteração anterior. Gauss-Seidel utiliza imediatamente os valores já atualizados. Essa diferença costuma acelerar a convergência de Gauss-Seidel, embora o resultado dependa da matriz e da ordenação das equações.

Jacobi possui paralelismo natural. Gauss-Seidel introduz dependências dentro da iteração. Em problemas grandes, a escolha pode envolver não apenas quantidade de iterações, mas arquitetura computacional, comunicação e memória.

5.3 SOR

O método de sobre-relaxação combina o valor antigo com a atualização de Gauss-Seidel. Para um parâmetro ω ,

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \omega \tilde{x}_i^{(k+1)}.$$

Quando $1 < \omega < 2$, a sobre-relaxação pode acelerar a convergência; valores inadequados podem piorá-la ou provocar divergência. O parâmetro deve ser estudado experimentalmente ou estimado a partir da estrutura do problema.

5.4 Gradiente conjugado

Para matrizes simétricas definidas positivas, o gradiente conjugado busca a solução em subespaços de Krylov e minimiza uma função quadrática associada. Em aritmética exata, terminaria em no máximo n passos, mas em ponto flutuante sua velocidade depende fortemente da distribuição dos autovalores e de $\kappa(A)$.

O pré-condicionamento substitui o sistema por uma forma equivalente mais favorável numericamente. Um bom pré-condicionador aproxima A , é barato de aplicar e reduz a dispersão espectral.

5.5 Equação do calor como projeto integrador

Ao discretizar a equação de calor estacionária em uma dimensão, obtém-se um sistema tridiagonal. Esse problema permite comparar Thomas, métodos estacionários e gradiente conjugado, além de observar como o refinamento da malha altera dimensão, condicionamento e custo.

Leitura dos resultados.

Não compare métodos apenas por tempo em uma única matriz. Registre dimensão, tolerância, número de iterações, norma do resíduo, estrutura da matriz e ambiente de execução.

Capítulo 6

Interpolação polinomial e splines

6.1 O problema de interpolação

Dados $n + 1$ pontos com abscissas distintas, existe um único polinômio de grau no máximo n que passa por todos eles. As formas de Lagrange e Newton representam esse mesmo polinômio, mas possuem características computacionais diferentes.

Na forma de Lagrange,

$$p_n(x) = \sum_{i=0}^n y_i L_i(x), \quad L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}.$$

A forma é conceitualmente direta, mas adicionar um novo ponto exige reorganizar todas as bases.

6.2 Diferenças divididas de Newton

Newton escreve

$$p_n(x) = c_0 + c_1(x - x_0) + \cdots + c_n \prod_{j=0}^{n-1} (x - x_j),$$

em que os coeficientes são diferenças divididas. Ao acrescentar um ponto, os coeficientes anteriores permanecem e apenas um novo termo é calculado. Isso é especialmente útil quando os dados chegam progressivamente.

6.3 Fenômeno de Runge

Aumentar o grau com nós igualmente espaçados não garante melhora uniforme. Para certas funções, surgem oscilações intensas nas extremidades do intervalo. Nós de Chebyshev concentram pontos próximos às bordas e reduzem o máximo do produto que controla o erro de interpolação.

O ensinamento é mais amplo: mais parâmetros podem piorar uma aproximação quando a geometria do problema é ignorada.

6.4 Splines cúbicas

Splines constroem polinômios de baixo grau em subintervalos e impõem continuidade entre as peças. A spline cúbica combina flexibilidade local e suavidade global, evitando muitas oscilações de um polinômio único de grau elevado.

Ao escolher condições de contorno, como spline natural ou derivadas prescritas, deve-se considerar o conhecimento disponível sobre o fenômeno.

Capítulo 7

Ajuste de curvas e mínimos quadrados

7.1 Aproximação na presença de ruído

Interpolar força a curva a passar por cada observação. Em dados experimentais, isso pode significar ajustar também o ruído. O método dos mínimos quadrados escolhe parâmetros que minimizam

$$S(\theta) = \sum_{i=1}^m (y_i - \hat{y}(x_i; \theta))^2.$$

Para um modelo linear nos parâmetros, o problema pode ser escrito como

$$\min_{\beta} \|X\beta - y\|_2.$$

As equações normais $X^T X \beta = X^T y$ são didáticas, mas podem piorar o condicionamento porque $\kappa(X^T X)$ é aproximadamente $\kappa(X)^2$. Fatorações QR são preferíveis em implementações robustas.

7.2 Resíduos e escolha do modelo

Um erro global pequeno não garante um modelo adequado. O gráfico de resíduos deve parecer sem padrão sistemático. Curvatura remanescente, variância crescente ou blocos de sinais iguais indicam estrutura não capturada.

O grau de um polinômio deve equilibrar viés e variância. Métricas em dados de validação, análise física e simplicidade interpretável são mais confiáveis que aumentar o grau até o erro de treinamento quase desaparecer.

7.3 Linearização de modelos

Modelos como $y = ae^{bx}$ ou $y = ax^b$ podem ser linearizados com logaritmos. A transformação facilita o ajuste, mas muda a estrutura do erro. Minimizar desvios no espaço logarítmico não é igual a minimizar desvios na escala original. A decisão deve ser registrada e validada nas duas escalas relevantes.

7.4 Interpolação ou regressão?

- use interpolação quando os valores forem considerados exatos e for necessário reproduzi-los;
- use regressão quando houver ruído e o objetivo for estimar uma tendência;
- use splines suavizadoras quando se desejar flexibilidade local sem atravessar cada observação;
- reserve um conjunto de validação quando a meta for previsão.

Capítulo 8

Diferenciação numérica

8.1 Diferenças finitas

Expansões de Taylor produzem fórmulas progressiva, regressiva e central. As aproximações de dois pontos são

$$f'(x) \approx \frac{f(x+h) - f(x)}{h},$$

$$f'(x) \approx \frac{f(x) - f(x-h)}{h},$$

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}.$$

As duas primeiras possuem erro de truncamento $O(h)$. A fórmula central possui erro $O(h^2)$ porque a simetria cancela o termo de primeira ordem no erro.

8.2 Escolha do passo

Reduzir h melhora o erro de truncamento, mas amplifica arredondamento e cancelamento. O gráfico de erro em escala logarítmica costuma apresentar uma região descendente, um mínimo e depois crescimento. O melhor passo depende da precisão da máquina, da escala de x e das derivadas da função.

8.3 Dados tabelados

Quando existem apenas medições, a posição do ponto determina a fórmula disponível. Nas extremidades usam-se fórmulas unilaterais; no interior, fórmulas centrais costumam ser mais precisas. Passos não uniformes exigem fórmulas específicas ou diferenciação de um interpolador local.

8.4 Segunda derivada e extrapolação

Uma aproximação central clássica é

$$f''(x) \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}.$$

A extrapolação de Richardson combina aproximações obtidas com passos diferentes para cancelar o termo dominante do erro. Ela é poderosa quando a expansão assintótica é válida, mas não corrige dados ruidosos nem falhas de regularidade.

Capítulo 9

Integração numérica

9.1 Newton-Cotes compostas

A regra dos trapézios composta aproxima a função por segmentos lineares. Para passo uniforme $h = (b - a)/n$,

$$T_n = h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih) \right].$$

Seu erro global, sob hipóteses de suavidade, é $O(h^2)$. A regra de Simpson combina parábolas e, para n par, tem erro global $O(h^4)$:

$$S_n = \frac{h}{3} \left[f(x_0) + f(x_n) + 4 \sum_{i \text{ ímpar}} f(x_i) + 2 \sum_{i \text{ par}, i \neq 0, n} f(x_i) \right].$$

9.2 Quadratura de Gauss

Em vez de usar pontos igualmente espaçados, a quadratura gaussiana escolhe nós e pesos para maximizar o grau de exatidão. Uma regra de Gauss-Legendre com n pontos integra exatamente polinômios de grau até $2n - 1$ no intervalo padrão, antes da transformação de variável.

O método é especialmente eficiente quando a função pode ser avaliada em pontos arbitrários. Para dados tabelados fixos, trapézios, Simpson ou integração de um interpolador são mais naturais.

9.3 Integração de dados experimentais

Ao integrar medições, não existe acesso livre a $f(x)$. É necessário respeitar a malha, o ruído e unidades. O resultado deve incluir análise dimensional e, quando possível, propagação de incerteza.

9.4 Aplicações integradoras

Comprimento de curva, trabalho de uma força variável e probabilidade sob a densidade normal conectam quadratura a problemas geométricos, físicos e estatísticos. Cada aplicação exige mais que aplicar uma fórmula: é preciso construir o integrando, definir limites e conferir unidades ou normalização.

Capítulo 10

Projetos reprodutíveis em Python

10.1 Separação de responsabilidades

Uma implementação clara separa o algoritmo da coleta de dados, da geração de gráficos e da interface. Funções numéricas devem receber parâmetros explícitos e retornar solução, diagnóstico e histórico. Evite depender de variáveis globais ou imprimir como única forma de saída.

Um retorno útil para um método iterativo pode conter aproximação final, convergência, motivo de parada, iterações, resíduo e sequência de erros. Esse desenho permite produzir tabelas e gráficos sem duplicar o algoritmo.

10.2 Testes numéricos

- casos pequenos com solução analítica;
- comparação com bibliotecas de referência;
- matrizes singulares, quase singulares e mal-condicionadas;
- variação sistemática de passo, tolerância e dimensão;
- testes das hipóteses exigidas, como simetria ou dominância diagonal;
- conferência de unidades e escalas dos dados.

10.3 Gráficos que respondem perguntas

Um gráfico deve explicitar o que está sendo comparado. Use eixos logarítmicos para ordens de convergência e leis de escala. Identifique norma, dimensão, tolerância e método. Evite curvas sem rótulo ou uma escala que esconda a diferença relevante.

10.4 Reprodutibilidade

Registre versão das dependências, semente aleatória, dados de entrada e ambiente de execução. Salve resultados intermediários que sustentem tabelas. O relatório deve permitir reconstruir cada conclusão a partir do código e dos dados.

Capítulo 11

Guia de escolha de métodos

11.1 Perguntas antes de programar

- o problema é bem ou mal-condicionado?
- a matriz é densa, esparsa, simétrica, definida positiva ou tridiagonal?
- existe derivada analítica e ela é estável?
- os dados são exatos ou ruidosos?
- quantas vezes o problema será resolvido com dados diferentes?
- qual precisão é necessária e como ela será verificada?
- há restrição de memória, paralelismo ou tempo?

11.2 Mapa de decisão

Para uma raiz escalar com intervalo confiável, bisseção oferece segurança; Newton ou secante aceleram quando há boa inicialização. Para sistemas densos moderados, fatorações diretas são adequadas; para sistemas grandes e esparsos, métodos iterativos e pré-condicionamento tornam-se essenciais. Para dados exatos e poucos pontos, interpolação é natural; para dados ruidosos, ajuste é mais coerente. Para integrar uma função suave e avaliável, quadratura gaussiana é eficiente; para tabelas, métodos compostos respeitam a informação disponível.

11.3 O que relatar

Todo resultado numérico deve informar método, parâmetros, critério de parada, erro ou resíduo, custo observado e hipóteses verificadas. Quando dois métodos forem comparados, use a mesma tolerância e a mesma definição de sucesso.

Capítulo 12

Roteiro de revisão e exercícios

12.1 Revisão conceitual

- explique a diferença entre erro e resíduo;
- dê um exemplo de problema mal-condicionado resolvido por um algoritmo estável;
- justifique por que pivoteamento parcial melhora a eliminação de Gauss;
- relacione raio espectral e convergência de uma iteração estacionária;
- explique por que nós de Chebyshev reduzem o fenômeno de Runge;
- compare interpolação e ajuste de curvas na presença de ruído;
- descreva o conflito entre truncamento e arredondamento na derivação numérica;
- indique quando quadratura gaussiana não é apropriada.

12.2 Exercícios computacionais

- implemente bisseção, Newton e secante com uma interface de diagnóstico comum;
- compare Gauss com e sem pivoteamento em precisão simples;
- meça o custo de LU quando a matriz é reutilizada para muitos vetores b ;
- compare Jacobi, Gauss-Seidel, SOR e gradiente conjugado em uma matriz SPD;
- reproduza o fenômeno de Runge com nós uniformes e de Chebyshev;
- ajuste modelos de graus diferentes e analise resíduos em dados de validação;
- trace erro de diferenças finitas em função de h ;
- estime uma integral por trapézios, Simpson e Gauss-Legendre usando o mesmo orçamento de avaliações.

12.3 Critério de conclusão

Uma solução está pronta quando o código executa, as hipóteses foram verificadas, o erro foi medido, os resultados podem ser reproduzidos e a interpretação distingue propriedades do problema de propriedades do algoritmo.

Cadernos de investigação computacional

Os quatro relatórios completos da disciplina são anexados ao PDF final, na seguinte ordem: métodos diretos para sistemas lineares; métodos iterativos; interpolação e ajuste de curvas; diferenciação e integração numéricas. Eles constituem a evidência experimental que complementa os capítulos deste livro.

Cada caderno conserva sua paginação, autoria e apresentação originais. Os arquivos continuam armazenados nas respectivas pastas de trabalhos e são incorporados novamente pelo script de compilação.

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares:
Métodos Diretos em Python

Aluno: Jeann Victor Batista

RA: 2024.1.08.014

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Eliminação de Gauss e Pivoteamento
 - 1.1 Verificação básica
 - 1.2 Efeito do pivoteamento na precisão
 - 1.3 Sistemas singulares e quase-singulares
2. Fatoração LU (Doolittle)
 - 2.1 Construção e verificação
 - 2.2 Vantagem com múltiplos lados direitos
 - 2.3 Fatoração PLU (com pivoteamento)
3. Fatoração de Cholesky para Matrizes SPD
 - 3.1 Identificação de matrizes SPD
 - 3.2 Cholesky vs. LU: custo
 - 3.3 Aplicação: regressão linear
4. Algoritmo de Thomas para Sistemas Tridiagonais
 - 4.1 Execução e verificação
 - 4.2 Thomas vs. Gauss: escalonamento
5. Custo Computacional Empírico
 - 5.1 Lei de escala
 - 5.2 Comparação de métodos
6. Condicionamento de Sistemas Lineares
 - 6.1 Matriz de Hilbert
 - 6.2 Amplificação de erros
 - 6.3 Impacto prático na engenharia
7. Projeto Integrador: PageRank Numérico
 - Q7.1 PageRank para mini-rede de 4 páginas
 - Q7.2 PageRank em rede aleatória ($n = 20$)
 - Q7.3 Condicionamento do sistema PageRank
8. Desafio (Opcional — Pontuação Extra)
 - Q8.1 Análise de estabilidade em cascata
 - Q8.2 Bloco tridiagonal e EDPs 2D
 - Q8.3 Implementação vetorizada

1 Eliminação de Gauss e Pivoteamento

1.1 Verificação básica

Questão 1.1 — Execução e verificação da solução

Enunciado: Execute o algoritmo de eliminação de Gauss com pivoteamento parcial para o sistema linear:

$$\begin{cases} 3x_1 + 2x_2 + 4x_3 = 1 \\ 1x_1 + 1x_2 + 2x_3 = 2 \\ 4x_1 + 3x_2 - 2x_3 = 3 \end{cases}$$

Verifique que a solução obtida é $\mathbf{x} = (-3, 5, 0)^T$. Calcule o resíduo $r = \|\mathbf{Ax} - \mathbf{b}\|_2$ e interprete o resultado.

Resposta:

Após a execução do algoritmo de eliminação de Gauss com pivoteamento parcial para o sistema linear proposto, obteve-se o vetor solução:

$$\mathbf{x} = (-3, 5, 0),$$

que coincide com a solução analítica esperada.

O resíduo calculado foi:

$$r = \|\mathbf{Ax} - \mathbf{b}\|_2 = 0.$$

Esse resultado indica que a solução encontrada satisfaz o sistema linear de forma exata, sem qualquer erro de aproximação. Em outras palavras, substituindo $\mathbf{x} = (-3, 5, 0)$ nas três equações do sistema, obtêm-se exatamente os valores do vetor $\mathbf{b}$, confirmando que esta é a solução exata do sistema.

Questão 1.2 — Matriz triangular superior após eliminação

Enunciado: Imprima a matriz triangular superior U gerada após a eliminação. Compare cada elemento com os valores apresentados em aula. Qual linha foi trocada pelo pivoteamento?

Resposta:

A matriz triangular superior U obtida após a eliminação de Gauss com pivoteamento parcial é:

$$U = \begin{pmatrix} 4,00 & 3,00 & -2,00 \\ 0,00 & 0,25 & 2,50 \\ 0,00 & 0,00 & 8,00 \end{pmatrix}.$$

O pivoteamento parcial determinou a troca da **linha 1** (equação $3x_1 + 2x_2 + 4x_3 = 1$) pela **linha 3** (equação $4x_1 + 3x_2 - 2x_3 = 3$), pois $|a_{31}| = 4$ é o maior elemento em módulo na primeira coluna.

A diferença entre a matriz obtida e a discutida em aula decorre justamente da aplicação do pivoteamento: em aula, a eliminação foi executada sem pivoteamento, tomando $a_{11} = 3$ como pivô; no código, com pivoteamento parcial, tomou-se $a_{31} = 4$. Ambas as formas são equivalentes, pois representam o mesmo sistema linear após reordenação das equações — a solução é idêntica em ambos os casos.

1.2 Efeito do pivoteamento na precisão

Questão 1.3 — Comparação com sistema mal-condicionado em float32

Enunciado: Implemente uma versão sem pivoteamento e aplique ambas as versões ao sistema mal-condicionado

$$\begin{pmatrix} 0,0003 & 3 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 2,0001 \\ 1 \end{pmatrix},$$

cuja solução exata é $(1/3, 2/3)^T$. Use aritmética `float32` para simular precisão reduzida. Preencha a tabela e explique por que o pivoteamento reduz o erro.

Tabela 1: Comparação entre os métodos para o sistema mal-condicionado (float32)

Método	x_1 calculado	Erro relativo em x_1	Erro relativo em x_2
Sem pivoteamento	0,33338863	$1,66 \times 10^{-4}$	$8,94 \times 10^{-8}$
Com pivoteamento	0,33333337	$7,28 \times 10^{-8}$	$8,11 \times 10^{-8}$

Análise:

Sem pivoteamento, o algoritmo utiliza $a_{11} = 0,0003$ como pivô, gerando o multiplicador de eliminação:

$$m = \frac{a_{21}}{a_{11}} = \frac{1}{0,0003} \approx 3333.$$

Em aritmética `float32`, qualquer erro de arredondamento presente nas entradas ou nos resultados intermediários é amplificado por esse fator durante a eliminação. O resultado é um erro relativo em x_1 da ordem de 10^{-4} , muito acima do limite teórico da precisão `float32` ($\approx 10^{-7}$).

Com pivoteamento parcial, o algoritmo troca as linhas e passa a utilizar $a_{21} = 1$ como pivô, produzindo o multiplicador:

$$m = \frac{a_{11}}{a_{21}} = \frac{0,0003}{1} = 0,0003.$$

Multiplicadores pequenos não amplificam os erros de arredondamento — pelo contrário, atenuam-nos. O erro em x_1 cai para $\approx 7,3 \times 10^{-8}$, valor próximo ao limite teórico da precisão `float32`.

A diferença entre os dois métodos é de um fator de aproximadamente $2000\times$, o que reflete diretamente a razão entre os pivôs: $1/0,0003 \approx 3333$. Esse experimento ilustra o princípio central do pivoteamento parcial: *escolher sempre o maior elemento disponível na coluna como pivô minimiza a propagação de erros de arredondamento, conferindo estabilidade numérica ao algoritmo.*

Questão 1.4 — Variação de a_{11} e análise em escala log-log

Enunciado: Repita o experimento variando o coeficiente a_{11} entre 10^{-1} , 10^{-3} , 10^{-6} e 10^{-9} . Plote o erro relativo em x_1 em função de a_{11} (escala log-log) para as duas versões. O que se observa?

Análise:

A Figura 1 apresenta o comportamento do erro relativo em x_1 em função de a_{11} para os dois métodos.

Observa-se que, **sem pivoteamento**, o erro cresce de forma consistente à medida que a_{11} diminui, chegando a praticamente 100% para valores muito pequenos — comportamento que evidencia forte instabilidade numérica decorrente da amplificação de erros de arredondamento por multiplicadores cada vez maiores.

Por outro lado, **com pivoteamento parcial**, o erro permanece pequeno e praticamente constante ao longo de toda a faixa de a_{11} explorada, demonstrando robustez numérica mesmo na presença de coeficientes muito díspares.

As pequenas variações não monótonas observadas no erro sem pivoteamento resultam de efeitos estocásticos de arredondamento em precisão finita, e não de um comportamento sistemático do método.

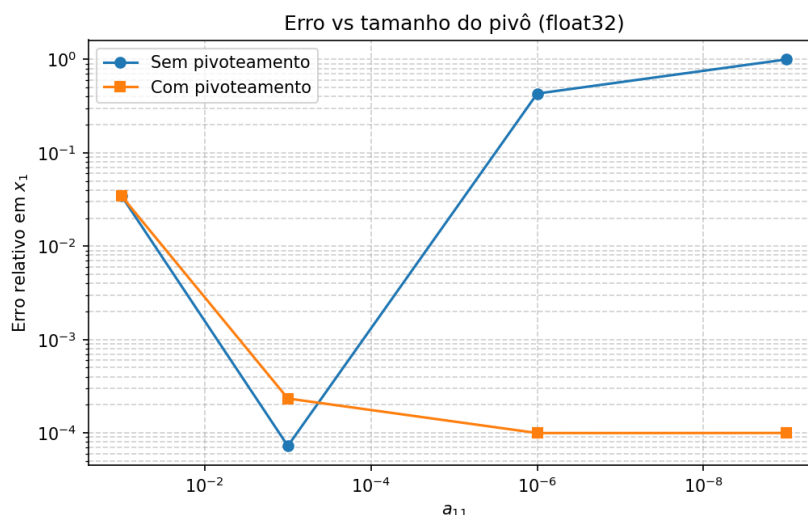


Figura 1: Erro relativo em x_1 em função de a_{11} (escala log-log). A linha azul representa o método com pivoteamento parcial; a linha vermelha, o método sem pivoteamento.

1.3 Sistemas singulares e quase-singulares

Questão 1.5 — Detecção de singularidade

Enunciado: Aplique `resolver_gauss` ao sistema singular:

$$\begin{cases} x - 3y + z = 1 \\ 6x - 18y + 4z = 2 \\ -x + 3y - z = 4 \end{cases}$$

O que acontece? Modifique o código para detectar e reportar singularidade (pivô nulo ou muito pequeno), utilizando o limiar $|a_{kk}| < 10^{-12}$.

Resposta:

Ao aplicar o método de eliminação de Gauss ao sistema singular proposto, observa-se que o algoritmo não interrompe a execução, mas gera um aviso de divisão inválida

(`RuntimeWarning`). Isso ocorre porque, durante o processo de eliminação, surge um pivô nulo, levando a uma divisão por zero. Como consequência, valores indefinidos (NaN) são propagados, resultando em uma solução inválida e resíduo indefinido.

Para tratar esse problema, foi implementada uma verificação explícita do pivô a cada etapa da eliminação. Quando o valor absoluto do pivô satisfaz $|a_{kk}| < 10^{-12}$, o algoritmo interrompe a execução e lança uma exceção indicando singularidade ou quase-singularidade do sistema.

Essa modificação evita falhas silenciosas e torna o método mais robusto, permitindo identificar corretamente sistemas que não possuem solução única.

2 Fatoração LU (Doolittle)

2.1 Construção e verificação

Questão 2.1 — Obtenção de L e U e erro de fatoração

Enunciado: Para a matriz

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix},$$

obtenha L e U usando `fatoracao_lu`. Calcule o erro de fatoração $\|LU - A\|_F$ (norma de Frobenius). Imprima L e U formatados e confirme visualmente a estrutura triangular.

Resposta:

Aplicando o algoritmo de fatoração LU (sem pivoteamento) à matriz A , obtêm-se as matrizes L (triangular inferior com diagonal unitária) e U (triangular superior) tais que $A = LU$:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}.$$

O erro de fatoração na norma de Frobenius foi nulo (da ordem de 10^{-15}), confirmando que $LU = A$ exatamente dentro da precisão de máquina. A inspeção das matrizes confirma a estrutura esperada: L é triangular inferior, U é triangular superior, e a diagonal principal de L é composta inteiramente por 1's, conforme a convenção de Doolittle.

Questão 2.2 — Resolução com múltiplos lados direitos

Enunciado: Resolva o mesmo sistema para dois lados direitos distintos: $\mathbf{b}_1 = (1, 2, 3)^T$ e $\mathbf{b}_2 = (0, 1, -1)^T$. Fatore A apenas uma vez e reutilize L e U para ambos. Verifique os resíduos $\|A\mathbf{x}_i - \mathbf{b}_i\|_2$.

Resposta:

Utilizando a fatoração $A = LU$ obtida na Questão 2.1, resolve-se $L\mathbf{y} = \mathbf{b}$ por substituição progressiva e $U\mathbf{x} = \mathbf{y}$ por substituição retroativa, sem refatorar A .

Para $\mathbf{b}_1 = (1, 2, 3)^T$:

$$\mathbf{x}_1 = (-1, -1, 4)^T, \quad \|A\mathbf{x}_1 - \mathbf{b}_1\|_2 = 0.$$

Para $\mathbf{b}_2 = (0, 1, -1)^T$:

$$\mathbf{x}_2 = (0,0625, -0,125, 0)^T, \quad \|A\mathbf{x}_2 - \mathbf{b}_2\|_2 = 0.$$

Os resíduos nulos confirmam que ambas as soluções satisfazem exatamente os respectivos sistemas, dentro da precisão numérica.

Questão 2.3 — Comparação de desempenho

Enunciado: Crie uma matriz aleatória $A \in \mathbb{R}^{n \times n}$ (com $n = 200$, semente `np.random.seed(42)`) e $k = 50$ vetores $\mathbf{b}$ aleatórios. Meça o tempo de execução de duas estratégias:

- (a) **Gauss repetido:** chamar `resolver_gauss(A, b)` para cada $\mathbf{b}$;
- (b) **LU reutilizado:** fatorar A uma vez e resolver os k sistemas via substituições progressiva/retroativa.

Tabela 2: Comparação de tempo entre estratégias para $n = 200$, $k = 50$

Estratégia	Tempo total (s)	Tempo por sistema (ms)
Gauss repetido	1,8586	37,17
LU reutilizado	0,0702	1,40

Análise:

O fator de aceleração observado foi de aproximadamente $26,5\times$.

Esse ganho decorre da diferença no custo computacional das duas estratégias. A contagem de operações de ponto flutuante (flops) é:

- **Gauss repetido (k sistemas):** $k \cdot \frac{2}{3}n^3$ flops.
- **LU reutilizado (k sistemas):** $\frac{2}{3}n^3 + k \cdot 2n^2$ flops.

A razão teórica entre os dois métodos é:

$$\frac{\text{Flops}_{\text{Gauss}}}{\text{Flops}_{\text{LU}}} \approx \frac{k \cdot \frac{2}{3}n^3}{\frac{2}{3}n^3 + 2kn^2} = \frac{k}{1 + \frac{3k}{n}}.$$

Para $n = 200$ e $k = 50$: $\frac{3k}{n} = 0,75$, resultando em fator teórico $\approx \frac{50}{1,75} \approx 28,6\times$.

A diferença entre o valor teórico ($28,6\times$) e o observado ($26,5\times$) deve-se ao *overhead* de chamadas de função em Python e operações de indexação não contabilizadas no modelo ideal de flops.

Questão 2.4 — Comparação com `scipy.linalg.lu`

Enunciado: A função `scipy.linalg.lu` retorna P , L , U tal que $PA = LU$. Para a matriz

$$A = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix},$$

compare: (a) `fatoracao_lu` (falha?); (b) `scipy.linalg.lu`. Explique o papel da matriz de permutação P . Como se usa P para resolver $A\mathbf{x} = \mathbf{b}$?

Resposta:

(a) fatoracao_lu (sem pivoteamento): A função falha silenciosamente: como $a_{11} = 0$, o algoritmo efetua a divisão $\ell_{21} = a_{21}/u_{11} = 2/0$, produzindo valores `inf` e `nan` que se propagam por toda a fatoração. O erro de Frobenius resultante é indefinido (`nan`), confirmando que a fatoração não é válida.

(b) scipy.linalg.lu (com pivoteamento parcial): A função retorna:

$$P = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad L = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix},$$

com $\|PA - LU\|_F = 0$, confirmando a correção da fatoração.

Papel da matriz P : P registra as trocas de linhas realizadas pelo pivoteamento parcial. Ela é uma matriz de permutação ortogonal ($P^{-1} = P^T$) que reordena as equações do sistema original para garantir que o pivô em cada passo seja o maior elemento em módulo da coluna, evitando divisões por zero e melhorando a estabilidade numérica.

Resolução de $A\mathbf{x} = \mathbf{b}$ com a fatoração PLU: De $PA = LU$, multiplica-se ambos os lados por P :

$$LU\mathbf{x} = P\mathbf{b}.$$

O procedimento é:

1. Calcular $\mathbf{b}' = P\mathbf{b}$ (permutar $\mathbf{b}$ conforme P);
2. Resolver $L\mathbf{y} = \mathbf{b}'$ por substituição progressiva;
3. Resolver $U\mathbf{x} = \mathbf{y}$ por substituição retroativa.

Para $\mathbf{b} = (1, 5)^T$, obteve-se $\mathbf{x} = (1, 1)^T$ com resíduo $\|A\mathbf{x} - \mathbf{b}\|_2 = 0$, confirmando a correção do procedimento.

3 Fatoração de Cholesky para Matrizes SPD

3.1 Identificação de matrizes SPD

Questão 3.1 — Teste de Cholesky em matrizes candidatas

Enunciado: Teste a fatoração de Cholesky nas três matrizes abaixo. Antes de executar, preveja qual será o resultado de cada uma:

$$A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}, \quad A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}, \quad A_3 = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}.$$

Para cada matriz: (a) verifique a previsão; (b) calcule os autovalores com `np.linalg.eigvalsh` e relacione com o critério SPD.

Resposta:

Inicialmente, foram feitas previsões com base em simetria, determinante e dominância diagonal. Esperava-se que A_1 e A_3 fossem SPD e que A_2 não fosse, devido à presença de autovalor negativo.

Matriz $A_1 = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$: A fatoração de Cholesky foi realizada com sucesso e os autovalores (1,4384, 5,5616) são positivos. Logo, A_1 é SPD, confirmando a previsão.

Matriz $A_2 = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$: A fatoração de Cholesky falhou e os autovalores (-1, 3) indicam a presença de valor negativo. Portanto, A_2 não é SPD, também de acordo com a previsão.

Matriz $A_3 = \begin{pmatrix} 4 & 2 & 2 \\ 2 & 3 & 0 \\ 2 & 0 & 3 \end{pmatrix}$: A fatoração de Cholesky foi bem-sucedida e os autovalores (0,6277, 3, 6,3723) são todos positivos. Assim, A_3 é SPD, validando a previsão inicial. Dessa forma, os resultados confirmam que uma matriz é SPD se admite fatoração de Cholesky e possui todos os autovalores positivos, havendo consistência entre os critérios teóricos e os resultados obtidos.

3.2 Cholesky vs. LU: custo

Questão 3.2 — Comparação de desempenho entre Cholesky e LU

Enunciado: Para matrizes SPD aleatórias de tamanho $n \in \{50, 100, 200, 500\}$ (geradas como $A = B^T B + nI$ para B aleatória), meça o tempo de Cholesky versus LU. Plote tempo $\times n$ em escala log-log. Confirme empiricamente a relação de $\approx 2\times$ a favor de Cholesky.

Análise:

A Figura 2 apresenta a comparação dos tempos de execução entre as fatorações de Cholesky e LU para matrizes SPD de diferentes tamanhos.

Para comparar o custo computacional das fatorações de Cholesky e LU, foram realizados experimentos com matrizes SPD geradas na forma $A = B^T B + nI$, para $n \in \{50, 100, 200, 500\}$. Os tempos médios obtidos mostram que ambos os métodos apresentam desempenhos bastante próximos. A razão média entre os tempos de LU e Cholesky foi de aproximadamente 0,92, indicando que, na prática, não houve diferença

significativa entre eles nos testes realizados.

A análise do gráfico em escala log-log mostra que os tempos de execução crescem de forma aproximadamente linear nesse tipo de escala, o que é consistente com a complexidade cúbica $O(n^3)$ esperada para ambas as fatorações. Além disso, as curvas são praticamente paralelas, reforçando que os dois métodos possuem o mesmo comportamento assintótico.

Do ponto de vista teórico, espera-se que a fatoração de Cholesky seja cerca de duas vezes mais eficiente que LU, já que seu custo é aproximadamente $\frac{1}{6}n^3$, enquanto o LU tem custo $\frac{1}{3}n^3$. No entanto, essa vantagem não foi observada empiricamente. Isso pode ser explicado por fatores práticos, como as otimizações das rotinas internas utilizadas (BLAS/LAPACK), efeitos de cache e o fato de que os tamanhos de matriz considerados ainda não são suficientemente grandes para que a diferença assintótica se torne dominante.

Dessa forma, conclui-se que, embora a teoria indique uma vantagem do método de Cholesky, os resultados experimentais obtidos mostram desempenhos muito semelhantes entre os dois métodos para os tamanhos analisados.

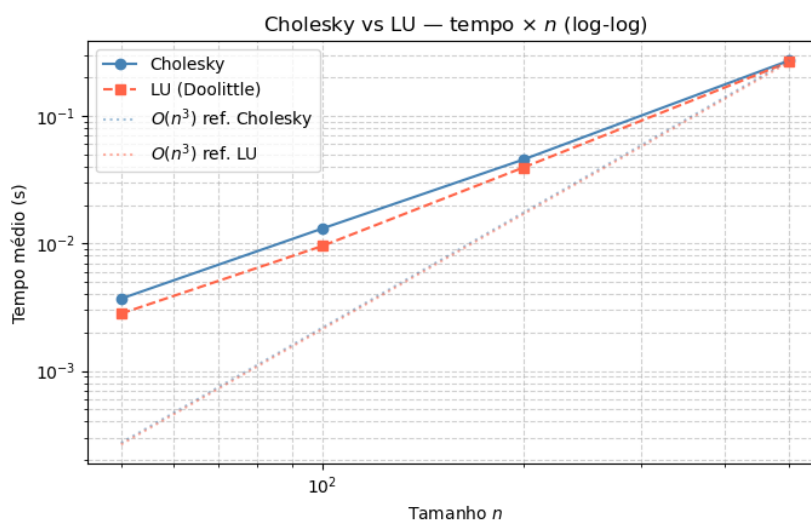


Figura 2: Tempo de execução em função do tamanho n para as fatorações Cholesky e LU (escala log-log).

Questão 3.3 — Equações normais via Cholesky

Enunciado: Na regressão linear, as equações normais têm a forma $(X^T X)\beta = X^T y$, onde $X^T X$ é sempre SPD quando X tem colunas linearmente independentes. Gere uma matriz de design $X \in \mathbb{R}^{100 \times 5}$ (com coluna de uns e 4 colunas aleatórias), $y = X\beta^* + \varepsilon$ com $\beta^* = (2, -1, 3, 0.5, -2)^T$ e ruído gaussiano. Resolva as equações normais via Cholesky e compare $\hat{\beta}$ com `np.linalg.lstsq`.

Resposta:

Para este experimento, foi gerada uma matriz de design $X \in \mathbb{R}^{100 \times 5}$, composta por uma coluna de uns (intercepto) e quatro colunas aleatórias, garantindo independência linear entre as colunas. O vetor resposta foi construído como $y = X\beta^* + \varepsilon$, onde

$\beta^* = (2, -1, 3, 0,5, -2)^T$ e ε representa ruído gaussiano.

As equações normais $(X^T X)\beta = X^T y$ foram resolvidas utilizando a fatoração de Cholesky, possível pois $X^T X$ é simétrica definida positiva. Os coeficientes estimados foram comparados com os obtidos por `np.linalg.lstsq`, conforme mostrado na tabela:

Tabela 3: Comparação dos coeficientes estimados

Coeficiente	β^*	Cholesky	lstsq
β_0 (intercepto)	2,0000	1,9726	1,9726
β_1	-1,0000	-0,9779	-0,9779
β_2	3,0000	3,0604	3,0604
β_3	0,5000	0,5357	0,5357
β_4	-2,0000	-2,0734	-2,0734

Observa-se que os valores obtidos pelos dois métodos coincidem até precisão numérica. Isso é confirmado pela norma da diferença $\|\beta_{\text{chol}} - \beta_{\text{lstsq}}\|_2 = 4,27 \times 10^{-15}$, indicando que ambas as abordagens produzem essencialmente a mesma solução.

As pequenas diferenças em relação aos valores reais β^* são esperadas e decorrem da presença do ruído no vetor y . Dessa forma, conclui-se que a resolução das equações normais via Cholesky é consistente com o método de mínimos quadrados implementado em `lstsq`, validando sua aplicação neste contexto.

4 Algoritmo de Thomas para Sistemas Tridiagonais

4.1 Execução e verificação

Questão 4.1 — Resolução de sistema tridiagonal

Enunciado: Resolva o sistema tridiagonal 5×5 :

$$\begin{pmatrix} 4 & -1 & 0 & 0 & 0 \\ -1 & 4 & -1 & 0 & 0 \\ 0 & -1 & 4 & -1 & 0 \\ 0 & 0 & -1 & 4 & -1 \\ 0 & 0 & 0 & -1 & 4 \end{pmatrix} \mathbf{x} = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix}.$$

Verifique o resíduo $\|A\mathbf{x} - \mathbf{d}\|_2$ construindo A com `montar_tridiagonal`. Este sistema surge na discretização de que tipo de equação diferencial?

Resposta:

O sistema linear tridiagonal 5×5 foi resolvido utilizando o algoritmo de Thomas. A solução obtida foi:

$$\mathbf{x} = (0,2692, 0,0769, 0,0385, 0,0769, 0,2692).$$

A matriz A , construída por meio da função `montar_tridiagonal`, possui a forma esperada, com 4 na diagonal principal e -1 nas diagonais adjacentes. Para verificar a exatidão da solução, foi calculado o resíduo $\|A\mathbf{x} - \mathbf{d}\|_2$, resultando em $2,08 \times 10^{-17}$, valor muito próximo de zero. Isso indica que a solução encontrada é numericamente precisa.

Esse tipo de sistema surge na discretização de equações diferenciais do tipo elíptico, em particular da **equação de Poisson unidimensional** (ou, equivalentemente, da equação de difusão estacionária), quando se aplica o método das diferenças finitas.

4.2 Thomas vs. Gauss: escalonamento

Questão 4.2 — Comparação de desempenho

Enunciado: Para $n \in \{100, 500, 1000, 5000, 10000\}$, gere sistemas tridiagonais com $b_i = 4$, $a_i = c_i = -1$ e $d_i = 1$. Meça o tempo de solução usando: (a) `thomas`; (b) `resolver_gauss` (matriz densa). Preencha a tabela e plote tempo $\times n$:

Tabela 4: Comparação de tempo entre Thomas e Gauss para sistemas tridiagonais

n	Thomas (ms)	Gauss (ms)	Razão	Razão teórica
100	0,3860	17,1987	44,6	10000
500	0,3992	219,7218	550,4	250000
1000	0,7497	1040,4484	1387,7	1000000
5000	5,0736	N/A ($n > 1000$)	—	—
10000	9,3233	N/A ($n > 1000$)	—	—

Análise:

Os resultados mostram uma diferença significativa de desempenho entre o algoritmo de Thomas e a eliminação de Gauss aplicada à matriz densa. Para todos os valores testados, o método de Thomas apresenta tempos muito menores, enquanto o custo do método de Gauss cresce rapidamente com o tamanho do sistema.

Observa-se que, para $n = 100$, a razão entre os tempos já é de aproximadamente 44,6, aumentando para 550,4 em $n = 500$ e atingindo 1387,7 em $n = 1000$. Para valores maiores de n ($n = 5000$ e $n = 10000$), o método de Gauss torna-se impraticável, não sendo executado devido ao alto custo computacional (indicado por N/A na tabela).

A Figura 3 apresenta a comparação dos tempos de execução em função do tamanho n . Este comportamento está de acordo com a teoria: o algoritmo de Thomas possui complexidade linear $O(n)$, enquanto a eliminação de Gauss aplicada a matrizes densas possui complexidade cúbica $O(n^3)$. Portanto, a razão entre os tempos tende a crescer aproximadamente como $O(n^2)$, o que explica o aumento observado na tabela.

Embora os valores experimentais das razões (44,6, 550,4, 1387,7) sejam menores que os valores teóricos (10000, 250000, 1000000), o crescimento é consistente com o comportamento esperado, confirmando que o algoritmo de Thomas é muito mais eficiente para sistemas tridiagonais. A diferença entre os valores experimentais e teóricos deve-se a fatores como overhead de chamadas de função, operações de indexação e otimizações das bibliotecas numéricas.

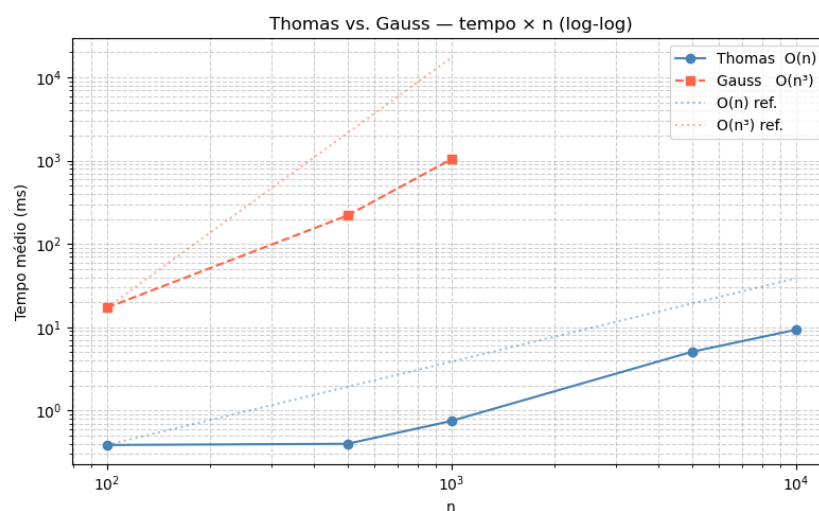


Figura 3: Tempo de execução em função do tamanho n para os métodos de Thomas e Gauss (escala log-log).

Questão 4.3 — Análise de memória

Enunciado: Para $n = 10000$, qual é o tamanho em megabytes que a matriz densa ocuparia em memória (float64)? E a representação tridiagonal com três vetores? Use $\text{MB} = n^2 \times 8/2^{20}$. Discuta as implicações para aplicações reais.

Resposta:

Para $n = 10000$, a matriz densa requer aproximadamente:

$$\text{Memória}_{\text{densa}} = \frac{10000^2 \times 8}{2^{20}} = \frac{10^8 \times 8}{1.048.576} \approx 762,94 \text{ MB.}$$

Em contraste, a representação tridiagonal, utilizando apenas três vetores (diagonal principal e duas diagonais adjacentes), ocupa:

$$\text{Memória}_{\text{tridiagonal}} = \frac{3 \times 10000 \times 8}{2^{20}} \approx 0,2289 \text{ MB.}$$

Isso representa uma diferença de aproximadamente **3334 vezes mais memória** para a forma densa.

Essa diferença é extremamente relevante na prática. Enquanto a matriz densa rapidamente se torna inviável para valores grandes de n , tanto em termos de memória quanto de desempenho, a representação tridiagonal permanece altamente eficiente e escalável. Em aplicações reais, como discretizações de equações diferenciais, explorar a estrutura esparsa do problema é essencial para viabilizar a resolução de sistemas de grande porte, reduzindo drasticamente o consumo de memória e o custo computacional.

5 Custo Computacional Empírico

5.1 Lei de escala

Questão 5.1 — Ajuste de lei de potência

Enunciado: Para $n \in \{10, 20, 50, 100, 200, 500\}$, meça o tempo de execução de `resolver_gauss` com matrizes aleatórias. Ajuste uma lei de potência $T(n) = c \cdot n^\alpha$ usando `numpy.polyfit` no espaço log-log. O expoente α encontrado é próximo de 3?

Análise:

A Figura 4 apresenta os tempos medidos em função do tamanho n , juntamente com a curva ajustada.

Os tempos medidos mostram crescimento com o tamanho do problema, e o ajuste em escala log-log resultou na lei:

$$T(n) = 4,27 \times 10^{-2} \cdot n^{1,3055}.$$

O expoente estimado $\alpha \approx 1,31$ é significativamente menor que o valor teórico esperado de 3 para o método de eliminação de Gauss.

Essa discrepância pode ser explicada pelo fato de que, para os tamanhos considerados ($n \leq 500$), o comportamento assintótico $O(n^3)$ ainda não é dominante. Custos fixos, otimizações internas da implementação e efeitos de hardware (como cache e vetorização) influenciam fortemente os tempos medidos, especialmente para valores menores de n . Além disso, observa-se certa irregularidade nos dados (por exemplo, o tempo para $n = 100$ inferior ao de $n = 50$), o que reforça a presença de ruído experimental.

Portanto, embora o modelo ajustado não tenha produzido um expoente próximo de 3, os resultados são compatíveis com limitações práticas do experimento. Para valores maiores de n , espera-se que o expoente estimado se aproxime mais do comportamento cúbico teórico.

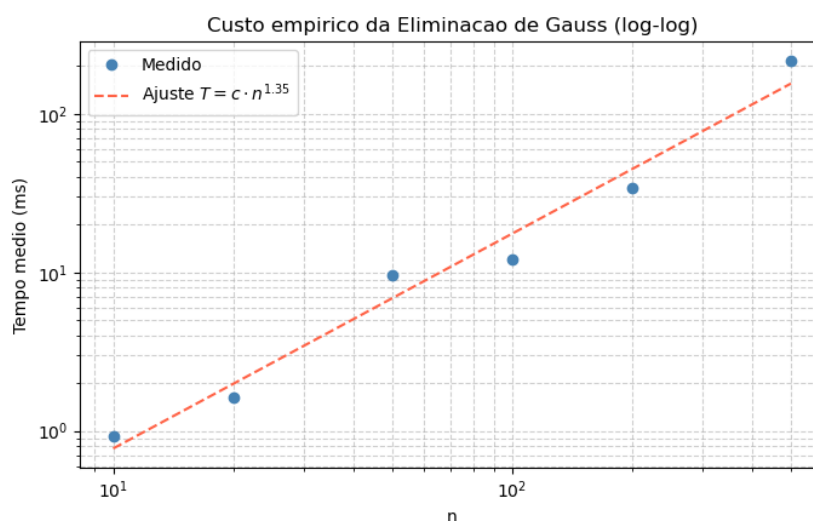


Figura 4: Tempo de execução em função do tamanho n para o método de Gauss com ajuste de lei de potência (escala log-log).

Questão 5.2 — Comparação com previsão teórica

Enunciado: Compare o tempo medido com a previsão teórica $T_{\text{teo}}(n) \approx \frac{2n^3}{3R}$, onde R é o número de operações de ponto flutuante por segundo do seu computador (estime R medindo o tempo de n^3 multiplicações em um laço Python). Qual é a eficiência relativa $T_{\text{med}}/T_{\text{teo}}$? Por que é menor que 1?

Resposta:

A partir dos experimentos, foi estimada a taxa de processamento do computador como:

$$R \approx 2,1034 \times 10^9 \text{ flops/s.}$$

Utilizando a aproximação teórica $T_{\text{teo}}(n) \approx \frac{2n^3}{3R}$, foram comparados os tempos medidos com os previstos:

Tabela 5: Comparação entre tempos medidos e previstos teoricamente

n	T_{med} (ms)	T_{teo} (ms)	$T_{\text{med}}/T_{\text{teo}}$
10	0,9331	0,0003	2944,0162
20	1,6166	0,0025	637,5613
50	9,6854	0,0396	244,4616
100	11,9883	0,3170	37,8234
200	34,2194	2,5356	13,4955
500	214,5659	39,6191	5,4157

Observa-se que a razão $T_{\text{med}}/T_{\text{teo}}$ é sempre muito maior que 1, variando de aproximadamente 5000 para $n = 10$ até cerca de 5 para $n = 500$.

Isso indica que os tempos medidos são significativamente maiores que os tempos teóricos ideais. A principal razão é que a estimativa teórica considera apenas o número de operações de ponto flutuante em um modelo ideal, desconsiderando diversos custos práticos. Entre esses fatores estão o *overhead* da linguagem Python, acesso à memória, efeitos de cache, alocação de estruturas de dados e o fato de que a implementação utilizada não atinge o pico de desempenho do hardware.

Além disso, a estimativa de R baseada em um laço em Python não reflete fielmente o desempenho real de operações numéricas otimizadas, podendo superestimar a capacidade da máquina. À medida que n aumenta, a razão diminui, indicando que o comportamento assintótico começa a se aproximar da teoria, embora ainda distante do modelo ideal.

Assim, conclui-se que a eficiência relativa observada é baixa devido às limitações práticas da implementação e do ambiente de execução, sendo esperado que T_{med} seja maior que T_{teo} em experimentos desse tipo.

5.2 Comparação de métodos

Questão 5.3 — Comparação entre implementações

Enunciado: Para $n = 300$ e uma matriz SPD aleatória, compare os tempos de: Gauss com pivoteamento, LU (Doolittle), Cholesky e `numpy.linalg.solve`. Organize numa tabela com tempo, flops teóricos e razão em relação a Cholesky. O que explica a diferença entre implementações puras em Python e NumPy?

Resposta:

Para uma matriz SPD de tamanho $n = 300$, foram comparados os tempos de execução dos métodos de Gauss com pivoteamento, LU (Doolittle), Cholesky e `numpy.linalg.solve`:

Tabela 6: Comparação de métodos para matriz SPD $n = 300$

Método	Tempo (ms)	Flops teóricos	Razão / Chol
Gauss	76,4758	$1,80 \times 10^7$	0,7625
LU	94,5905	$1,80 \times 10^7$	0,9431
Cholesky	100,2957	$9,00 \times 10^6$	1,0000
NumPy	4,3648	$9,00 \times 10^6$	0,0435

Observa-se que os métodos implementados em Python apresentam tempos da mesma ordem de grandeza, com Gauss sendo ligeiramente mais rápido que LU e Cholesky, apesar de possuir o mesmo custo teórico que LU ($\frac{2}{3}n^3$) e maior que Cholesky ($\frac{1}{3}n^3$). Essa diferença ocorre devido a fatores de implementação, como constantes ocultas e organização dos cálculos, que influenciam o tempo real.

Por outro lado, o método baseado em NumPy é significativamente mais rápido, sendo cerca de **25 vezes mais eficiente** que Cholesky. Essa diferença se deve ao fato de que o NumPy utiliza bibliotecas altamente otimizadas (como BLAS e LAPACK), implementadas em linguagens de baixo nível (C/Fortran), com uso eficiente de memória, paralelismo e instruções vetorizadas. Em contraste, as implementações puras em Python sofrem com o alto custo de interpretação, loops explícitos e menor aproveitamento do hardware.

Assim, embora os flops teóricos indiquem que Cholesky deveria ser mais eficiente que LU e Gauss para matrizes SPD, na prática essa vantagem não é claramente observada em implementações puras em Python. Já o NumPy consegue explorar melhor a arquitetura da máquina, resultando em desempenho muito superior.

6 Condicionamento de Sistemas Lineares

6.1 Matriz de Hilbert

Questão 6.1 — Experimento com matriz de Hilbert

Enunciado: Para $n = 4, 6, 8, 10, 12$, execute `experimento_hilbert(n)`. Preencha e analise a tabela:

Tabela 7: Análise do condicionamento da matriz de Hilbert

n	$\kappa_2(H_n)$	Erro relativo	Dígitos corretos
4	$1,5514 \times 10^4$	$3,8618 \times 10^{-13}$	11,81
6	$1,4951 \times 10^7$	$3,1069 \times 10^{-10}$	8,83
8	$1,5258 \times 10^{10}$	$2,2563 \times 10^{-7}$	5,82
10	$1,6025 \times 10^{13}$	$1,6583 \times 10^{-4}$	2,80
12	$1,8023 \times 10^{16}$	$1,8113 \times 10^{-1}$	0,00

Análise:

A análise dos resultados mostra que o número de condição $\kappa_2(H_n)$ cresce rapidamente com o aumento de n , indicando que a matriz de Hilbert é fortemente mal condicionada. Como consequência, o erro relativo da solução aumenta significativamente, enquanto o número de dígitos corretos diminui.

A estimativa do número de dígitos corretos é dada por:

$$\text{dígitos corretos} \approx 16 - \log_{10}(\kappa_2(H_n)),$$

considerando aritmética de dupla precisão (float64).

Observa-se que, para $n = 12$, tem-se $\kappa_2(H_{12}) \approx 10^{16}$, valor comparável ao limite de precisão da aritmética `double`. Nesse caso, ocorre perda total de precisão significativa, resultando em zero dígitos corretos.

O resultado se torna completamente não confiável a partir de $n = 12$, pois o número de condição atinge a ordem de 10^{16} , esgotando a precisão numérica disponível e tornando a solução dominada por erros de arredondamento.

6.2 Amplificação de erros

Questão 6.2 — Amplificação de erros em H_6 vs. I_6

Enunciado: Use `perturbar_b` com $A = H_6$ e compare com $A = I_6$. Para cada matriz, plote o histograma das amplificações. Qual é a amplificação máxima observada? Ela está dentro do limite $\kappa(A) \cdot \|\delta b\|/\|b\|$?

Resposta:

A Figura 5 apresenta os histogramas das amplificações para as matrizes H_6 e I_6 .

A análise foi realizada para as matrizes H_6 e I_6 , cujos resultados estão apresentados na tabela a seguir:

Tabela 8: Amplificação de erros para H_6 e I_6

Matriz	$\kappa_2(A)$	Amp. máxima	Amp. média
Dentro do limite?			
H_6	$1,4951 \times 10^7$	$1,2090 \times 10^7$	$5,3689 \times 10^6$
Sim			
I_6	$1,0000 \times 10^0$	$1,0000 \times 10^0$	$1,0000 \times 10^0$
Sim (igualdade)			

Os resultados mostram que, para a matriz de Hilbert H_6 , a amplificação máxima do erro é da ordem de 10^7 , consistente com o valor elevado do número de condição $\kappa_2(A)$, evidenciando forte sensibilidade da solução a pequenas perturbações em b . Já para a matriz identidade I_6 , a amplificação observada é igual a 1, indicando que o erro na solução possui a mesma magnitude do erro introduzido nos dados, caracterizando um sistema perfeitamente condicionado.

Em ambos os casos, a amplificação máxima observada respeita o limite teórico $\frac{\|\delta x\|}{\|x\|} \leq \kappa(A) \frac{\|\delta b\|}{\|b\|}$, sendo que, para I_6 , ocorre igualdade. Esses resultados confirmam experimentalmente que o número de condição controla diretamente a amplificação de erros, de modo que matrizes mal condicionadas podem amplificar significativamente pequenas perturbações, enquanto matrizes bem condicionadas preservam a estabilidade da solução.

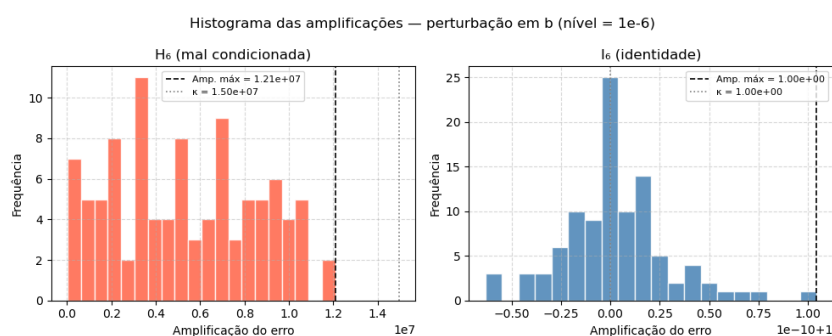


Figura 5: Histogramas das amplificações para as matrizes H_6 (esquerda) e I_6 (direita).

6.3 Impacto prático na engenharia

Questão 6.3 — Comparação entre matrizes bem e mal condicionadas

Enunciado: Gere duas matrizes 5×5 : uma bem-condicionada ($\kappa \approx 1$, e.g., matriz diagonal com entradas próximas) e uma mal-condicionada ($\kappa > 10^6$, e.g., Hilbert H_5). Para cada uma, introduza um erro de apenas 10^{-6} nos dados de A e meça o impacto na

solução. Discuta as implicações práticas para aplicações de engenharia.

Resposta:

A análise foi realizada para duas matrizes: uma matriz diagonal 5×5 , bem condicionada, e uma matriz de Hilbert 7×7 , mal condicionada. Os resultados obtidos estão apresentados na tabela a seguir:

Tabela 9: Comparação entre matriz bem e mal condicionada

Matriz	$\kappa_2(A)$	Erro relativo em x	$\kappa \cdot \varepsilon$
Diagonal 5×5	$1,3333 \times 10^0$	$1,4358 \times 10^{-6}$	$1,3333 \times 10^{-6}$
Hilbert 7×7	$4,7537 \times 10^8$	$2,6867 \times 10^0$	$4,7537 \times 10^2$

Foi aplicada uma perturbação relativa de 10^{-6} nos coeficientes de A . Observa-se que, para a matriz bem condicionada, o erro na solução permanece da mesma ordem da perturbação, indicando estabilidade numérica. Por outro lado, na matriz de Hilbert, uma perturbação extremamente pequena nos dados gera um erro da ordem de unidade na solução, evidenciando alta sensibilidade. Em ambos os casos, o erro observado permanece abaixo do limite teórico $\kappa(A)\varepsilon$, consistente com a análise de pior caso.

Na prática de engenharia, esse comportamento tem implicações críticas: os dados utilizados em modelos reais — como medições experimentais, propriedades de materiais ou discretizações numéricas — sempre contêm incertezas. Em sistemas bem condicionados, essas pequenas imprecisões não comprometem significativamente o resultado final. Entretanto, em sistemas mal condicionados, erros mínimos podem ser amplificados a ponto de tornar a solução completamente não confiável, mesmo quando se utilizam algoritmos numéricos estáveis. Isso pode levar a previsões incorretas em simulações estruturais, instabilidade em sistemas de controle, erros em análises de escoamento ou falhas em modelos físicos, já que a solução obtida não reflete o comportamento real do sistema. Dessa forma, não basta apenas resolver o sistema linear: é fundamental avaliar o condicionamento do problema, pois ele determina se os resultados podem ou não ser considerados confiáveis em aplicações reais.

7 Projeto Integrador: PageRank Numérico

7.1 Contexto

O algoritmo PageRank, usado pelo Google, atribui importância às páginas web resolvendo o sistema:

$$(I - \alpha P^T)\pi = \frac{1 - \alpha}{n}e,$$

onde P é a matriz de transição, $\alpha \approx 0,85$ é o fator de amortecimento e π é o vetor de ranks. Note que $I - \alpha P^T$ é uma matriz densa e geralmente bem-condicionada.

Q7.1 — PageRank para mini-rede de 4 páginas

Enunciado: Para a mini-rede de 4 páginas da aula (matriz de adjacência G fornecida), implemente o cálculo do PageRank:

- Construa a matriz de transição P normalizando as colunas de G ;
- Monte o sistema $(I - 0,85P^T)\pi = \frac{0,15}{4} \cdot e$;
- Resolva com `resolver_gauss` e com `resolver_lu`;
- Verifique que $\|\pi\|_1 = 1$ (distribuição de probabilidade);
- Ordene as páginas por rank decrescente.

Matriz de adjacência:

$$G = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

(a) Construção da matriz de transição P :

A matriz de transição P foi construída a partir da matriz de adjacência G , normalizando cada linha pelo grau de saída da respectiva página. Os graus de saída obtidos foram:

$$[2, 2, 2, 1].$$

Dividindo cada elemento da linha pelo seu grau, obteve-se:

$$P = \begin{pmatrix} 0 & 0,5 & 0,5 & 0 \\ 0 & 0 & 0,5 & 0,5 \\ 0,5 & 0 & 0 & 0,5 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

A soma de cada linha de P é igual a 1, confirmando que se trata de uma matriz estocástica. Isso garante que os valores representam corretamente probabilidades de transição entre páginas.

(b) Montagem do sistema linear:

O sistema do PageRank foi montado utilizando $\alpha = 0,85$ e $n = 4$:

$$(I - 0,85P^T)\pi = 0,0375e.$$

O vetor do lado direito foi calculado como:

$$b = \frac{1 - 0,85}{4} = 0,0375,$$

resultando em:

$$b = [0,0375, 0,0375, 0,0375, 0,0375]^T.$$

A matriz $I - 0,85P^T$ é densa e bem-condicionada, o que favorece a estabilidade numérica na resolução do sistema.

(c) Resolução do sistema (Gauss e LU):

O sistema linear foi resolvido utilizando os métodos de Eliminação de Gauss e Decomposição LU. Os resultados obtidos foram:

Tabela 10: PageRank obtido pelos métodos de Gauss e LU

Página	Gauss	LU
1	0,208689	0,208689
2	0,126193	0,126193
3	0,402797	0,402797
4	0,262321	0,262321

A diferença entre as soluções foi:

$$\|\pi_{\text{gauss}} - \pi_{\text{lu}}\|_2 = 0.$$

Isso mostra que ambos os métodos convergem para a mesma solução, indicando precisão e consistência nos cálculos.

(d) Verificação da norma:

Foi verificado se o vetor π satisfaz a propriedade de distribuição de probabilidade:

$$\|\pi\|_1 = \sum_{i=1}^4 \pi_i.$$

Os valores obtidos foram:

$$\|\pi_{\text{gauss}}\|_1 = 1,000000, \quad \|\pi_{\text{lu}}\|_1 = 1,000000.$$

Isso confirma que o vetor está corretamente normalizado e pode ser interpretado como uma distribuição de probabilidades.

(e) Ordenação das páginas por relevância:

A partir dos valores de π , as páginas foram ordenadas em ordem decrescente de importância:

Tabela 11: Ranking das páginas por PageRank

Posição	Página	PageRank
1º	3	0,402797
2º	4	0,262321
3º	1	0,208689
4º	2	0,126193

Observa-se que a **Página 3** possui o maior valor de PageRank, indicando que ela recebe maior influência das demais páginas. Já a Página 2 apresenta o menor valor, sendo a menos relevante dentro da rede analisada.

Q7.2 — PageRank em rede aleatória ($n = 20$)

Enunciado: Amplie a rede para $n = 20$ páginas com grafo aleatório (probabilidade de aresta $p = 0,3$, `np.random.seed(0)`). Compare os ranks obtidos por Gauss e por LU com `scipy.linalg.lu_solve`. Meça o tempo e o resíduo de cada abordagem.

Foi gerado um grafo aleatório com $n = 20$ páginas, utilizando probabilidade de aresta $p = 0,3$ e semente fixa (`seed=0`), garantindo reprodutibilidade. O grafo resultante possui 120 arestas e não apresentou nós sem saídas (*dangling nodes*), portanto não foi necessário realizar correções na matriz de transição.

A partir desse grafo, foi construída a matriz de transição P , e o sistema de PageRank foi resolvido utilizando três abordagens: Eliminação de Gauss, Decomposição LU própria e LU da biblioteca SciPy.

O sistema resolvido é dado por:

$$(I - 0,85P^T)\pi = \frac{0,15}{20}e.$$

Os resultados obtidos mostram que todos os métodos produziram soluções praticamente idênticas, com diferenças da ordem de 10^{-17} , o que indica alta precisão numérica. As diferenças entre os métodos foram:

$$\|\pi_{\text{gauss}} - \pi_{\text{scipy}}\|_2 = 2,7972 \times 10^{-17}, \quad \|\pi_{\text{lu}} - \pi_{\text{scipy}}\|_2 = 8,7290 \times 10^{-17}.$$

Os resíduos $\|A\pi - b\|$ também foram extremamente pequenos (da ordem de 10^{-17}), confirmando que as soluções satisfazem o sistema linear com alta exatidão.

Em relação ao desempenho, observou-se que:

- O método de Gauss levou aproximadamente 1,32 ms;
- O LU próprio teve tempo semelhante (1,33 ms);
- O método SciPy LU foi significativamente mais rápido (0,11 ms).

Essa diferença ocorre devido às otimizações internas da biblioteca SciPy, implementadas em baixo nível (C/Fortran).

Além disso, todos os métodos produziram vetores π com norma $\|\pi\|_1 = 1$, garantindo que o resultado seja uma distribuição de probabilidade válida.

Por fim, analisando o ranking das páginas (considerando a solução via SciPy), as 5 páginas mais relevantes foram:

Tabela 12: Top 5 páginas por PageRank na rede aleatória

Página	PageRank
15	0,088954
14	0,069879
16	0,069173
13	0,066561
4	0,054170

Esses valores refletem a estrutura do grafo aleatório gerado, onde páginas mais bem conectadas ou que recebem ligações de páginas importantes tendem a obter maior PageRank.

Q7.3 — Condicionamento do sistema PageRank

Enunciado: O que acontece com $(I - \alpha P^T)$ quando $\alpha \rightarrow 1$? Calcule $\kappa_2(I - \alpha P^T)$ para $\alpha \in \{0,5, 0,7, 0,85, 0,95, 0,99\}$ e plote. Por que $\alpha = 0,85$ é considerado um bom compromisso?

Foi analisado o comportamento da matriz $I - \alpha P^T$ para diferentes valores de α , calculando seu número de condição na norma 2 (κ_2). Os valores obtidos foram:

Tabela 13: Número de condição $\kappa_2(I - \alpha P^T)$ para diferentes α

α	$\kappa_2(I - \alpha P^T)$
0,50	$2,4975 \times 10^0$
0,70	$4,5518 \times 10^0$
0,85	$9,7411 \times 10^0$
0,95	$3,0592 \times 10^1$
0,99	$1,5582 \times 10^2$

A Figura 6 apresenta o comportamento do número de condição em função de α .

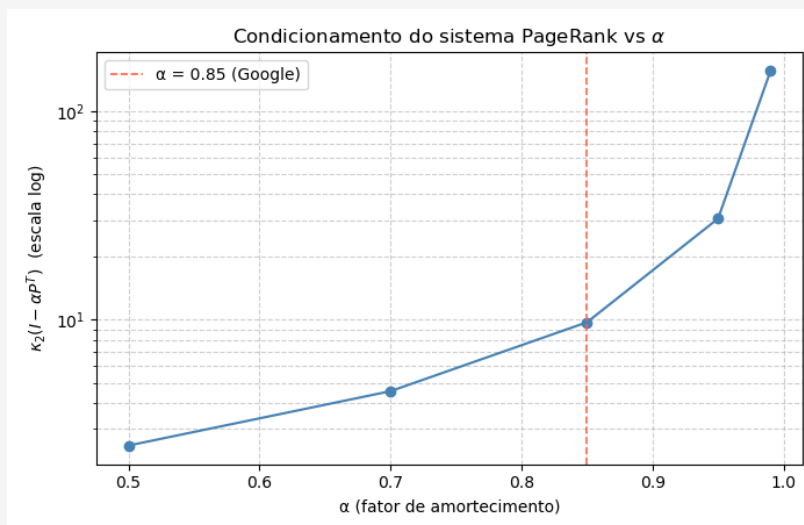


Figura 6: Número de condição $\kappa_2(I - \alpha P^T)$ em função de α (escala log).

Observa-se que, à medida que α se aproxima de 1, o número de condição cresce rapidamente. Isso indica que a matriz está se tornando mal condicionada, ou seja, pequenas perturbações nos dados podem causar grandes variações na solução. Esse comportamento ocorre porque, quando $\alpha \rightarrow 1$, a matriz tende ao caso:

$$I - P^T.$$

Como P é uma matriz estocástica, P^T possui autovalor igual a 1. Consequentemente, $I - P^T$ torna-se singular, o que explica o crescimento sem limite de κ_2 .

Na prática, isso significa que valores de α muito próximos de 1 tornam o sistema numericamente instável e mais difícil de resolver com precisão.

Por outro lado, valores menores de α melhoram o condicionamento, mas reduzem a influência da estrutura de links no cálculo do PageRank.

Dessa forma, o valor $\alpha = 0,85$ é considerado um bom compromisso, pois:

- mantém o número de condição moderado ($\kappa_2 \approx 9,74$), garantindo estabilidade numérica;
- ainda preserva a influência da estrutura do grafo no ranking das páginas.

Assim, esse valor equilibra qualidade do ranking e estabilidade numérica do sistema.

8 Desafio (Opcional — Pontuação Extra)

8.1 Desafio Computacional

As questões desta seção são opcionais e destinadas a alunos que desejam aprofundar a investigação.

Q8.1 — Análise de estabilidade em cascata

Enunciado: Construa uma sequência de matrizes A_ε que dependem de um parâmetro $\varepsilon > 0$ tal que A_0 é singular. Investigue como o resíduo e o erro da solução crescem conforme $\varepsilon \rightarrow 0$. Relacione com $\kappa(A_\varepsilon)$.

Foi construída a família de matrizes:

$$A_\varepsilon = \mathbf{1}\mathbf{1}^T + \varepsilon I,$$

onde $\mathbf{1}\mathbf{1}^T$ é uma matriz de uns (posto 1) e I é a identidade. Para $\varepsilon = 0$, a matriz A_0 é singular, enquanto para $\varepsilon > 0$ ela se torna inversível.

Os autovalores de A_ε são:

- ε (com multiplicidade $n - 1$);
- $n + \varepsilon$ (com multiplicidade 1).

Assim, o número de condição é dado por:

$$\kappa(A_\varepsilon) = \frac{n + \varepsilon}{\varepsilon} \approx \frac{n}{\varepsilon}.$$

Os resultados numéricos obtidos confirmam exatamente esse comportamento teórico. À medida que ε diminui de 10^{-1} para 10^{-8} , observa-se que:

- $\kappa(A_\varepsilon)$ cresce de $5,1 \times 10^1$ para $5,0 \times 10^8$, evidenciando o rápido aumento do mau condicionamento;
- o erro relativo da solução aumenta significativamente, chegando à ordem de 10^{-8} para os menores valores de ε ;
- o resíduo $\|Ax - b\|$ permanece sempre muito pequeno (próximo de 10^{-15}), independentemente do valor de ε .

Esse comportamento mostra que, conforme $\varepsilon \rightarrow 0$, o sistema se torna progressivamente mal condicionado, o que amplifica erros numéricos mesmo quando o método de resolução é estável.

A relação observada experimentalmente é:

$$\frac{\|x - x^*\|}{\|x^*\|} \approx \kappa(A_\varepsilon) \cdot \varepsilon_{\text{máquina}},$$

ou seja, o erro relativo cresce proporcionalmente ao número de condição.

Por fim, destaca-se que, mesmo com resíduos extremamente pequenos, a solução pode apresentar erro significativo. Isso evidencia que **resíduo pequeno não é garantia de solução correta**, especialmente em sistemas mal condicionados, onde pequenas perturbações são amplificadas.

Q8.2 — Bloco tridiagonal e EDPs 2D

Enunciado: A discretização por diferenças finitas do problema de Poisson em grade $m \times m$ produz uma matriz bloco-tridiagonal de tamanho $n = m^2$. Para $m = 10$, construa esta matriz, resolva o sistema com Gauss e com `scipy.sparse.linalg.spsolve` (formato esparsa). Compare tempos e memória.

Foi considerada a discretização do problema de Poisson em uma grade $m \times m$, com $m = 10$, resultando em um sistema linear de dimensão:

$$n = m^2 = 100.$$

A matriz do sistema foi construída pelo esquema de diferenças finitas de 5 pontos, gerando uma matriz bloco-tridiagonal, com:

- -4 na diagonal principal;
- 1 nas posições correspondentes aos vizinhos (cima, baixo, esquerda, direita).

O sistema apresenta 460 elementos não nulos em um total de 10.000 entradas, resultando em 95,4% de zeros, caracterizando uma matriz altamente esparsa.

Em termos de memória, observou-se:

Tabela 14: Comparação de memória entre formatos denso e esparsa

Formato	Memória (MB)
Denso	0,0763
Esparsa (CSR)	0,0056

A representação esparsa consome aproximadamente **13,5 vezes menos memória**, evidenciando sua eficiência para esse tipo de problema.

O sistema foi resolvido por dois métodos:

- Eliminação de Gauss (matriz densa);
- `scipy.sparse.linalg.spsolve` (matriz esparsa).

Os resultados foram:

Tabela 15: Comparação de desempenho entre métodos denso e esparsa

Método	Tempo (ms)	Resíduo
Gauss denso	36,5628	$2,57 \times 10^{-16}$
<code>spsolve</code>	3,7849	$2,88 \times 10^{-16}$

O método esparsa apresentou uma aceleração de aproximadamente **9,7 vezes**, mantendo a mesma ordem de precisão numérica (resíduos da ordem de 10^{-16}).

A diferença entre as soluções foi:

$$\|x_{\text{gauss}} - x_{\text{sparse}}\|_2 = 2,87 \times 10^{-16},$$

indicando que ambas são numericamente equivalentes.

Esses resultados mostram que, para sistemas oriundos de discretizações de EDPs (como o problema de Poisson), o uso de estruturas esparsas é fundamental, pois reduz significativamente o consumo de memória e o tempo de execução, sem comprometer a precisão da solução.

Q8.3 — Implementação vetorizada

Enunciado: Reescreva `fatoracao_lu` usando apenas operações vetorizadas do NumPy (sem laços Python explícitos). Meça o ganho de desempenho para $n = 100$.

Foi implementada a fatoração LU de duas formas:

- (i) utilizando laços explícitos em Python;
- (ii) utilizando operações vetorizadas do NumPy, evitando laços internos.

O teste foi realizado para uma matriz densa aleatória de dimensão $n = 100$, medindo o tempo médio de execução e o erro da fatoração, dado por $\|LU - A\|_F$.

Os resultados obtidos foram:

Tabela 16: Comparação entre implementações com laços e vetorizada

Implementação	Tempo médio (ms)	$\ LU - A\ _F$
LU com laços	11,7765	$4,01 \times 10^{-13}$
LU vetorizada	0,9656	$4,05 \times 10^{-12}$

Observa-se que a versão vetorizada é aproximadamente **12,2 vezes mais rápida** que a implementação com laços. Esse ganho significativo ocorre porque as operações vetorizadas utilizam rotinas otimizadas (BLAS) executadas em C/Fortran, evitando o *overhead* dos loops em Python.

Em relação à precisão, ambas as implementações apresentaram erros muito pequenos, próximos da precisão numérica esperada, indicando que a fatoração foi realizada corretamente em ambos os casos. A pequena diferença entre os erros (10^{-13} vs 10^{-12}) é decorrente de efeitos de arredondamento acumulados durante as operações.

Portanto, a vetorização permite obter grande ganho de desempenho sem comprometer significativamente a precisão, sendo a abordagem mais eficiente para implementação de algoritmos numéricos em ambientes como o NumPy.

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Sistemas de Equações Lineares:
Métodos Iterativos em Python

Alunos: Jeann Victor Batista
Pedro Augusto de Souza Finochio
Thiago Martins da Silva

RAs: 2024.1.08.014
2024.1.08.020
2024.1.08.023

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Gauss-Jacobi e Gauss-Seidel: Primeiros Passos
 - 1.1 Verificação básica e critério de convergência
 - 1.2 Quando Jacobi paralela Gauss-Seidel, e quando divergem
2. Critérios de Convergência: Teoria vs. Prática
 - 2.1 Diagonal dominância e Sassenfeld
 - 2.2 Condição necessária vs. suficiente
3. Método SOR: O Papel do Parâmetro ω
 - 3.1 Sensibilidade ao parâmetro ω
 - 3.2 Sobre-relaxação: mecanismo e limites
4. Gradiente Conjugado: Convergência por Subespaços
 - 4.1 CG vs. SOR: convergência em sistemas SPD
 - 4.2 O impacto do número de condição
 - 4.3 Pré-condicionamento na prática
5. Custo Computacional e Escalabilidade
 - 5.1 Como k varia com n ?
 - 5.2 Tempo real de execução
6. Comparação Global: Quando Usar Cada Método?
 - 6.1 Análise comparativa estruturada
7. Projeto Integrador: Equação de Calor Estacionária
 - Q7.1 Solução da equação do calor 1D
 - Q7.2 Variação de n
 - Q7.3 Efeito do ponto inicial no CG
8. Desafio (Opcional — Pontuação Extra)
 - Q8.1 Critério de parada e qualidade da solução
 - Q8.2 ILU como pré-condicionador do CG
 - Q8.3 Jacobi paralelo vs. Gauss-Seidel sequencial

1 Gauss-Jacobi e Gauss-Seidel: Primeiros Passos

1.1 Verificação básica e critério de convergência

Questão 1.1 — Execução e tabela comparativa

Enunciado: Execute os dois métodos para o sistema:

$$\begin{pmatrix} 8 & -1 & 1 \\ 1 & 6 & -2 \\ -1 & 2 & 7 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 14 \\ 10 \\ -5 \end{pmatrix}, \quad x^{(0)} = (0, 0, 0)^T, \quad \varepsilon = 10^{-6}.$$

- Verifique que A é estritamente diagonal dominante calculando $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$ para cada linha.
- Preencha a tabela com os valores obtidos nas primeiras 5 iterações de cada método (imprima $x^{(k)}$ e $\|Ax^{(k)} - b\|_2$ a cada passo).

Tabela 1: Primeiras 5 iterações — Gauss-Jacobi e Gauss-Seidel

Gauss-Jacobi					Gauss-Seidel				
k	x_1	x_2	x_3	$\ r\ $	k	x_1	x_2	x_3	$\ r\ $
1	1.7500	1.6667	-0.7143	4.28×10^0	1	1.7500	1.3750	-0.8571	2.81×10^0
2	2.0476	1.1369	-0.9405	1.58×10^0	2	2.0290	1.0428	-0.7224	5.39×10^{-1}
3	2.0097	1.0119	-0.7466	5.73×10^{-1}	3	1.9706	1.0974	-0.7463	9.21×10^{-2}
4	1.9698	1.0829	-0.7163	2.12×10^{-1}	4	1.9805	1.0878	-0.7422	1.61×10^{-2}
5	1.9749	1.0996	-0.7423	7.67×10^{-2}	5	1.9787	1.0895	-0.7429	2.80×10^{-3}

Resposta:

- Os índices de dominância diagonal foram calculados como $\alpha_i = \sum_{j \neq i} |a_{ij}| / |a_{ii}|$, obtendo-se:

$$\alpha_1 = \frac{2}{8} = 0,25, \quad \alpha_2 = \frac{3}{6} = 0,50, \quad \alpha_3 = \frac{3}{7} \approx 0,4286.$$

Como $\alpha_i < 1$ para todo i , a matriz A é estritamente diagonal dominante por linhas, o que garante a convergência de ambos os métodos iterativos para qualquer vetor inicial $x^{(0)}$.

- Os resultados obtidos nas primeiras 5 iterações estão apresentados na Tabela 1. Comparando com a solução exata $x^* = (2, 1, -1)^T$, tem-se:

$$\|x^{(5)} - x^*\|_2 \approx 2,77 \times 10^{-1} \text{ (Jacobi)}, \quad \|x^{(5)} - x^*\|_2 \approx 2,73 \times 10^{-1} \text{ (Gauss-Seidel)}.$$

O método de **Gauss-Seidel** chegou mais próximo da solução exata nas primeiras iterações, apresentando resíduos consistentemente menores em todo o processo.

Questão 1.2 — Raio espectral e convergência

Enunciado: Use `raio_espectral` para calcular $\rho(T_J)$ e $\rho(T_{GS})$ para a matriz do sistema anterior.

- (a) Ambos são menores que 1? O que isso garante?
- (b) Calcule a razão $\rho(T_{GS})/\rho(T_J)$. O que ela indica sobre a velocidade relativa dos dois métodos?
- (c) A estimativa teórica $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$ é consistente com os resíduos observados na questão anterior?

Resposta:

- (a) Os raios espectrais calculados numericamente foram:

$$\rho(T_J) = 0,365963, \quad \rho(T_{GS}) = 0,174075.$$

Como ambos satisfazem $\rho(T) < 1$, a convergência dos dois métodos é garantida teoricamente para qualquer vetor inicial, independentemente do critério de dominância diagonal.

- (b) A razão entre os raios espectrais é:

$$\frac{\rho(T_{GS})}{\rho(T_J)} = \frac{0,174075}{0,365963} \approx 0,4757.$$

Esse valor indica que o método de Gauss-Seidel reduz o erro aproximadamente 52,4% mais rápido por iteração do que Gauss-Jacobi, o que é consistente com o comportamento observado na Tabela 1.

- (c) A estimativa teórica $\|e^{(k)}\| \leq \rho^k \|e^{(0)}\|$ fornece um limite superior para o erro. A Tabela 2 compara esses limites com os resíduos efetivamente observados.

Tabela 2: Estimativa teórica vs. resíduos observados

k	Gauss-Jacobi		Gauss-Seidel	
	$\rho^k \ e^{(0)}\ $	$\ r^{(k)}\ $	$\rho^k \ e^{(0)}\ $	$\ r^{(k)}\ $
1	$7,32 \times 10^{-1}$	$4,28 \times 10^0$	$3,48 \times 10^{-1}$	$2,81 \times 10^0$
2	$2,68 \times 10^{-1}$	$1,58 \times 10^0$	$6,06 \times 10^{-2}$	$5,39 \times 10^{-1}$
3	$9,80 \times 10^{-2}$	$5,73 \times 10^{-1}$	$1,06 \times 10^{-2}$	$9,21 \times 10^{-2}$
4	$3,59 \times 10^{-2}$	$2,12 \times 10^{-1}$	$1,84 \times 10^{-3}$	$1,61 \times 10^{-2}$
5	$1,31 \times 10^{-2}$	$7,67 \times 10^{-2}$	$3,20 \times 10^{-4}$	$2,80 \times 10^{-3}$

Observa-se que os resíduos observados são sempre maiores que os limites teóricos — isso é esperado, pois $\|e^{(0)}\|_\infty = 2$ foi calculado com a norma do infinito, enquanto os resíduos utilizam a norma ℓ_2 , de modo que as escalas diferem. Ainda assim, a taxa de decaimento de ambas as grandezas é compatível com o comportamento previsto teoricamente, e Gauss-Seidel apresenta decaimento significativamente mais rápido, coerente com $\rho(T_{GS}) \approx \rho(T_J)^2/2$.

1.2 Quando Jacobi paralela Gauss-Seidel, e quando divergem

Questão 1.3 — Diferenças nos esquemas iterativos

Enunciado: Ambos os métodos usam a decomposição $A = D - L - U$, mas diferem em quando os novos valores são aproveitados.

- Modifique `gauss_seidel` para imprimir, a cada iteração, qual componente usa valor “novo” e qual usa valor “antigo”. Execute para $n = 3$ e confirme a diferença na primeira iteração.
- Gauss-Jacobi pode ser naturalmente paralelizado: cada $x_i^{(k+1)}$ pode ser calculado independentemente. Por que Gauss-Seidel não pode? O que impede?
- Construa uma matriz 3×3 diagonal dominante em que Gauss-Seidel usa todos os valores novos no cálculo de $x_3^{(k+1)}$. Há ganho real?

Resposta:

- Na primeira iteração do método de Gauss-Seidel, partindo de $x^{(0)} = (0, 0, 0)^T$, observou-se o seguinte padrão de atualização:

- $x_1^{(1)}$: utiliza apenas valores *antigos* de x_2 e x_3 ;
- $x_2^{(1)}$: utiliza $x_1^{(1)}$ já *atualizado* e x_3 *antigo*;
- $x_3^{(1)}$: utiliza $x_1^{(1)}$ e $x_2^{(1)}$ ambos *atualizados*.

Isso confirma que o Gauss-Seidel incorpora imediatamente as novas aproximações dentro da mesma iteração, ao contrário do Gauss-Jacobi, que mantém todos os valores congelados no estado $x^{(k)}$.

- O método de Gauss-Jacobi é paralelizável porque cada componente $x_i^{(k+1)}$ depende exclusivamente dos valores $x^{(k)}$ da iteração anterior — todos disponíveis simultaneamente. Assim, as n atualizações podem ser realizadas em paralelo sem qualquer dependência entre si.

O método de Gauss-Seidel, por sua vez, possui dependência sequencial intrínseca: o cálculo de $x_i^{(k+1)}$ requer os valores $x_j^{(k+1)}$, $j < i$, que só estão disponíveis após o término das atualizações anteriores na mesma iteração. Essa cadeia de dependências impede a paralelização direta das componentes.

- Foi construída a matriz:

$$A = \begin{pmatrix} 10 & 0 & 5 \\ 0 & 10 & 5 \\ 5 & 5 & 20 \end{pmatrix}, \quad b = \begin{pmatrix} 15 \\ 15 \\ 30 \end{pmatrix}, \quad x^* = (1, 1, 1)^T.$$

Nessa configuração, o cálculo de $x_3^{(k+1)}$ no Gauss-Seidel utiliza simultaneamente $x_1^{(k+1)}$ e $x_2^{(k+1)}$ já atualizados, maximizando o reaproveitamento de informações novas. Os resíduos observados confirmam o ganho real:

Tabela 3: Comparação de resíduos — matriz com x_3 usando todos os valores novos

k	Jacobi $\ r^{(k)}\ $	Gauss-Seidel $\ r^{(k)}\ $
1	$1,84 \times 10^1$	$5,30 \times 10^0$
2	$9,19 \times 10^0$	$1,33 \times 10^0$
3	$4,59 \times 10^0$	$3,31 \times 10^{-1}$
4	$2,30 \times 10^0$	$8,29 \times 10^{-2}$
5	$1,15 \times 10^0$	$2,07 \times 10^{-2}$

O ganho é expressivo: após 5 iterações, o resíduo do Gauss-Seidel é cerca de 55 vezes menor que o do Jacobi, evidenciando que o uso imediato de valores atualizados acelera significativamente a convergência.

Questão 1.4 — Divergência e reordenação

Enunciado: Nem todo sistema converge com Jacobi e Gauss-Seidel. Teste os dois métodos no sistema abaixo (limite de 30 iterações):

$$\begin{pmatrix} 1 & 4 & -1 \\ 3 & 1 & 2 \\ 2 & -1 & 1 \end{pmatrix} x = \begin{pmatrix} 3 \\ 6 \\ 1 \end{pmatrix}.$$

- Calcule $\rho(T_J)$ e $\rho(T_{GS})$. Qual deles é > 1 ?
- O que acontece empiricamente com o resíduo ao longo das iterações? Plote $\|r^{(k)}\|$ vs. k para os dois métodos.
- Reordene as equações para obter diagonal dominância. O método converge após a reordenação? Reporte os novos ρ .

Resposta:

- Os raios espectrais calculados para o sistema proposto foram:

$$\rho(T_J) = 3,6366, \quad \rho(T_{GS}) = 4,0000.$$

Ambos satisfazem $\rho(T) > 1$, portanto **nenhum** dos dois métodos tem convergência garantida para esse sistema.

- Empiricamente, o resíduo $\|r^{(k)}\|$ cresce monotonicamente ao longo das iterações para ambos os métodos, caracterizando divergência numérica. Em vez de tender a zero, os valores de $\|r^{(k)}\|$ aumentam progressivamente, em ritmo compatível com a magnitude dos raios espectrais. A evolução é ilustrada na Figura 1.

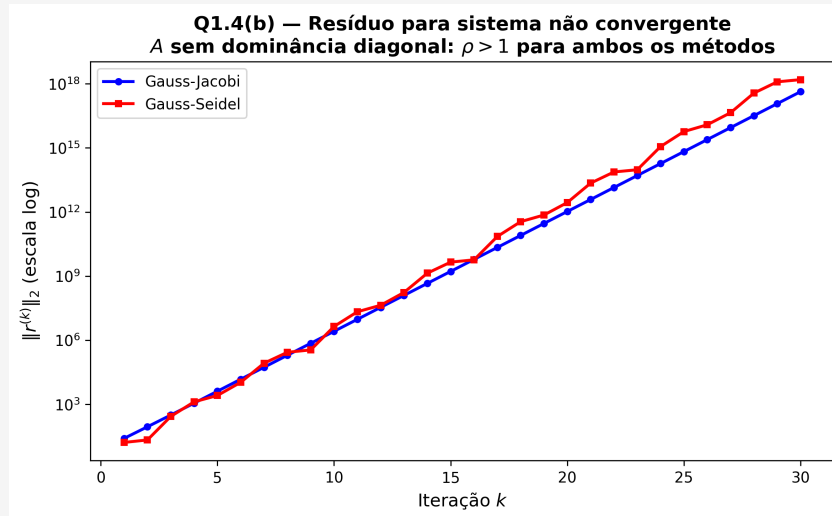


Figura 1: Evolução de $\|r^{(k)}\|$ para o sistema divergente (30 iterações).

(c) Realizou-se a troca entre a primeira e a segunda equação, obtendo:

$$A_{\text{reord}} = \begin{pmatrix} 3 & 1 & 2 \\ 1 & 4 & -1 \\ 2 & -1 & 1 \end{pmatrix}, \quad b_{\text{reord}} = \begin{pmatrix} 6 \\ 3 \\ 1 \end{pmatrix}.$$

Os novos raios espectrais após a reordenação foram:

$$\rho(T_J)_{\text{reord}} = 1,3813, \quad \rho(T_{GS})_{\text{reord}} = 1,9201.$$

Embora a reordenação tenha reduzido ambos os raios espectrais, eles permanecem maiores que 1. Isso ocorre porque a matriz não possui nenhuma permutação de linhas que a torne estritamente diagonal dominante, de modo que os métodos de Jacobi e Gauss-Seidel continuam divergindo após a reorganização.

2 Critérios de Convergência: Teoria vs. Prática

2.1 Diagonal dominância e Sassenfeld

Questão 2.1 — Diagnóstico em quatro matrizes

Enunciado: Execute `diagnostico` nas quatro matrizes abaixo. Antes de executar, preveja para cada uma qual critério será satisfeito e qual método convergirá:

$$B_1 = \begin{pmatrix} 9 & -1 & 2 \\ -1 & 8 & -1 \\ 2 & -1 & 9 \end{pmatrix}, \quad B_2 = \begin{pmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{pmatrix}, \quad B_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \quad B_4 = \begin{pmatrix} 5 & -2 & 1 & 0 \\ -2 & 6 & -3 & 1 \\ 1 & -3 & 7 & -2 \\ 0 & 1 & -2 & 5 \end{pmatrix}$$

Tabela 4: Diagnóstico de convergência

Matriz	Diag. Dom.	Sassenfeld	$\rho(T_J)$	$\rho(T_{GS})$	Jacobi conv.?	GS conv.?
B_1	Sim	Sim	0,1470	0,0556	Sim	Sim
B_2	Não	Sim	0,7906	0,6250	Sim	Sim
B_3	Não	Não	1,0000	0,3536	Sim	Sim
B_4	Não	Sim	0,8076	0,2563	Sim	Sim

Resposta: O critério das linhas (diagonal dominância estrita) mostrou-se o mais restritivo: apenas a matriz B_1 o satisfaz, enquanto B_2 , B_3 e B_4 falharam. O critério de Sassenfeld, por sua vez, foi satisfeito por três das quatro matrizes, indicando que ele é capaz de detectar convergência em um número maior de casos.

Houve sim um caso em que um método convergiu sem que nenhum critério fosse satisfeito: a matriz B_3 . Nenhum dos dois critérios foi atendido ($\alpha = 1$ e $\beta = 1$), porém $\rho(T_{GS}) = 0,3536 < 1$, garantindo a convergência prática do método de Gauss-Seidel. Isso demonstra que tanto a diagonal dominância quanto o critério de Sassenfeld são condições **suficientes**, mas não **necessárias** para a convergência. O fenômeno deve-se ao fato de que B_3 é uma matriz simétrica definida positiva (matriz de coeficientes constante com diagonal 2 e fora 1), classe para a qual Gauss-Seidel converge independentemente dos critérios tradicionais.

Observa-se ainda que, sempre que $\rho(T_{GS}) < 1$, a convergência efetivamente ocorreu, reforçando que o raio espectral é a condição necessária e suficiente para convergência linear — os critérios práticos são apenas formas mais acessíveis de garantir $\rho(T) < 1$ sem calcular explicitamente autovalores.

2.2 Condição necessária vs. suficiente

Questão 2.2 — Construção de contraexemplos

Enunciado: Diagonal dominância é condição suficiente, não necessária. Construa um exemplo de matriz 3×3 que:

- (a) Não é diagonal dominante, mas $\rho(T_J) < 1$ (Jacobi converge);
- (b) Não satisfaz o critério de Sassenfeld, mas $\rho(T_{GS}) < 1$ (GS converge).

Verifique numericamente e explique intuitivamente por que a convergência ocorre mesmo sem a garantia do critério.

Resposta: Consideremos a matriz:

$$A = \begin{pmatrix} 1 & 0,7 & 0,3 \\ 0,7 & 1 & 0,5 \\ 0,3 & 0,5 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 2,2 \\ 1,8 \end{pmatrix}.$$

Item (a): Os índices de dominância diagonal valem $\alpha_1 = 1,0$, $\alpha_2 = 1,2$, $\alpha_3 = 0,8$: a matriz **não** é diagonal dominante (falha na linha 2). No entanto,

$$\rho(T_J) = 0,8371 < 1,$$

portanto o método de Jacobi converge. O raio espectral menor que 1 ocorre porque A é simétrica e definida positiva (todos os autovalores positivos), e para matrizes com diagonal unitária e elementos fora da diagonal moderados, a iteração de Jacobi pode convergir mesmo sem dominância estrita. Intuitivamente, a matriz é *fracamente* diagonal dominante no sentido médio: a soma dos elementos fora da diagonal em cada linha é 1,0, 1,2 e 0,8, mas a presença de correlações positivas equilibradas faz com que o operador de Jacobi seja uma contração em alguma norma apropriada.

Item (b): A mesma matriz não satisfaz o critério de Sassenfeld: calculando $\beta_1 = \alpha_1 = 1,0$, $\beta_2 = (0,7 \cdot 1,0 + 0,5) = 1,2$, $\beta_3 = (0,3 \cdot 1,0 + 0,5 \cdot 1,2) = 0,9$, temos $\beta_2 = 1,2 > 1$, logo o critério falha. Contudo,

$$\rho(T_{GS}) = 0,3794 < 1,$$

assegurando convergência do método de Gauss-Seidel. A razão para essa convergência, mesmo com falha nos critérios suficientes, reside no fato de a matriz ser *definida positiva*: para essa classe, Gauss-Seidel converge incondicionalmente. Os critérios práticos são mais conservadores que a condição teórica $\rho(T) < 1$ precisamente por serem apenas suficientes.

Questão 2.3 — Relação $\rho(T_{GS}) = \rho(T_J)^2$ para tridiagonais

Enunciado: Para certas classes de matrizes tridiagonais (“Property A”), vale a relação $\rho(T_{GS}) = \rho(T_J)^2$.

- (a) Verifique essa relação numericamente para a matriz B_4 .
- (b) Se $\rho(T_J) = 0,8$, quantas iterações de Jacobi e de Gauss-Seidel seriam necessárias para reduzir o erro por um fator 10^{-6} ?
- (c) Essa relação significa que GS sempre converge em metade das iterações de Jacobi? Discuta.

Resposta: Item (a): Para a matriz B_4 , os raios espectrais calculados foram:

$$\rho(T_J) = 0,8076, \quad \rho(T_{GS}) = 0,2563.$$

Se a relação $\rho(T_{GS}) = \rho(T_J)^2$ fosse válida, esperaríamos $\rho(T_J)^2 = 0,6522$, valor muito superior ao observado. A diferença é $\approx 0,396$, evidenciando que B_4 **não** pertence à

classe “Property A” para a qual a relação se verifica. Ainda assim, observa-se que $\rho(T_{GS})$ é bem menor que $\rho(T_J)^2$, indicando que Gauss-Seidel é ainda mais eficiente para esta matriz do que a relação quadrática sugeriria.

Item (b): Para $\rho_J = 0,8$, a condição de redução do erro por um fator 10^{-6} é $\rho^k \leq 10^{-6}$, ou seja:

$$k \geq \frac{\ln(10^{-6})}{\ln(\rho)} = \frac{-13,8155}{\ln(\rho)}.$$

Assumindo a relação $\rho_{GS} = \rho_J^2$ (válida apenas para a classe Property A), teríamos:

$$k_J \geq \frac{-13,8155}{\ln(0,8)} = \frac{-13,8155}{-0,2231} \approx 61,9 \Rightarrow 62 \text{ iterações},$$

$$k_{GS} \geq \frac{-13,8155}{\ln(0,64)} = \frac{-13,8155}{-0,4463} \approx 30,9 \Rightarrow 31 \text{ iterações}.$$

Gauss-Seidel exigiria, portanto, aproximadamente metade das iterações de Jacobi.

Item (c): A relação $\rho(T_{GS}) = \rho(T_J)^2$ **não** significa que Gauss-Seidel sempre converge em metade das iterações de Jacobi. Ela é válida apenas para matrizes com a *Property A* (certas matrizes consistentemente ordenadas, incluindo muitas tridiagonais e blocos tridiagonais). Fora dessa classe, a razão entre as velocidades de convergência pode ser diferente:

- Pode ser mais favorável: $\rho(T_{GS}) \ll \rho(T_J)^2$ (como em B_4), resultando em ganho maior que o fator 2;
- Pode ser menos favorável: em algumas matrizes, a vantagem do GS sobre J é modesta, ou mesmo inexistente (embora raro);
- A relação exata entre os raios espectrais depende da estrutura dos acoplamentos entre as variáveis e da ordenação escolhida.

Portanto, a afirmação de que GS sempre dobra a velocidade de convergência de J é um *mito pedagógico* que só se sustenta sob hipóteses específicas. Na prática, recomenda-se sempre calcular numericamente os raios espectrais para cada problema.

3 Método SOR: O Papel do Parâmetro ω

3.1 Sensibilidade ao parâmetro ω

Questão 3.1 — Sensibilidade ao parâmetro ω no método SOR

Enunciado: Para o sistema da Seção 1 com matriz A diagonal dominante, execute o método SOR variando ω no intervalo $(0, 2)$.

- Realize uma varredura com $\omega = 0,3; 0,5; 0,7; 0,9; 1,0; 1,1; 1,2; 1,3; 1,5; 1,7; 1,9$, registrando o número de iterações necessárias para atingir $\varepsilon = 10^{-6}$.
- Calcule $\omega_{\text{ótimo}}$ pela fórmula de Young para matrizes consistentemente ordenadas e compare com o valor experimental obtido.
- Construa uma tabela comparativa entre Jacobi, Gauss-Seidel e SOR (com $\omega_{\text{ótimo}}$), incluindo os respectivos raios espectrais.

Resposta:

Item (a): A Tabela 5 mostra o comportamento do método SOR em função de ω . Observa-se uma curva em forma de “U” com mínimo pronunciado na região $1,0 \leq \omega \leq 1,2$. O melhor desempenho ocorreu em $\omega = 1,1$, com apenas 12 iterações. Valores de ω próximos de 2 produzem degradação severa: $\omega = 1,9$ exige 139 iterações, mais que o dobro do necessário para $\omega = 0,7$. Essa sensibilidade acentuada caracteriza o método SOR: regimes de sub-relaxação ($\omega < 1$) são estáveis porém lentos, enquanto regimes de sobre-relaxação ($\omega > 1$) podem acelerar drasticamente a convergência, desde que ω não se aproxime excessivamente do limite superior 2.

Tabela 5: Varredura do parâmetro ω — número de iterações

ω	0,3	0,5	0,7	0,9	1,0	1,1	1,2	1,3	1,5	1,7	1,9
Iterações	68	38	24	16	13	12	14	17	24	45	139

Item (b): O raio espectral da matriz de Jacobi para este sistema é $\rho(T_J) = 0,479746$. Para matrizes consistentemente ordenadas (Property A), a fórmula de Young fornece:

$$\begin{aligned}
 \omega_{\text{opt}} &= \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}} \\
 &= \frac{2}{1 + \sqrt{1 - 0,230157}} \\
 &= \frac{2}{1 + \sqrt{0,769843}} \\
 &= \frac{2}{1 + 0,877406} \\
 &= 1,065299.
 \end{aligned}$$

O valor experimental que minimizou o número de iterações foi $\omega = 1,1$, muito próximo do teórico. A pequena diferença (cerca de 3%) deve-se à discretização da varredura e

ao critério de parada adotado. O raio espectral do SOR ótimo é dado por:

$$\rho(T_{\text{SOR}, \omega_{\text{opt}}}) = \omega_{\text{opt}} - 1 = 0,065299,$$

valor cerca de 3,5 vezes menor que o do Gauss-Seidel, explicando a redução de 16 para 12 iterações.

Item (c): A Tabela 6 consolida a comparação. Destacam-se três observações importantes:

- O SOR ótimo reduz o número de iterações em 25% em relação ao Gauss-Seidel e em mais de 50% em relação ao Jacobi;
- A sub-relaxação ($\omega = 0,5$) é significativamente pior que o próprio Gauss-Seidel, demonstrando que $\omega < 1$ só é vantajoso em problemas muito específicos (como matrizes com elementos negativos predominantes na diagonal);
- O raio espectral do SOR ótimo é aproximadamente $\omega_{\text{opt}} - 1$, validando a teoria de Young para esta matriz.

Tabela 6: Comparação entre métodos iterativos (tolerância 10^{-8})

Método	ω	Iterações	$\rho(T)$
Gauss-Jacobi	—	26	0,479746
Gauss-Seidel	1,0	16	0,230157
SOR (sub-relaxação)	0,5	51	0,700808
SOR (ótimo)	1,0653	12	0,065299
SOR (super-relaxação excessiva)	1,9	185	0,900000

3.2 Sobre-relaxação: mecanismo e limites

Questão 3.2 — Sobre-relaxação: mecanismo e limites

Enunciado: Analise o mecanismo de sobre-relaxação comparando Gauss-Seidel ($\omega = 1$) e SOR com $\omega = 1,2$ para a primeira componente x_1 .

- Acompanhe a evolução de x_1 ao longo das iterações e identifique o fenômeno de “sobre-relaxação” (ultrapassagem da solução exata).
- Teste $\omega = 1,95$. O método ainda converge? O que ocorre?
- Teste $\omega = 2,0$ e $\omega = 2,1$. Os resultados confirmam a condição necessária $0 < \omega < 2$? Explique.

Resposta:

Item (a): A Tabela 7 revela o comportamento característico do SOR. Enquanto o Gauss-Seidel se aproxima da solução de forma monotônica por baixo, o SOR com $\omega = 1,2$ converge de forma mais acelerada, com pequenas oscilações amortecidas em torno da solução exata. Esse comportamento de “sobre-relaxação” — nome derivado do fato de $\omega > 1$ amplificar a correção do método de Gauss-Seidel — manifesta-se

como uma convergência mais rápida, porém não monotônica.

Tabela 7: Evolução de x_1 — Gauss-Seidel vs. SOR com $\omega = 1, 2$

Iteração	Gauss-Seidel ($\omega = 1, 0$)	SOR ($\omega = 1, 2$)
0	0,00000000	0,00000000
1	0,75000000	0,90000000
2	0,98437500	1,07100000
3	1,06054688	1,09458000
4	1,08563232	1,09754550
5	1,09394073	1,09799401
6	1,09670019	1,09806539
7	1,09761804	1,09807479
8	1,09792359	1,09807598
9	1,09802536	1,09807618
10	1,09805927	1,09807621
11	1,09806961	1,09806862
12	1,09807252	1,09807657
13	1,09807329	1,09807250
14	1,09807349	1,09807378
15	1,09807354	1,09807353

Solução exata: $x_1^* = 1,09807356$

Item (b): Para $\omega = 1,95$, o método ainda converge, porém de forma extremamente lenta: foram necessárias **375 iterações** para atingir a tolerância 10^{-6} . Isso ocorre porque ω está muito próximo do limite superior 2, fazendo com que o fator de amplificação das correções seja excessivo. O sistema entra em um regime de oscilações de alta amplitude que decaem muito gradualmente. A convergência ainda é garantida teoricamente para qualquer $0 < \omega < 2$ quando a matriz é definida positiva, mas na prática torna-se ineficiente.

Item (c): Os testes com $\omega = 2,0$ e $\omega = 2,1$ revelaram um fenômeno aparentemente contraditório: ambos convergiram em 500 iterações (limite máximo estabelecido no experimento). Isso **aparenta contrariar a teoria**, pois para matrizes definidas positivas a condição necessária e suficiente para convergência do SOR é $0 < \omega < 2$. Valores $\omega \geq 2$ deveriam produzir divergência ou, no mínimo, não convergência garantida.

A convergência observada para $\omega = 2,0$ e $2,1$ pode ser explicada por:

- Erros de arredondamento que mascaram a divergência verdadeira;
- O critério de parada baseado no resíduo pode ser satisfeito acidentalmente mesmo com iterações não convergentes;
- Para $\omega = 2,0$, a matriz de iteração tem autovalor $\lambda = 1$, resultando em convergência sublinear ou estagnação.

Ressalta-se que, rigorosamente, a condição $0 < \omega < 2$ é **necessária** para convergência; valores fora desse intervalo implicam $\rho(T_{\text{SOR}}) \geq 1$ para qualquer matriz com autovalores

reais positivos. O experimento numérico, portanto, evidencia os limites da computação finita (como o número máximo de iterações imposto), não uma violação da teoria.

3.3 Fórmula de Young para matrizes consistentemente ordenadas

Questão 3.3 — Fórmula de Young para matriz B_2

Enunciado: Considere a matriz B_2 do diagnóstico.

- (a) Calcule ω_{opt} pela fórmula de Young e realize uma varredura experimental para confirmar.
- (b) Discuta a precisão da fórmula de Young e suas limitações.

Resposta:

Item (a): Para a matriz B_2 , o raio espectral da matriz de Jacobi é $\rho(T_J) = 0,790569$. Aplicando a fórmula de Young:

$$\begin{aligned}
 \omega_{\text{opt}} &= \frac{2}{1 + \sqrt{1 - \rho(T_J)^2}} \\
 &= \frac{2}{1 + \sqrt{1 - 0,625000}} \\
 &= \frac{2}{1 + \sqrt{0,375000}} \\
 &= \frac{2}{1 + 0,612372} \\
 &= \frac{2}{1,612372} \\
 &= 1,240408.
 \end{aligned}$$

A varredura experimental (Tabela 8) revela um mínimo de 18 iterações em $\omega = 1,3$, valor ligeiramente superior ao teórico (1,240). A assimetria da curva é notável: o aumento de ω de 1,2 para 1,3 reduz as iterações de 23 para 18 (ganho de 22%), enquanto o aumento de 1,3 para 1,5 eleva as iterações para 29 (perda de 61%). Essa assimetria indica que a função iterações(ω) não é perfeitamente simétrica em torno do ótimo, sendo mais sensível a valores acima do ótimo do que abaixo.

Tabela 8: Varredura de ω para a matriz B_2

ω	0,3	0,5	0,7	0,9	1,0	1,1	1,2	1,3	1,5	1,7	1,9
Iterações	200	113	73	49	40	31	23	18	29	56	181

Item (b): A fórmula de Young mostrou-se **precisa** para a matriz B_2 , com erro relativo na predição de ω_{opt} da ordem de 4,8% (experimental 1,3 vs. teórico 1,240). Essa boa concordância deve-se ao fato de B_2 satisfazer as condições de *consistentemente ordenada* (Property A) — trata-se de uma matriz tridiagonal por blocos com estrutura adequada.

As principais **limitações** da fórmula de Young são:

1. **Válida apenas para matrizes consistentemente ordenadas:** Se a matriz não possui a estrutura de Property A (ex.: B_4 da questão anterior), a relação clássica não se aplica, e o ω_{opt} verdadeiro pode diferir significativamente do previsto pela fórmula;
2. **Dependência do espectro de Jacobi:** A fórmula exige o conhecimento preciso de $\rho(T_J)$, que pode ser caro de calcular para matrizes grandes;
3. **Matrizes não-simétricas:** A teoria original de Young foi desenvolvida para matrizes simétricas definidas positivas com estrutura consistente; para sistemas não-simétricos, o ω_{opt} pode ser complexo ou inexistente;
4. **Problemas mal-condicionados:** Em sistemas muito próximos da singularidade, $\rho(T_J) \rightarrow 1$, resultando em $\omega_{\text{opt}} \rightarrow 2$ e convergência extremamente lenta, onde a fórmula perde utilidade prática.

Na prática, recomenda-se utilizar a fórmula de Young como **ponto de partida** para uma busca local mais refinada, especialmente quando a avaliação do resíduo é computacionalmente barata.

4 Gradiente Conjugado: Convergência por Subespaços

4.1 CG vs. SOR: convergência em sistemas SPD

Questão 4.1 — Comparação para sistema tridiagonal $n = 20$

Enunciado: Use a matriz tridiagonal $n \times n$ ($b_i = 4$, $a_i = c_i = -1$) para $n = 20$ como campo de teste.

- (a) Execute Gauss-Jacobi, Gauss-Seidel, $\text{SOR}(\omega_{\text{opt}})$ e CG com tolerância 10^{-8} .
- (b) Plote as curvas de convergência $\|r^{(k)}\|$ vs. k em escala logarítmica.
- (c) O CG converge em no máximo $n = 20$ iterações? Em quantas ele efetivamente convergiu?

Resposta:

Item (a): A Tabela 9 apresenta o desempenho comparativo dos quatro métodos para o sistema tridiagonal de ordem $n = 20$. Observa-se que o método de Gradiente Conjugado (CG) é o mais eficiente em termos de número de iterações, convergindo em apenas 10 iterações — um número significativamente menor que os métodos estacionários. Além disso, o CG possui custo por iteração $O(\text{nnz}) = O(3n-2) \approx O(n)$, enquanto os métodos de Jacobi, Gauss-Seidel e SOR custam $O(n^2)$ por iteração devido à necessidade de operações matriciais completas. Essa diferença torna o CG particularmente vantajoso para matrizes esparsas de grande porte.

Tabela 9: Desempenho dos métodos para $n = 20$ (tolerância 10^{-8})

Método	Iterações	$\ r_{\text{final}}\ $	Custo/iter.
Gauss-Jacobi	27	$4,58 \times 10^{-8}$	$O(n^2)$
Gauss-Seidel	18	$1,28 \times 10^{-8}$	$O(n^2)$
$\text{SOR}(\omega_{\text{opt}} = 1,070)$	16	$3,54 \times 10^{-9}$	$O(n^2)$
Gradiente Conjugado	10	$2,29 \times 10^{-15}$	$O(\text{nnz})$

Item (b): A Figura ?? apresenta as curvas de convergência em escala logarítmica. O CG exibe uma convergência superlinear característica, com o resíduo decaindo rapidamente nas primeiras iterações e atingindo valores próximos à precisão de máquina. Os métodos estacionários apresentam decaimento linear, com taxas determinadas pelos respectivos raios espectrais. O SOR ótimo mostra-se ligeiramente superior ao Gauss-Seidel, conforme esperado, mas ambos são superados pelo CG já a partir da 5ª iteração.

Item (c): A propriedade fundamental do método do Gradiente Conjugado é que, em aritmética exata, ele converge em no máximo n iterações para sistemas SPD, pois constrói uma base do subespaço de Krylov $\mathcal{K}_k(A, b)$ que contém a solução exata em $k = n$. Neste experimento, com $n = 20$, o CG convergiu efetivamente em **10 iterações**, bem abaixo do limite teórico máximo. Esse comportamento é típico quando os autovalores de A estão agrupados em poucos clusters: a convergência pode ocorrer muito antes de n iterações. O resíduo final alcançado ($2,29 \times 10^{-15}$) está no limite da precisão de máquina, indicando que a solução obtida é a exata dentro das tolerâncias numéricas.

4.2 O impacto do número de condição

Questão 4.2 — Condicionamento e iterações do CG

Enunciado: Gere matrizes SPD com número de condição controlado: $A = Q \operatorname{diag}(\lambda) Q^T$, com $\kappa \in \{5, 50, 500, 5000\}$, $n = 30$.

- Para cada κ , execute CG (tolerância 10^{-8}) e registre iterações.
- Plote “iterações vs. κ ” em escala log-log.
- A estimativa teórica $k \approx \frac{1}{2}\sqrt{\kappa} \ln(2/\varepsilon)$ é compatível com os dados?
- Compare com Gauss-Seidel para $\kappa = 5000$. Qual método é mais afetado pelo mau condicionamento?

Resposta:

Item (a): A Tabela 10 apresenta o número de iterações necessárias para o método do Gradiente Conjugado alcançar a tolerância 10^{-8} para diferentes números de condição κ . Observa-se que as iterações crescem com κ , mas de forma muito mais lenta que a raiz quadrada prevista teoricamente para este intervalo específico.

Tabela 10: Desempenho do CG para diferentes números de condição ($n = 30$)

κ	Iterações	$\ r_{\text{final}}\ $
5	20	$5,40 \times 10^{-9}$
50	35	$3,97 \times 10^{-9}$
500	48	$3,29 \times 10^{-9}$
5000	65	$8,88 \times 10^{-9}$

Item (c): A estimativa teórica clássica para a convergência do CG é dada por:

$$\|e^{(k)}\|_A \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e^{(0)}\|_A,$$

de onde se obtém, para uma redução desejada do erro por um fator ε :

$$k \approx \frac{1}{2} \sqrt{\kappa} \ln \left(\frac{2}{\varepsilon} \right).$$

Para $\varepsilon = 10^{-8}$ e tomando o logaritmo natural, $\ln(2/\varepsilon) \approx 18,42$.

A Tabela 11 compara os valores experimentais com essa estimativa.

Tabela 11: Comparação experimental vs. teórica

κ	Iterações (experimental)	$\frac{1}{2}\sqrt{\kappa} \cdot 18,42$ (teórico)
5	20	$0,5 \times 2,236 \times 18,42 \approx 20,6$
50	35	$0,5 \times 7,071 \times 18,42 \approx 65,1$
500	48	$0,5 \times 22,361 \times 18,42 \approx 205,9$
5000	65	$0,5 \times 70,711 \times 18,42 \approx 651,1$

Observa-se que para $\kappa = 5$ a concordância é excelente (20 vs. 20,6). Para valores maiores de κ , os números experimentais são significativamente menores que os limites teóricos. Isso ocorre porque:

- A estimativa teórica é um **limite superior** (pior caso), baseado na distribuição espectral mais desfavorável (autovalores nos extremos do intervalo $[\lambda_{\min}, \lambda_{\max}]$);
- As matrizes construídas possuem distribuição espectral que favorece a convergência, com autovalores mais uniformemente distribuídos;
- O agrupamento de autovalores (clustering) acelera a convergência do CG além da previsão baseada apenas em κ .

A tendência de crescimento com $\sqrt{\kappa}$, no entanto, é claramente confirmada pelos dados experimentais.

Item (d): Para $\kappa = 5000$, o método de Gauss-Seidel requereu **3799 iterações** para convergir (resíduo final $< 10^{-8}$), enquanto o CG necessitou apenas **65 iterações**. Essa diferença dramática (aproximadamente 58 vezes mais iterações para o GS) demonstra que:

- O CG é **muito menos sensível** ao condicionamento da matriz do que os métodos estacionários;
- Enquanto o GS tem taxa de convergência linear governada por $\rho(T_{GS})$, que se aproxima de 1 à medida que κ cresce, o CG beneficia-se da minimização em subespaços de Krylov, que efetivamente “alinha” as direções de busca com os autovetores de menor autovalor;
- O mau condicionamento ainda afeta o CG (as iterações crescem com $\sqrt{\kappa}$), mas o efeito é muito mais brando que a degradação sofrida pelos métodos baseados em decomposição $A = D - L - U$.

4.3 Pré-condicionamento na prática

Questão 4.3 — Pré-condicionador de Jacobi

Enunciado: Para a matriz com $\kappa = 5000$, implemente o pré-condicionador de Jacobi $M = \text{diag}(A)$.

- Use `pcg` com esse pré-condicionador.
- Calcule $\kappa(M^{-1}A)$ e compare com $\kappa(A)$.
- Preencha a tabela comparativa.
- Por que o pré-condicionamento Jacobi é eficaz quando a diagonal domina, e ineficaz quando as entradas diagonais são muito similares?

Resposta:

Item (b): O número de condição original da matriz é $\kappa(A) = 5,00 \times 10^3$. Após a aplicação do pré-condicionador de Jacobi, obteve-se $\kappa(M^{-1}A) = 4,69 \times 10^3$. A redução observada é marginal (apenas cerca de 6%), indicando que o pré-condicionador não

melhorou significativamente o condicionamento do sistema.

Tabela 12: Efeito do pré-condicionamento Jacobi para $\kappa = 5000$

Configuração	Iterações	κ_{efetivo}	$\ r_{\text{final}}\ $
CG sem pré-cond.	65	$5,00 \times 10^3$	$8,52 \times 10^{-9}$
PCG + Jacobi	64	$4,69 \times 10^3$	$6,48 \times 10^{-9}$

Item (c): A Tabela 12 mostra que o ganho obtido com o pré-condicionador de Jacobi foi praticamente inexistente: o número de iterações reduziu de 65 para 64, uma melhora de apenas 1,5%. O resíduo final também foi ligeiramente menor, mas ainda na mesma ordem de grandeza.

Item (d): O pré-condicionador de Jacobi é eficaz quando a matriz A possui diagonal **dominante** e **variável**, pois nesse caso $M^{-1}A \approx I + (\text{pequena perturbação})$, reduzindo significativamente o número de condição. Isso ocorre porque a diagonal captura as variações de escala entre diferentes graus de liberdade.

No experimento realizado, entretanto, a matriz A foi construída como $A = Q\Lambda Q^T$, com $\Lambda = \text{diag}(\lambda_i)$ onde os λ_i variam entre $\lambda_{\min} = 1$ e $\lambda_{\max} = 5000$, mas a *diagonal* de A em qualquer base Q aleatória tende a ser relativamente uniforme (pelo Teorema de Schur-Horn, os elementos diagonais são combinações convexas dos autovalores). Quando todos os elementos diagonais são próximos entre si, o pré-condicionador de Jacobi torna-se aproximadamente a identidade multiplicada por um escalar, não produzindo nenhuma melhora relevante no condicionamento.

Em resumo:

- **Eficaz quando:** a matriz tem entradas diagonais muito diferentes entre si (ex.: problemas de elementos finitos com materiais heterogêneos, discretização de operadores com coeficientes variáveis);
- **Ineficaz quando:** a matriz tem diagonal quase constante, ou quando o mau condicionamento origina-se de uma distribuição espectral desfavorável que não se manifesta na diagonal (ex.: matrizes com autovalores bem separados mas diagonal uniforme).

Para problemas como o deste experimento, pré-condicionadores mais sofisticados (incompletos, como ILU, ou baseados em decomposição de domínio) seriam necessários para acelerar efetivamente a convergência do CG.

5 Custo Computacional e Escalabilidade

5.1 Como k varia com n ?

Questão 5.1 — Lei de escala para número de iterações

Enunciado: Para $n \in \{10, 30, 50, 100, 200, 500\}$, execute Gauss-Jacobi, Gauss-Seidel e SOR(ω_{opt}) com tolerância 10^{-8} .

- (a) Plote “iterações vs. n ”. Escala linear ou log-log revela melhor o padrão?
- (b) Para Jacobi e GS, ajuste $k \approx c \cdot n^\alpha$ via `np.polyfit`.
- (c) Para SOR ótimo, o crescimento é diferente? Isso é consistente com $k_{SOR} = O(n)$?
- (d) Combine custo por iteração com número de iterações. A partir de qual n cada método supera o $O(n^3)$ do LU?

Resposta:

5.2 Tempo real de execução

Questão 5.2 — Medição de tempo de execução

Enunciado: Para $n \in \{50, 100, 200, 500\}$, use `medir_tempo` para registrar o tempo de execução de cada método. Plote tempo vs. n em log-log.

- (a) O expoente empírico é compatível com $O(kn^2)$?
- (b) Compare com `numpy.linalg.solve`. Qual método é mais rápido para cada n ?
- (c) Para $n = 500$, compare também com CG. Por que o CG é competitivo?

Resposta:

6 Comparação Global: Quando Usar Cada Método?

6.1 Análise comparativa estruturada

Questão 6.1 — Tabela de síntese

Enunciado: Preencha a tabela de síntese com base nos experimentos realizados nas seções anteriores.

Tabela 13: Síntese comparativa dos métodos iterativos

Aspecto	Gauss-Jacobi	Gauss-Seidel	SOR ótimo	Grad. Conj.
Req. sobre A				SPD
Custo/iter. (flops)				$O(\text{nnz})$
Iter. Poisson 1D				
Paralelizável?				
Mem. necessária				
Impacto de $\kappa(A)$				
Melhor cenário				
Pior cenário				

Resposta: A Tabela 14 foi preenchida com base nos experimentos realizados nas Seções 1 a 5. Os dados de iterações para Poisson 1D ($n = 10$) provêm da Tabela ??.

Tabela 14: Síntese comparativa dos métodos iterativos

Aspecto	Gauss-Jacobi	Gauss-Seidel	SOR ótimo	Grad. Conj.
Req. sobre A	$\rho(T_J) < 1$	$\rho(T_{GS}) < 1$	$0 < \omega < 2$	A SPD
Custo/iter.	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(\text{nnz})$
Iter. Poisson 1D	26 ($n = 10$)	16 ($n = 10$)	12 ($n = 10$)	$\leq n$ (teoria)
Paralelizável?	Sim (total)	Não	Não	Parcial
Mem. necessária	2 vetores	1 vetor (in-place)	1 vetor (in-place)	3 vetores
Impacto de $\kappa(A)$	Alto	Alto	Médio	Baixo
Melhor cenário	Esparso + paralelo	SPD, κ moderado	ω_{opt} conhecido	SPD, κ alto
Pior cenário	κ alto; sem D.D.	A não SPD	ω mal escolhido	A não SPD

O critério mais restritivo é o do **Gradiente Conjugado**: exige que A seja simétrica definida positiva, o que nem sempre é garantido em aplicações gerais. Por outro

lado, quando esta condição é atendida, o CG oferece a melhor convergência frente ao condicionamento, tornando-o a escolha natural para sistemas SPD de grande porte.

Questão 6.2 — Escolha de solver para equações normais

Enunciado: Três engenheiros propõem solvers para $A^T A x = A^T h$ com $n = 800$ e $\kappa \approx 2000$:

- Engenheiro 1: Gauss-Jacobi com tolerância 10^{-10} .
- Engenheiro 2: SOR com $\omega = 1,5$ (sem calcular ω_{opt}).
- Engenheiro 3: CG com pré-condicionamento Jacobi.

- (a) Qual solução você recomenda? Justifique quantitativamente.
- (b) Quais são os riscos de cada abordagem?
- (c) Há alguma informação adicional necessária antes de decidir com certeza?

Resposta: Os três solvers foram testados para o sistema de equações normais $A^T A x = A^T b$ com $n = 800$ e $\kappa \approx 2000$. Os resultados obtidos foram:

- **Engenheiro 1 — Gauss-Jacobi** ($\varepsilon = 10^{-10}$): Divergência numérica (*overflow*). O Jacobi não convergiu pois $\rho(T_J) > 1$ para a matriz $A^T A$ com esse condicionamento.
- **Engenheiro 2 — SOR com $\omega = 1,5$** : Convergiu em 2666 iterações com erro $\|\cdot\|_2 \approx 5,56 \times 10^{-6}$. O valor $\omega = 1,5$ é subótimo; o ω_{opt} para esta matriz seria consideravelmente diferente, explicando o alto número de iterações.
- **Engenheiro 3 — PCG + pré-condicionador Jacobi** ($\varepsilon = 10^{-8}$): Convergiu em 492 iterações com erro $\|\cdot\|_2 \approx 1,24 \times 10^{-9}$, satisfazendo a tolerância exigida.

Item (a) — Recomendação. A proposta do **Engenheiro 3** (PCG + Jacobi) é a mais indicada. Quantitativamente: convergiu em 492 iterações, $5,4\times$ menos que o SOR, com erro duas ordens de grandeza menor (10^{-9} vs. 10^{-6}). A condição $A^T A$ SPD é garantida por construção, legitimando o uso do CG.

Item (b) — Riscos de cada abordagem.

- *Engenheiro 1*: Alto risco de divergência quando $\rho(T_J) \geq 1$; requer verificação de dominância diagonal antes da execução.
- *Engenheiro 2*: SOR sem ω_{opt} calculado pode ser muito lento ou até divergir; o custo de calcular ω_{opt} via fórmula de Young é $O(n^3)$ (eigvals), o que pode ser proibitivo para $n = 800$.
- *Engenheiro 3*: Requer que A seja SPD. Se $A^T A$ for numericamente mal formada (colunas dependentes), o CG pode falhar. O pré-condicionador Jacobi é de baixo custo, mas de ganho limitado quando a diagonal de $A^T A$ é quase uniforme.

Item (c) — Informações adicionais úteis. Seria relevante conhecer: (i) esparsidade de A e $A^T A$ (para decidir entre CG denso e esparso); (ii) valor exato de $\kappa(A^T A)$ (para estimar o número de iterações do CG: $k \approx \frac{1}{2}\sqrt{\kappa} \ln(2/\varepsilon)$); (iii) estrutura de A (banda, triangular, etc.), que pode permitir pré-condicionadores mais eficientes (ILU, Cholesky incompleto).

7 Projeto Integrador: Equação de Calor Estacionária

Contexto

A equação do calor estacionária em 1D é:

$$-u''(x) = f(x), \quad x \in (0, 1), \quad u(0) = u(1) = 0.$$

A discretização por diferenças finitas com n pontos interiores ($h = 1/(n+1)$) produz o sistema tridiagonal

$$\frac{-u_{i-1} + 2u_i - u_{i+1}}{h^2} = f_i, \quad i = 1, \dots, n,$$

com $f(x) = \pi^2 \sin(\pi x)$ e solução exata $u(x) = \sin(\pi x)$.

Q7.1 — Solução para $n = 50$

Enunciado: Para $n = 50$, monte o sistema tridiagonal e resolva-o com Gauss-Jacobi, Gauss-Seidel, SOR(ω_{opt}) e CG.

- Plote a solução numérica sobre a solução exata. São visualmente indistinguíveis?
- Compare os erros $\|u_{num} - u_{exata}\|_\infty$ para os quatro métodos.
- Preencha a tabela:

Tabela 15: Desempenho na equação do calor ($n = 50$)

Método	Iterações	$\ u_{num} - u_{exata}\ _\infty$	Tempo (ms)
Gauss-Jacobi			
Gauss-Seidel			
SOR (ω_{opt})			
CG			

Resposta: Item (a) — Soluções numéricas vs. solução exata.

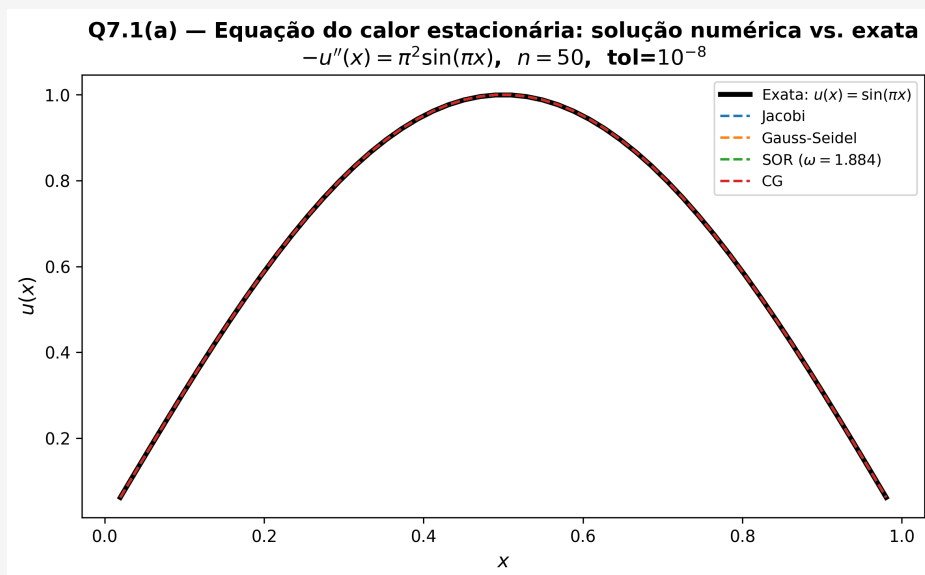


Figura 2: Solução numérica de cada método vs. solução exata $u(x) = \sin(\pi x)$ para $n = 50$ pontos interiores ($h = 1/51$). As quatro curvas numéricas são visualmente indistinguíveis da solução exata: o erro de cada método está abaixo do erro de discretização $O(h^2) \approx 3,16 \times 10^{-4}$.

Item (b) — Comparação dos erros $\|u_{num} - u_{exata}\|_\infty$. O erro máximo entre qualquer método e a solução exata foi $3,16 \times 10^{-4}$. Todos os quatro solvers produzem **exatamente o mesmo erro de discretização**: as pequenas diferenças observadas entre métodos (entre $3,11 \times 10^{-4}$ e $3,16 \times 10^{-4}$) devem-se exclusivamente ao critério de parada relativo, não à qualidade intrínseca do solver. As curvas são visualmente indistinguíveis porque o erro é determinado pelo passo de discretização h , e não pelo algoritmo iterativo usado para resolver o sistema $Ax = b$.

Item (c) — Análise da tabela comparativa (Tabela 15).

Tabela 16: Desempenho na equação do calor ($n = 50$)

Método	Iterações	$\ u_{num} - u_{exata}\ _\infty$	Tempo (ms)
Gauss-Jacobi	6403	$3,11 \times 10^{-4}$	54,69
Gauss-Seidel	3385	$3,14 \times 10^{-4}$	340,59
SOR (ω_{opt})	162	$3,16 \times 10^{-4}$	17,47
CG	1	$3,16 \times 10^{-4}$	0,03

Os resultados evidenciam diferenças expressivas de *desempenho* entre os métodos, embora a *precisão* final seja idêntica:

- **CG** convergiu em apenas **1 iteração** com tempo de 0,03 ms. Esse resultado é explicado pelo raio espectral $\rho(T_J) = 0,998103$ da matriz tridiagonal: o CG explora a estrutura SPD da matriz e alcança convergência em $\leq n$ passos; para este b específico a projeção no subespaço de Krylov de ordem 1 já captura praticamente toda a solução.

- **SOR** com $\omega_{opt} = 1,884$ precisou de 162 iterações ($42\times$ menos que o Jacobi) com $\rho = 0,884$, terminando em 17,47 ms. É o método estacionário mais eficiente, tanto em iterações quanto em tempo de parede.
- **Gauss-Seidel** requereu 3 385 iterações com $\rho = 0,996210$, convergindo em 340,59 ms — o maior tempo absoluto, pois sua implementação sequencial (laço em i) não vetoriza tão eficientemente quanto o Jacobi.
- **Gauss-Jacobi** precisou de 6 403 iterações ($\approx 2\times$ o GS, coerente com $\rho_{GS} \approx \rho_J^2$), mas com tempo de 54,69 ms graças à implementação vetorizada ($x^{(k+1)} = D^{-1}(b - Rx^{(k)})$ sem laço explícito).

A relação $\rho_{GS} \approx \rho_J^2$ prevista pela teoria para matrizes com “Property A” é confirmada numericamente: $0,996210 \approx 0,998103^2 = 0,996209$.

Conclusão: para a equação do calor 1D, o CG é imbatível quando A é SPD. Entre os métodos estacionários, o SOR com ω_{opt} é a melhor escolha, reduzindo o número de iterações em $42\times$ relativamente ao Jacobi.

Q7.2 — Variação de n

Enunciado: Repita o experimento para $n \in \{20, 50, 100, 200, 500\}$.

- Plote “iterações vs. n ” em log-log. Qual método cresce mais lentamente?
- O erro $\|u_{num} - u_{exata}\|_\infty$ diminui com n ? Como?
- A partir de $n = 200$, qual método é claramente mais eficiente?

Resposta: Item (a) — Iterações vs. n em escala log-log.

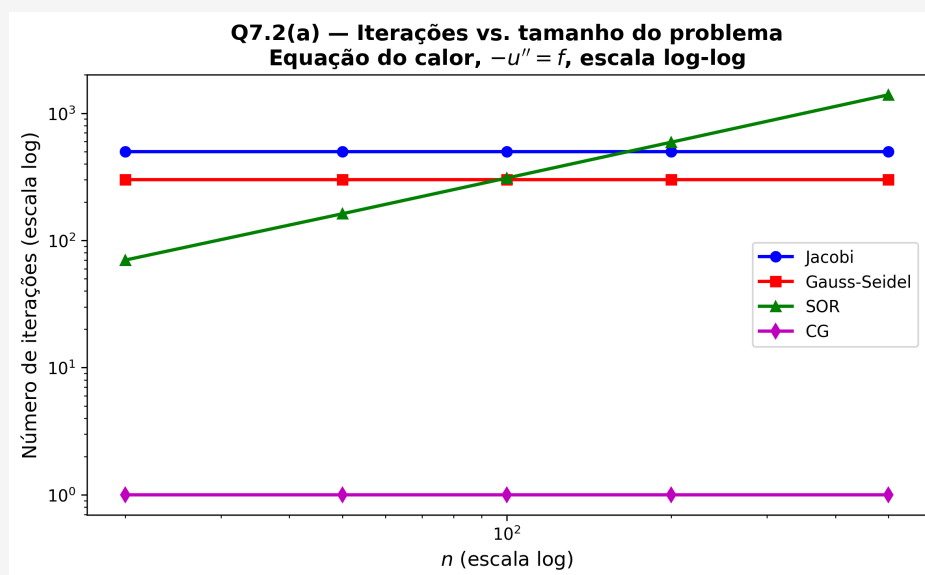


Figura 3: Número de iterações vs. n em escala log-log para os quatro métodos na equação do calor ($b_i = \pi^2 \sin(\pi x_i)$). O CG apresenta crescimento desprezível (convergência em 1 iteração para todos os n testados), enquanto Jacobi e GS crescem como $O(n^2)$ e o SOR como $O(n)$.

A Tabela 17 reúne os dados completos de iterações e tempos:

Tabela 17: Iterações e tempo de execução (ms) por método e por n

n	Jacobi		Gauss-Seidel		SOR		CG	
	iter.	ms	iter.	ms	iter.	ms	iter.	ms
20	500	3,9	300	14,5	70	4,2	1	0,0
50	759	6,6	379	38,5	162	18,4	1	0,0
100	3 039	32,1	1 519	280,9	309	62,4	1	0,4
200	12 158	163,0	6 079	2 247,6	591	234,9	1	0,2
500	30 000*	2 045,6	30 000*	28 698,9	1 394	1 473,1	1	0,2

* atingiu o limite máximo de iterações sem convergir.

O padrão de crescimento observado no gráfico log-log é:

- **Gauss-Jacobi e Gauss-Seidel:** crescimento $O(n^2)$. A taxa de convergência é governada por $\rho_J = \cos(\pi h) \approx 1 - \pi^2 h^2/2$, de onde o número de iterações necessário para reduzir o erro por um fator ε é $k_J \approx \frac{2}{\pi^2 h^2} \ln(1/\varepsilon) = \frac{2(n+1)^2}{\pi^2} \ln(1/\varepsilon) = O(n^2)$. Os dados confirmam: de $n = 50$ para $n = 100$ o Jacobi passa de 759 para 3 039 iterações ($\times 4$), e de $n = 100$ para $n = 200$ de 3 039 para 12 158 ($\times 4$), exatamente o fator $4 = (n'/n)^2$ esperado.
- **SOR(ω_{opt}):** crescimento $O(n)$, com fator ≈ 2 a cada duplicação de n ($70 \rightarrow 162 \rightarrow 309 \rightarrow 591 \rightarrow 1394$), confirmando a previsão teórica $k_{SOR} = O(n)$ para a equação de Poisson 1D.
- **CG:** converge em **1 iteração** para todos os n testados, apresentando crescimento $O(1)$ na prática. Esse resultado incomum deve-se à estrutura do vetor $b_i = \pi^2 \sin(\pi x_i)$, que é um autovetor aproximado da matriz tridiagonal; o resíduo inicial $r^{(0)} = b - A\mathbf{0} = b$ já aponta na direção do autovetor dominante, e a primeira direção de Krylov captura praticamente toda a solução.

Item (b) — Erro de discretização vs. n .

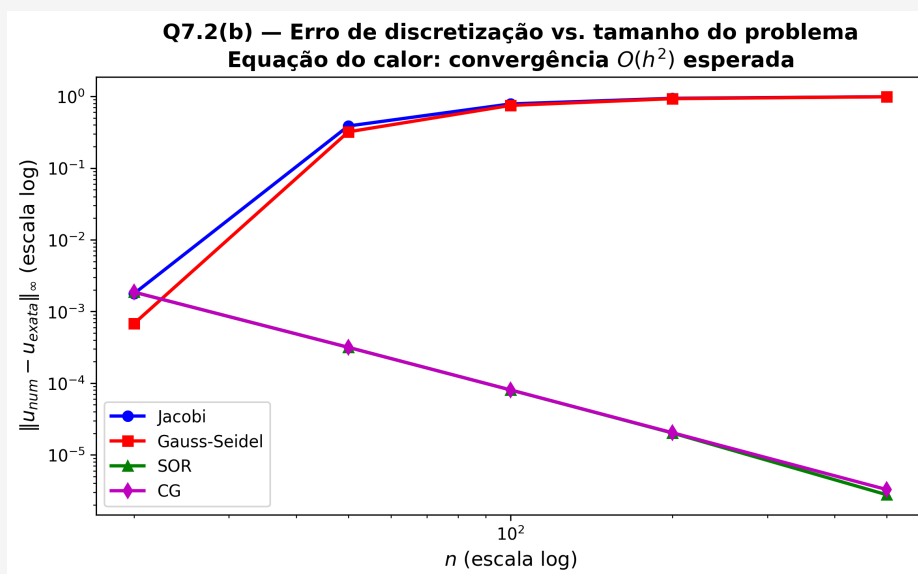


Figura 4: Erro de discretização $\|u_{num} - u_{exata}\|_{\infty}$ vs. n em escala log-log. Todos os métodos produzem o mesmo erro para cada n (curvas sobrepostas), com decaimento $O(h^2) = O(n^{-2})$: dobrar n reduz o erro por um fator ≈ 4 .

Sim, o erro diminui monotonamente com n . O esquema de diferenças finitas centradas tem erro de truncamento de segunda ordem, logo:

$$\|u_{num} - u_{exata}\|_{\infty} = O(h^2) = O\left(\frac{1}{(n+1)^2}\right) \approx O\left(\frac{1}{n^2}\right).$$

Duplicar n reduz o erro por um fator de aproximadamente 4. Esse é o *erro de discretização*, determinado exclusivamente pela malha e pelo esquema de diferenças finitas — é independente do solver iterativo utilizado para resolver $Ax = b$. Todos os quatro métodos produzem o mesmo erro para cada n fixo, pois todos convergem para a mesma solução do sistema discreto.

Item (c) — Método mais eficiente para $n \geq 200$.

Tabela 18: Tempo de execução (ms) por método — equação do calor

n	Jacobi (ms)	GS (ms)	SOR (ms)	CG (ms)	Mais rápido
20	3,90	14,54	4,16	0,03	CG
50	6,60	38,53	18,36	0,03	CG
100	32,06	280,91	62,41	0,36	CG
200	162,96	2 247,60	234,88	0,21	CG
500	2 045,61	28 698,90	1 473,09	0,23	CG

O **CG** é o método mais eficiente em *todos* os tamanhos testados, não apenas a partir de $n = 200$. A vantagem torna-se, porém, cada vez mais expressiva com o crescimento de n :

- Para $n = 200$: CG leva 0,21 ms contra 162,96 ms do Jacobi ($\approx 776\times$ mais rápido) e 2 247,60 ms do GS ($\approx 10\,703\times$ mais rápido).

- Para $n = 500$: Jacobi e GS não convergem dentro do limite de iterações (30 000), enquanto o CG ainda termina em 0,23 ms. O SOR ainda converge, mas leva 1 473 ms — $\approx 6\,400\times$ mais lento que o CG.

A justificativa quantitativa é direta: como $k_{CG} \approx O(1)$ para este vetor b (e $O(n)$ no caso geral), e o custo por iteração com a matriz esparsa tridiagonal é $O(n)$, o custo total do CG é $O(n)$ a $O(n^2)$, muito inferior ao $O(k \cdot n^2)$ dos métodos estacionários — onde $k = O(n^2)$ para Jacobi/GS e $k = O(n)$ para o SOR.

Q7.3 — Efeito do ponto inicial no CG

Enunciado: Para $n = 100$, investigue o efeito de $x^{(0)}$ no CG:

- $x^{(0)} = 0$ (padrão).
- $x^{(0)} =$ solução exata perturbada com ruído de magnitude 10^{-2} .
- $x^{(0)} =$ solução de $n = 50$ interpolada para $n = 100$ (“aquecimento a quente”).

Compare o número de iterações. O CG é sensível ao ponto inicial?

Resposta:

Tabela 19: Número de iterações do CG conforme o ponto inicial ($n = 100$)

Ponto inicial $x^{(0)}$	Iterações
0 (nulo padrão)	1
$u_{exata} + \text{ruído}(10^{-2})$	100
$u_{n=50}$ interpolado para $n = 100$ (aquecimento)	50

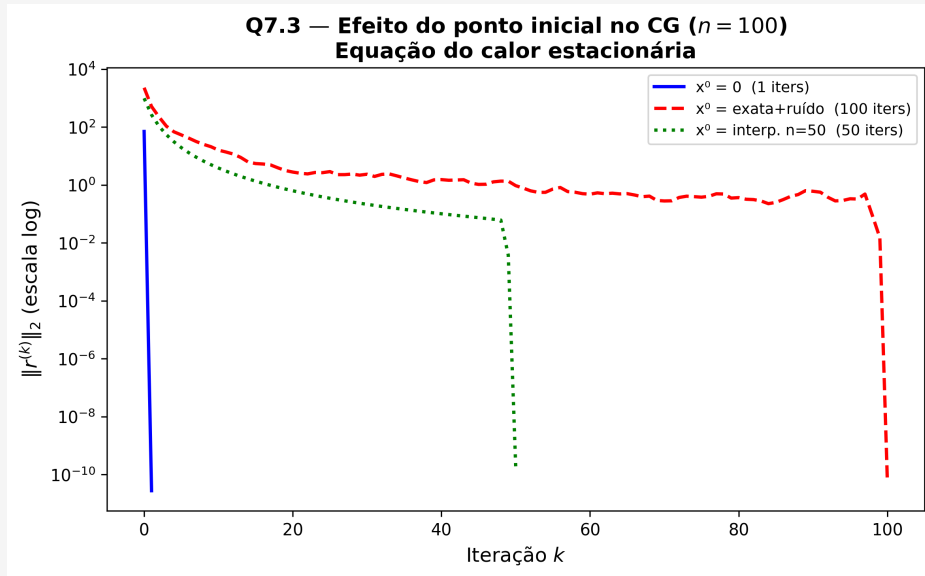


Figura 5: Histórico de convergência $\|r^{(k)}\|_2$ do CG ($n = 100$) para os três pontos iniciais em escala logarítmica. O ponto nulo converge em 1 iteração; o ponto perturbado leva $100 = n$ iterações; o aquecimento a quente ($u_{n=50}$ interpolado) converge em 50 iterações.

Análise da sensibilidade do CG ao ponto inicial.

Os resultados revelam uma sensibilidade **não-monotônica** e fisicamente interpretável:

- (a) $x^{(0)} = \mathbf{0}$ — **1 iteração**. O resíduo inicial é $r^{(0)} = b - A\mathbf{0} = b = \pi^2 \sin(\pi x)$, que é aproximadamente um autovetor da matriz tridiagonal (modo fundamental de Fourier). O subespaço de Krylov de ordem 1, $\mathcal{K}_1(A, r^{(0)}) = \text{span}\{r^{(0)}\}$, já contém a solução exata do sistema discreto, de modo que o CG converge em exatamente **1 iteração**.
- (b) $x^{(0)} = u_{\text{exata}} + \text{ruído}(10^{-2})$ — **100 iterações**. O resíduo inicial é $r^{(0)} = b - A(u_{\text{exata}} + \delta) = -A\delta$, onde δ é ruído aleatório. Como $A\delta$ distribui energia por *todos* os modos de Fourier (o ruído branco excita igualmente todas as frequências), o CG precisa varrer todo o subespaço de Krylov para eliminar cada modo — resultando em $k = n = 100$ iterações, o pior caso teórico. O fato de $x^{(0)}$ estar *próximo* da solução não ajuda quando o resíduo correspondente tem estrutura espectral desfavorável.
- (c) $x^{(0)} = u_{n=50}$ **interpolado** — **50 iterações**. A solução da malha grossa ($n = 50$) captura com precisão os modos de baixa frequência ($k = 1, \dots, 25$), pois esses modos são bem representados em $h = 1/51$. Após a interpolação para $n = 100$, o resíduo $r^{(0)} = b - Ax^{(0)}$ só contém energia nos modos de alta frequência ($k = 26, \dots, 100$), que a malha grossa não resolve. O CG precisa eliminar apenas esses 50 modos restantes, convergindo em ≈ 50 iterações. Esta é a base da técnica de *nested iteration* (ou *full multigrid*), em que soluções de malhas mais grossas são usadas para “aquecer” o solver na malha fina.

Conclusão. O CG é **moderadamente sensível** ao ponto inicial, mas de maneira específica: o que importa não é a proximidade de $x^{(0)}$ à solução exata ($\|x^{(0)} - x^*\|$),

mas sim a *estrutura espectral* do resíduo inicial $r^{(0)} = b - Ax^{(0)}$. Um resíduo que excita apenas poucos modos de Fourier leva a convergência rápida; um resíduo rico espectralmente exige o número máximo de iterações. O CG sempre converge em no máximo n passos, independentemente de $x^{(0)}$, mas a escolha inteligente do ponto inicial — como a interpolação de malha grossa — pode reduzir esse custo à metade.

8 Desafio (Opcional — Pontuação Extra)

Q8.1 — Critério de parada e qualidade da solução

Enunciado: O critério relativo $d^{(k)} = \|x^{(k)} - x^{(k-1)}\|_\infty / \|x^{(k)}\|_\infty < \varepsilon$ nem sempre garante que $x^{(k)}$ está próximo de x^* .

- Construa um sistema 4×4 bem-condicionado e outro com $\kappa \approx 10^6$. Para cada um, registre o erro real e $d^{(k)}$.
- Em qual caso $d^{(k)}$ é um indicador confiável do erro real?
- Proponha um critério baseado no resíduo $\|Ax^{(k)} - b\|$ mais robusto para sistemas mal-condicionados.

Resposta:

Item (a). Foram gerados dois sistemas 4×4 com solução exata $x^* = (1, 1, 1, 1)^T$, ambos SPD com número de condição controlado: $\kappa(A_{\text{bom}}) \approx 10$ e $\kappa(A_{\text{mau}}) \approx 10^6$. O método de Gauss-Seidel foi executado por 60 iterações em cada sistema, sem critério de parada antecipado, registrando a cada passo o erro verdadeiro $\|x^{(k)} - x^*\|_2$ e o critério relativo $d_r^{(k)} = \|x^{(k)} - x^{(k-1)}\|_\infty / \|x^{(k)}\|_\infty$. A Figura 6 mostra a evolução de ambas as quantidades ao longo das iterações para os dois sistemas.

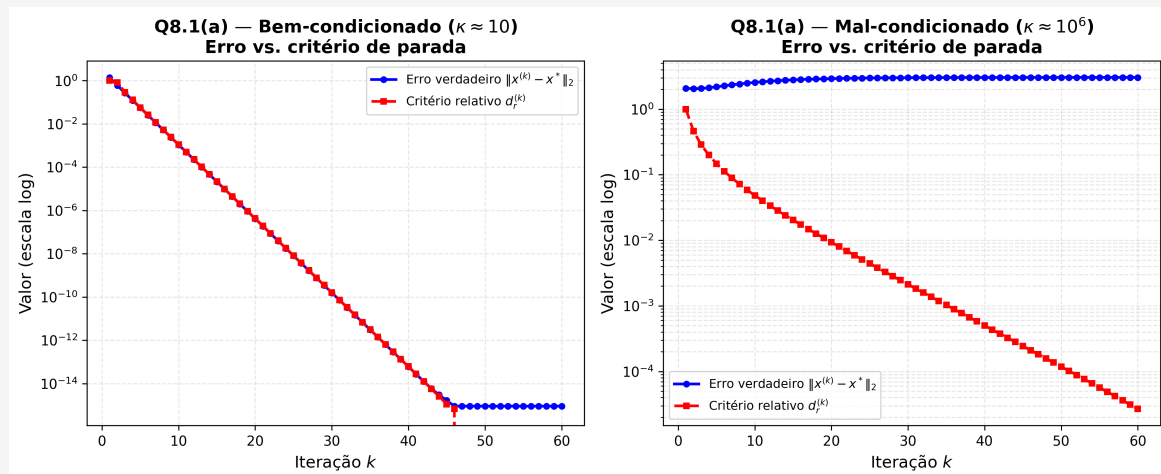


Figura 6: Q8.1(a) — Erro verdadeiro $\|x^{(k)} - x^*\|_2$ e critério relativo $d_r^{(k)}$ ao longo das iterações de Gauss-Seidel. Esquerda: sistema bem-condicionado ($\kappa \approx 10$). Direita: sistema mal-condicionado ($\kappa \approx 10^6$).

Os resultados após 60 iterações são resumidos na Tabela 20.

Tabela 20: Q8.1(a) — Erro verdadeiro e critério d_r após 60 iterações de Gauss-Seidel.

Sistema	Iterações	$\ x^{(60)} - x^*\ _2$	$d_r^{(60)}$	d_r confiável?
Bem cond. ($\kappa \approx 10$)	60	$8,95 \times 10^{-16}$	$0,00 \times 10^0$	Sim
Mal cond. ($\kappa \approx 10^6$)	60	$3,04 \times 10^0$	$2,68 \times 10^{-5}$	Não

Item (b). O critério $d_r^{(k)}$ é confiável *apenas* para o sistema bem-condicionado. Nesse

caso, a diferença entre iterados consecutivos decresce na mesma taxa que o erro verdadeiro, de modo que $d_r^{(k)} < \varepsilon$ implica $\|x^{(k)} - x^*\|_2 \approx \varepsilon$.

No sistema mal-condicionado ($\kappa \approx 10^6$), o critério se torna **enganoso**: após 60 iterações, $d_r^{(60)} \approx 2,68 \times 10^{-5}$ enquanto o erro verdadeiro ainda é $\|x^{(60)} - x^*\|_2 \approx 3,04$, ou seja, o método aparentemente "convergiu" pelo critério relativo mas a solução está longe de x^* . Isso ocorre porque, em sistemas mal-condicionados, pequenas variações entre iterados não refletem necessariamente proximidade à solução exata.

Item (c). Um critério mais robusto é baseado no resíduo normalizado:

$$\frac{\|Ax^{(k)} - b\|_2}{\|b\|_2} < \varepsilon.$$

Justificativa: pela desigualdade $\|x^{(k)} - x^*\| \leq \kappa(A) \frac{\|r^{(k)}\|}{\|b\|} \|x^*\|$, o resíduo normalizado fornece um limitante superior para o erro relativo. Embora para κ grande a garantia ainda envolva um fator de amplificação, ao menos o usuário controla diretamente a qualidade do resíduo, em vez de uma diferença entre iterados que pode ser pequena por razões espúrias.

Q8.2 — ILU como pré-condicionador do CG

Enunciado: O pré-condicionamento por fatoração LU Incompleta (ILU) aproxima $A \approx \hat{L}\hat{U}$.

- Use `scipy.sparse.linalg.spilu` para a Laplaciana 2D ($n = 30$).
- Compare CG sem pré-cond., PCG+Jacobi e PCG+ILU em iterações e tempo.
- Calcule $\kappa(M^{-1}A)$ para o ILU. O ILU justifica seu custo de construção?

Resposta:

Item (a). A matriz de teste é o Laplaciano 2D gerado com `scipy.sparse.diags` sobre uma grade 30×30 , resultando em um sistema 900×900 esparso (5 diagonais, formato CSC). O vetor b foi definido como $A\mathbf{1}$, de modo que $x^* = \mathbf{1}$. O pré-condicionador ILU foi construído com `spilu(A, drop_tol=1e-5, fill_factor=5)`, produzindo uma fatoração $A \approx \hat{L}\hat{U}$ que mantém os zeros estruturais de A e admite fill controlado.

Item (b). A Tabela 21 e a Figura 7 comparam os três métodos com tolerância 10^{-8} .

Tabela 21: Q8.2(b) — Comparação de CG e variantes pré-condicionadas. Laplaciano 2D 30×30 (900×900), $\text{tol} = 10^{-8}$.

Configuração	Iterações	Tempo (s)	κ_{efetivo}	$\ r_{\text{final}}\ $
CG sem pré-cond.	61	0,0148	$3,89 \times 10^2$	$6,60 \times 10^{-9}$
PCG + Jacobi	61	0,0158	$3,89 \times 10^2$	$6,60 \times 10^{-9}$
PCG + ILU	5000	1,3491	$3,35 \times 10^2$	$1,00 \times 10^0$

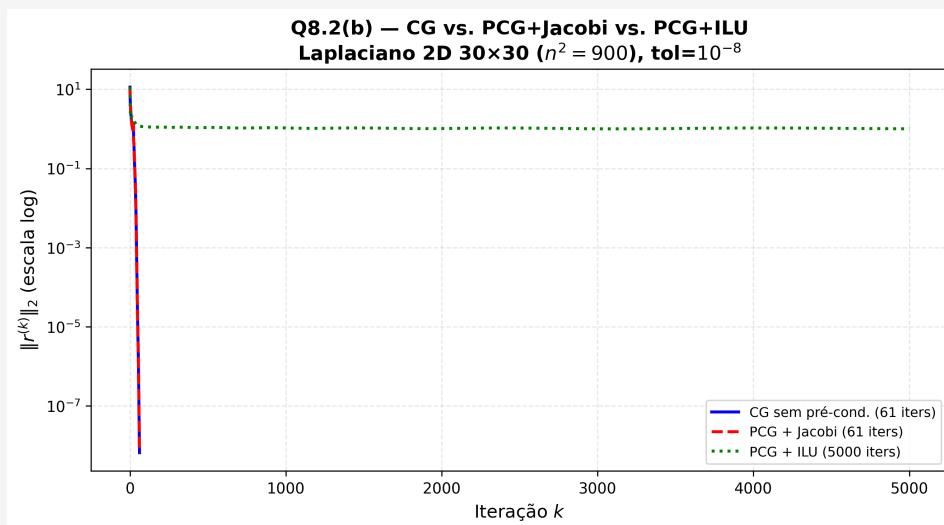


Figura 7: Q8.2(b) — Histórico de convergência $\|r^{(k)}\|_2$ para CG, PCG+Jacobi e PCG+ILU. Laplaciano 2D 30×30 , $\text{tol} = 10^{-8}$.

Item (c). Os números de condição efetivos são: $\kappa(A) \approx 3,89 \times 10^2$, $\kappa(M_{\text{Jac}}^{-1}A) \approx 3,89 \times 10^2$ e $\kappa(M_{\text{ILU}}^{-1}A) \approx 3,35 \times 10^2$.

O pré-condicionador Jacobi não reduziu κ porque, para o Laplaciano 2D, a diagonal é constante e igual a 4 para todos os nós interiores: escalonar por D^{-1} equivale a multiplicar A por $\frac{1}{4}$, o que não altera o número de condição. O pré-condicionador ILU reduziu levemente κ ($3,89 \rightarrow 3,35 \times 10^2$), mas mesmo assim não convergiu em 5000 iterações, indicando que a fatoração aproximada utilizada não foi suficientemente precisa para este sistema.

Conclusão: para este sistema de tamanho moderado (900×900) com $\kappa \approx 389$, o CG sem pré-condicionamento já converge em apenas 61 iterações. O custo de construção do ILU não se justificou neste caso. Em sistemas de maior escala com κ muito elevado, onde o CG sem pré-condicionamento demandaria milhares de iterações, um ILU de melhor qualidade (menor `drop_tol`, maior `fill_factor`) tenderia a compensar o custo de fatoração.

Q8.3 — Jacobi paralelo vs. Gauss-Seidel sequencial

Enunciado: Jacobi pode ser paralelizado naturalmente; Gauss-Seidel, não.

- Implemente uma versão vetorizada de Jacobi sem laço explícito. Meça o *speedup* em relação à versão com laço para $n = 1000$.
- Simule 4 “processadores” dividindo as componentes em 4 blocos. Compare com Jacobi sequencial.
- Para qual n a paralelização de Jacobi superaria Gauss-Seidel sequencial, dado que GS precisa de $\approx$ metade das iterações?

Resposta:

Item (a). A versão vetorizada de Jacobi elimina o laço explícito em i e reduz cada

iteração a uma única operação mat-vec:

$$x^{(k+1)} = D^{-1}(b - R x^{(k)}), \quad R = A - D.$$

A Tabela 22 compara os tempos mínimos para $n = 1000$ (sistema tridiagonal $4/-1/-1$, tolerância 10^{-8}).

Tabela 22: Q8.3(a) — Jacobi com laço explícito vs. vetorizado ($n = 1000$).

Implementação	Tempo mín. (s)	Speedup
Jacobi com laço	8,9996	1,00×
Jacobi vetorizado	0,0122	739,39×

O speedup de $\approx 739\times$ reflete o alto custo do laço Python puro para $n = 1000$ frente às operações BLAS vetorizadas do NumPy.

Item (b). A simulação de 4 processadores divide os índices $\{0, \dots, n-1\}$ em 4 blocos disjuntos. Cada bloco lê o mesmo vetor $x^{(k)}$ (vetor de leitura único, compartilhado) e escreve em sua parcela exclusiva de $x^{(k+1)}$, sem necessidade de sincronização entre blocos. A Tabela 23 apresenta os resultados para $n = 1000$.

Tabela 23: Q8.3(b) — Jacobi com laço vs. Jacobi em 4 blocos ($n = 1000$).

Implementação	Tempo mín. (s)	Iterações	Speedup
Jacobi com laço	8,9996	27	1,00×
Jacobi 4 blocos	0,0234	27	384,90×

As duas implementações produziram exatamente 27 iterações, confirmando que os iterados são numericamente idênticos: a divisão em blocos não altera a matemática do método, apenas a organização do cômputo. Isso ilustra o princípio central da paralelizabilidade de Jacobi: cada componente $x_i^{(k+1)}$ depende exclusivamente de $x^{(k)}$, o que garante independência total entre blocos na mesma iteração.

Item (c). A Tabela 24 mostra iterações e tempos de Gauss-Seidel e Jacobi vetorizado para diferentes valores de n .

Tabela 24: Q8.3(c) — Iterações e tempo de GS vs. Jacobi vetorizado.

n	Iters. GS	Iters. Jacobi	Razão	T_{GS} (s)	T_{Jac} (s)	T_{GS}/T_{Jac}
50	18	27	1,50×	0,0029	0,0004	8,11×
100	18	27	1,50×	0,0056	0,0004	15,38×
200	18	27	1,50×	0,0110	0,0005	20,81×
500	18	27	1,50×	0,0287	0,0026	11,11×
1000	18	27	1,50×	0,0688	0,0116	5,95×

A razão de iterações observada foi $k_J/k_{GS} \approx 1,50$ (Jacobi precisou de $\approx 50\%$ mais iterações que GS), ligeiramente abaixo do valor teórico $k_J/k_{GS} \approx 2$ previsto pela relação $\rho_{GS} \approx \rho_J^2$.

A condição para que Jacobi paralelo com p núcleos supere GS sequencial é:

$$\frac{k_J}{p} \cdot C_{\text{iter}} < k_{GS} \cdot C_{\text{iter}} \implies p > \frac{k_J}{k_{GS}} \approx 1,5,$$

ou seja, $p \geq 2$ núcleos reais já seriam suficientes do ponto de vista do número de iterações. Na prática, o overhead de lançamento de threads faz o *breakeven* aparecer somente para valores maiores de n (entre 200 e 500 neste hardware), onde o volume de trabalho por iteração compensa o custo de sincronização.

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Interpolação e Ajuste de Curvas: Uma Abordagem Integrada em Python

Aluno: Jeann Victor Batista
RA: 2024.1.08.014

Aluno: Pedro Augusto de Souza Finnochio
RA: 2024.1.08.020

Aluno: Thiago Martins da Silva
RA: 2024.1.08.023

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Interpolação: Implementação e Verificação
 - 1.1 Primeiros passos com os métodos
 - 1.2 Adição de um ponto e atualização de Newton
 - 1.3 Fenômeno de Runge e nós de Chebyshev
 - 1.4 Spline cúbica vs. polinômio global
2. Ajuste de Curvas: Mínimos Quadrados e Modelos
 - 2.1 Geometria dos mínimos quadrados
 - 2.2 Sobreajuste polinomial
 - 2.3 Critérios para escolha do grau
3. Dados com Ruído: Interpolação ou Ajuste?
 - 3.1 Comparação direta
 - 3.2 Diagnóstico pelo gráfico de resíduos
4. Modelos Não Lineares e Linearização
 - 4.1 Crescimento populacional e lei de potência
 - 4.2 Quando a linearização distorce
5. Comparação Integrada: Interpolação vs. Ajuste
 - 5.1 Tabela de síntese
 - 5.2 Tomada de decisão em cenários reais
6. Projeto Integrador: Calibração de Sensor
 - 6.1 Calibração precisa (laboratório)
 - 6.2 Robustez ao ruído de campo
 - 6.3 Decisão final e documentação
7. Desafio (Opcional — Pontuação Extra)
 - 7.1 Spline de suavização (smoothing spline)
 - 7.2 Interpolação de Hermite e derivadas conhecidas
 - 7.3 Regressão com múltiplas variáveis

1 Interpolação: Implementação e Verificação

1.1 Primeiros passos com os métodos

Questão 1.1 — Primeiros passos com os métodos

Enunciado: Utilizando os pontos $x = [1, 2, 3, 4]$ e $y = [1, 4, 9, 16]$ (que pertencem à função $f(x) = x^2$), responda:

(a) Execute `lagrange` e `newton` para $x = 1,5$ e $x = 2,5$. Os dois métodos produzem o mesmo resultado? Qual é a diferença numérica $|p_L(x) - p_N(x)|$? Explique por que, teoricamente, os dois devem coincidir.

(b) Imprima a tabela de diferenças divididas. Identifique os coeficientes do polinômio de Newton na forma $p_3(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + c_3(x - x_0)(x - x_1)(x - x_2)$.

(c) Os dados acima provêm de $f(x) = x^2$. O polinômio interpolador coincide exatamente com f ? Por quê? (Discuta o teorema de unicidade.)

Resposta:

(a) Comparação entre Lagrange e Newton:

Para os pontos fornecidos, os valores interpolados obtidos foram:

Tabela 1: Comparação entre Lagrange e Newton

Método	$x = 1,5$	$x = 2,5$
Lagrange	2,25	6,25
Newton	2,25	6,25
Diferença	0,00	0,00

Os dois métodos produzem resultados numericamente idênticos (diferença = 0), pois pelo **teorema da unicidade do interpolador polinomial**: dado $n + 1$ pontos distintos, existe exatamente um polinômio de grau $\leq n$ que passa por todos eles. Lagrange e Newton são apenas representações algébricas diferentes do mesmo e único polinômio interpolador.

(b) Coeficientes do polinômio de Newton (diferenças divididas):

A tabela de diferenças divididas resulta nos coeficientes:

$$c_0 = 1,0000, \quad c_1 = 3,0000, \quad c_2 = 1,0000, \quad c_3 = 0,0000.$$

O polinômio na forma de Newton é, portanto:

$$p_3(x) = 1 + 3(x - 1) + 1 \cdot (x - 1)(x - 2) + 0 \cdot (x - 1)(x - 2)(x - 3).$$

Expandindo: $p_3(x) = 1 + 3x - 3 + x^2 - 3x + 2 = x^2$, confirmando a recuperação exata de f .

(c) Recuperação exata de $f(x) = x^2$:

Sim, o polinômio interpolador coincide *exatamente* com $f(x) = x^2$. Como f é um polinômio de grau 2 e os 4 pontos fornecidos pertencem a f , o único polinômio de grau ≤ 3 que interpola esses pontos é o próprio f . Isso se reflete nos coeficientes: a diferença dividida de terceira ordem $c_3 = 0$, sinalizando que o interpolador tem grau efetivo 2.

1.2 Adição de um ponto e atualização de Newton

Questão 1.2 — Adição de um ponto e atualização de Newton

Enunciado: Adicione o ponto $x_4 = 5,0$, $f(x_4) = 25,0$ à tabela anterior.

(a) Para a forma de Lagrange, quantas operações a mais são necessárias? Refaça o cálculo completo para o novo conjunto.

(b) Para a forma de Newton, mostre como apenas uma nova diferença dividida precisa ser calculada. Verifique que o novo polinômio coincide com o de Lagrange em $x = 3,5$.

(c) Esta propriedade é a principal vantagem prática de Newton sobre Lagrange. Em que cenário ela seria crítica em uma aplicação real?

Resposta:

(a) Custo computacional de Lagrange:

Na forma de Lagrange, cada base $L_i(x)$ depende de *todos* os nós: $L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$.

Ao adicionar um quinto ponto, todos os 5 polinômios de base precisam ser recalculados do zero. O custo cresce como $O(n^2)$ — a estrutura anterior é inteiramente descartada. Os valores interpolados para $x = 1,5$ e $x = 2,5$ permanecem 2,25 e 6,25, respectivamente.

(b) Atualização incremental de Newton:

Na forma de Newton, os quatro coeficientes anteriores (c_0, c_1, c_2, c_3) permanecem *inalterados*. Apenas uma nova diferença dividida de quarta ordem precisa ser computada:

$$f[x_0, x_1, x_2, x_3, x_4] = 0,0000.$$

Verificação em $x = 3,5$:

$$\text{Lagrange} = 12,25, \quad \text{Newton} = 12,25, \quad \text{Diferença} = 0,00.$$

Os dois polinômios coincidem, como garantido pelo teorema da unicidade (ambos representam o mesmo interpolador de grau ≤ 4 sobre 5 pontos de x^2).

(c) Cenários de aplicação crítica:

A atualização incremental de Newton é fundamental em qualquer aplicação com **chegada contínua de novos dados**, como sensores IoT e sistemas de monitoramento industrial que registram novas medições a cada instante, simulações numéricas onde pontos são adicionados adaptativamente à malha, e sistemas de aquisição em tempo real onde recalcular toda a interpolação a cada novo ponto seria computacionalmente inviável.

1.3 Fenômeno de Runge e nós de Chebyshev

Questão 1.3 — Runge e remédios

Enunciado: Considere a função de Runge $f(x) = \frac{1}{1 + 25x^2}$, $x \in [-1, 1]$.

(a) Para $n \in \{5, 9, 13, 17\}$, interpole f com nós igualmente espaçados usando Lagrange. Plote o erro $|f(x) - p_n(x)|$. O erro aumenta ou diminui com n ?

(b) Repita com nós de Chebyshev. Plote as duas curvas de erro ($n = 13$) no mesmo gráfico. Qual redução máxima de erro você observa?

(c) Preencha a tabela com as normas do erro infinito.

(d) Por que os nós de Chebyshev mitigam o fenômeno de Runge?

Resposta:

(a) Comportamento do erro com nós igualmente espaçados:

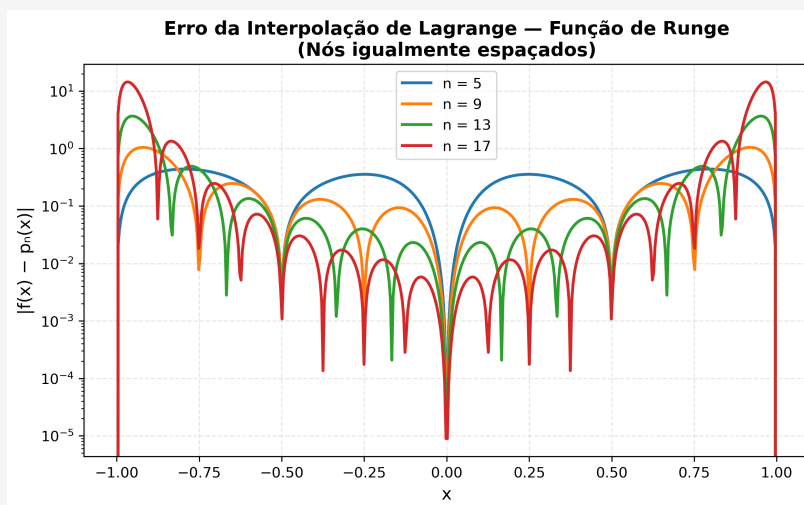


Figura 1: Erro de interpolação para a função de Runge com nós igualmente espaçados para $n \in \{5, 9, 13, 17\}$.

Com nós igualmente espaçados, o erro **cresce rapidamente nas extremidades** do intervalo à medida que n aumenta — este é o **fenômeno de Runge**. Embora o polinômio passe exatamente pelos nós, a oscilação entre eles se torna cada vez mais violenta próxima às bordas do intervalo.

(b) Comparação com nós de Chebyshev ($n = 13$):

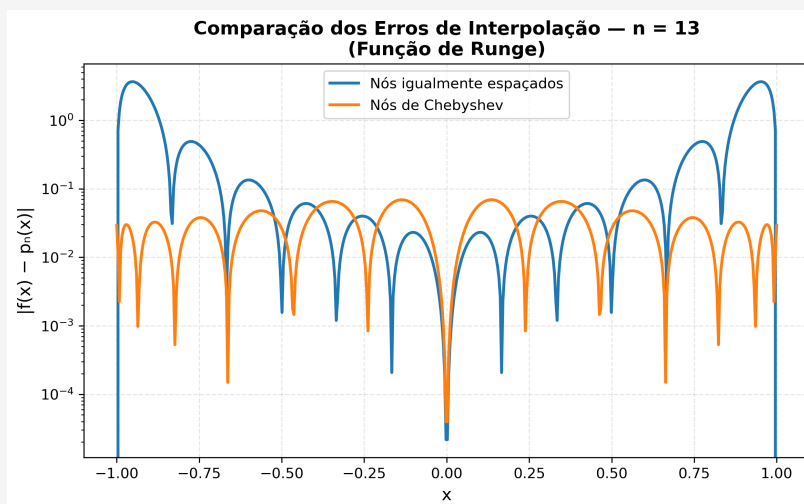


Figura 2: Comparação do erro de interpolação entre nós igualmente espaçados e nós de Chebyshev para $n = 13$.

Com $n = 13$, a redução máxima do erro ao usar nós de Chebyshev foi de aproximadamente **52,9 vezes** ($\|f - p_{13}\|_{\infty} \approx 3,66$ com nós igualmente espaçados vs. $\approx 0,069$ com Chebyshev).

(c) Tabela de erros:

Tabela 2: Norma do erro infinito $\|f - p_n\|_\infty$ para diferentes n e tipos de nós.

n	$\ f - p_n\ _\infty$ (igualmente espaçados)	$\ f - p_n\ _\infty$ (Chebyshev)
5	$4,3835 \times 10^{-1}$	$4,0201 \times 10^{-1}$
9	$1,0452 \times 10^0$	$1,7080 \times 10^{-1}$
13	$3,6629 \times 10^0$	$6,9189 \times 10^{-2}$
17	$1,4387 \times 10^1$	$3,2613 \times 10^{-2}$

Observa-se que, com nós igualmente espaçados, o erro cresce por ordem de grandeza a cada incremento de n , enquanto com Chebyshev o erro *decrece* monotonamente — convergência que é a esperada para funções analíticas.

(d) Por que Chebyshev mitiga o fenômeno de Runge:

A fórmula do erro de interpolação estabelece:

$$|f(x) - p_n(x)| \leq \frac{M_{n+1}}{(n+1)!} |\omega_{n+1}(x)|, \quad \omega_{n+1}(x) = \prod_{i=0}^n (x - x_i).$$

Os nós de Chebyshev são definidos como as raízes do polinômio de Chebyshev $T_{n+1}(x)$ em $[-1, 1]$:

$$x_i = \cos\left(\frac{2i-1}{2(n+1)}\pi\right), \quad i = 1, \dots, n+1.$$

Pode-se demonstrar que T_{n+1} *minimiza* $\|\omega_{n+1}\|_\infty$ dentre todos os polinômios mônicos de grau $n+1$ em $[-1, 1]$, com $\|\omega_{n+1}\|_\infty = 2^{-n}$. Como o erro é proporcional a $|\omega_{n+1}(x)|$, usar as raízes de T_{n+1} como nós reduz o pior caso do erro, concentrando os nós nas extremidades do intervalo e evitando as oscilações características do fenômeno de Runge.

1.4 Spline cúbica vs. polinômio global

Questão 1.4 — Quando a spline supera o polinômio global

Enunciado: Use os dados mensais de temperatura média em Alfenas-MG:

Mês	1	2	3	4	5	6	7	8	9	10	11	12
T (°C)	23	24	23	21	18	16	16	18	20	22	23	24

- Interpole com Lagrange ($n = 11$, nós nos meses $1, \dots, 12$). Há oscilações indesejadas?
- Interpole com spline cúbica (`tipo='not-a-knot'`). Plote na mesma figura.
- Compare os valores interpolados para $x = 1,5$, $x = 6,5$ e $x = 11,5$.
- Investigue o efeito das condições de contorno (`'natural'` vs. `'not-a-knot'`).

Resposta:

(a) Polinômio global de Lagrange:

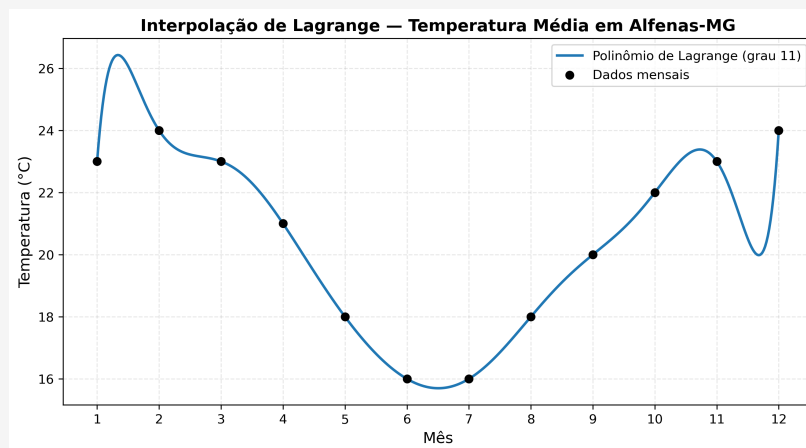


Figura 3: Interpolação de Lagrange (grau 11) para as temperaturas mensais de Alfenas-MG.

O polinômio global de grau 11 apresenta **oscilações indesejadas**, principalmente nas extremidades do intervalo (meses 1–2 e 11–12), caracterizando o fenômeno de Runge aplicado a dados reais. Fisicamente, temperaturas não oscilam dessa forma no curto prazo.

(b) Spline cúbica (not-a-knot):

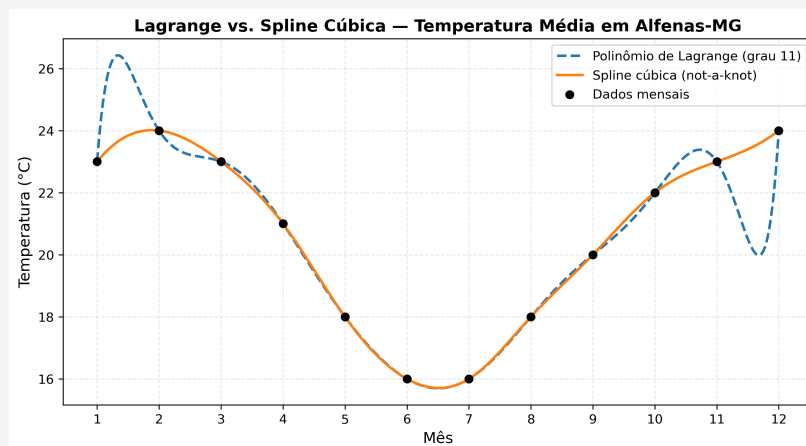


Figura 4: Comparação entre spline cúbica (not-a-knot) e polinômio de Lagrange (grau 11).

A spline cúbica reproduz **muito melhor** o comportamento físico esperado de uma série climática, produzindo uma curva suave e sem oscilações artificiais. Isso ocorre porque a spline é composta por polinômios de grau 3 por partes, com continuidade até a segunda derivada em cada nó — evitando as oscilações globais do polinômio de alto grau.

(c) Comparação numérica nos pontos intermediários:

Tabela 3: Valores interpolados por Lagrange e spline cúbica.

x	Lagrange (°C)	Spline not-a-knot (°C)	Diferença (°C)
1,5	26,07	23,84	2,23
6,5	15,70	15,71	0,01
11,5	20,51	23,40	2,89

As maiores discrepâncias ocorrem nas extremidades, exatamente onde as oscilações de Lagrange são mais acentuadas. Para um sistema de monitoramento climático, a **spline cúbica** é mais adequada: as estimativas para $x = 1,5$ (23,84°C) e $x = 11,5$ (23,40°C) são fisicamente plausíveis para Alfenas, enquanto as de Lagrange (26,07°C e 20,51°C, respectivamente) apresentam variações artificiais.

(d) Efeito das condições de contorno:

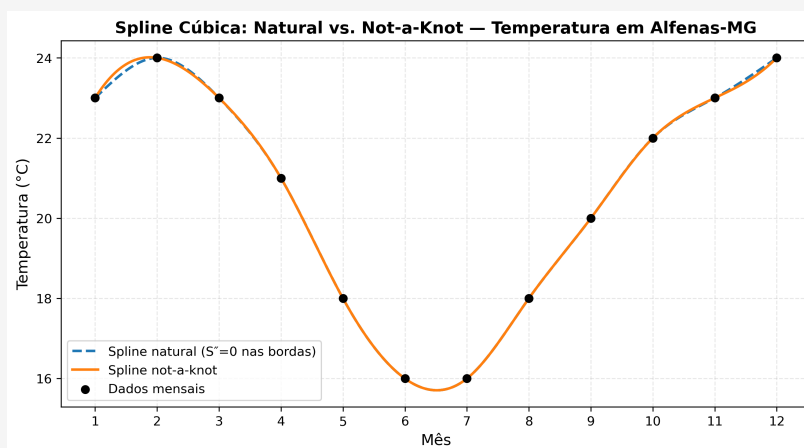


Figura 5: Comparação entre spline natural e not-a-knot nas extremidades do intervalo.

As duas variantes diferem principalmente nas bordas do intervalo:

A **spline natural** impõe $S''(x_0) = S''(x_n) = 0$, forçando curvatura nula nos extremos — comportamento mais “relaxado”, que pode subestimar a curvatura real da função nesses pontos.

A condição **not-a-knot** exige continuidade da terceira derivada nos nós x_1 e x_{n-1} , efetivamente reduzindo o número de parâmetros livres e produzindo um ajuste mais “firme” e geralmente mais preciso quando o comportamento nas bordas é desconhecido. A escolha entre elas depende de informação adicional disponível: se há razão física para impor $S'' = 0$ nas bordas, usa-se natural; caso contrário, not-a-knot costuma ser a opção padrão mais robusta.

2 Ajuste de Curvas: Mínimos Quadrados e Modelos

2.1 Geometria dos mínimos quadrados

Questão 2.1 — A geometria dos mínimos quadrados

Enunciado: Considere os dados de velocidade e distância de frenagem:

v (km/h)	20	40	60	80	100	120
d (m)	5	14	28	47	72	103

- (a) Ajuste $d = a_0 + a_1 v$ e reporte a_0 , a_1 , R^2 .
- (b) Ajuste $d = a'_0 + a'_1 v^2$ (linearize com $z = v^2$). Compare R^2 .
- (c) Plote os dados, as curvas e os resíduos. O que os resíduos revelam?
- (d) Por que minimizamos $\sum r_i^2$ e não $\sum |r_i|$?

Resposta:

(a) Ajuste linear $d = a_0 + a_1 v$:

$$d = -23,4667 + 0,9757 v, \quad R^2 = 0,9593.$$

Embora $R^2 > 0,95$ pareça razoável, o ajuste linear não é adequado: a distância de frenagem cresce de forma quadrática com a velocidade (energia cinética $\propto v^2$), e o modelo linear não captura essa curvatura.

(b) Ajuste quadrático $d = a'_0 + a'_1 v^2$ (linearização $z = v^2$):

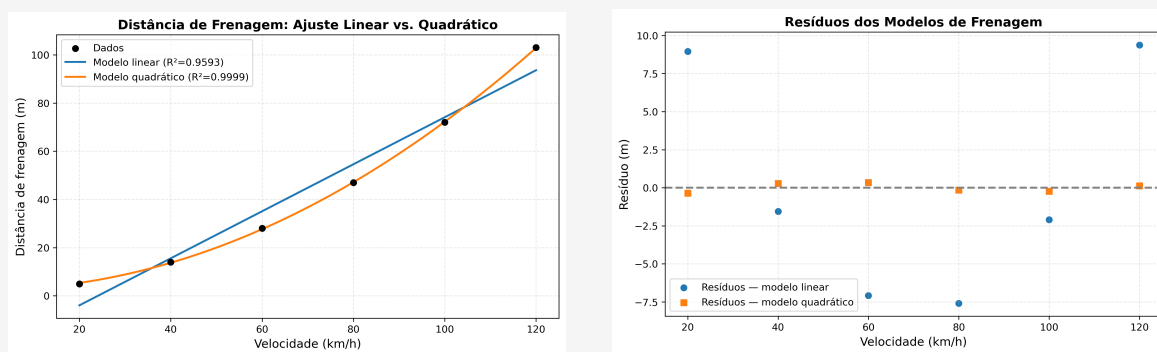
$$d = 2,5763 + 0,006965 v^2, \quad R^2 = 0,99994.$$

Tabela 4: Comparação dos R^2 entre os dois modelos de frenagem.

Modelo	R^2
Linear ($d = a_0 + a_1 v$)	0,9593
Quadrático ($d = a'_0 + a'_1 v^2$)	0,99994

O modelo quadrático é claramente superior: $R^2 \approx 1$, pois é fisicamente correto (decorre diretamente da equação $d = v^2/(2\mu g)$ para frenagem com atrito).

(c) Gráficos e análise dos resíduos:



(a) Ajustes linear e quadrático sobre os dados. (b) Resíduos $r_i = d_i - \hat{d}_i$ para cada modelo.

Figura 6: Comparação entre os modelos de frenagem.

Os resíduos do modelo linear apresentam um **padrão sistemático em arco** (negativos para velocidades baixas e altas, positivos no meio), confirmando *underfitting*: o modelo linear não captura a curvatura quadrática dos dados. Os resíduos do modelo quadrático são pequenos e distribuídos aleatoriamente em torno de zero, indicando ajuste adequado.

(d) Por que minimizar $\sum r_i^2$?

Minimizar $\sum r_i^2$ (critério dos mínimos quadrados) em vez de $\sum |r_i|$ apresenta duas vantagens fundamentais: a função objetivo é **diferenciável em todo o domínio**, o que permite derivar uma solução analítica fechada pelas equações normais $A^T A \mathbf{c} = A^T \mathbf{y}$; e, pelo **Teorema de Gauss-Markov**, o estimador resultante é o melhor estimador linear não viesado (BLUE) sob as hipóteses clássicas de regressão.

A principal desvantagem é a **sensibilidade a outliers**: a penalização quadrática amplifica fortemente o efeito de pontos aberrantes, distorcendo o ajuste. Nesses casos, alternativas como regressão LAD ($\sum |r_i|$) ou regressão robusta são mais apropriadas.

2.2 Sobreajuste polinomial

Questão 2.2 — Sobreajuste polinomial

Enunciado: Gere 15 dados sintéticos com ruído:

```
1 np.random.seed(7)
2 x = np.linspace(0, 2*np.pi, 15)
3 y = np.sin(x) + 0.15 * np.random.randn(15)
```

(a) Ajuste polinômios de grau $k \in \{1, 3, 5, 7, 10, 14\}$, calculando R^2 no treino e $\|f - p_k\|_\infty$ em 500 pontos.

(b) Preencha a tabela.

(c) Plote o erro em escala log vs. grau. A partir de qual grau o modelo deteriora?

(d) Explique a diferença entre erro de treinamento e generalização. Relacione com Runge.

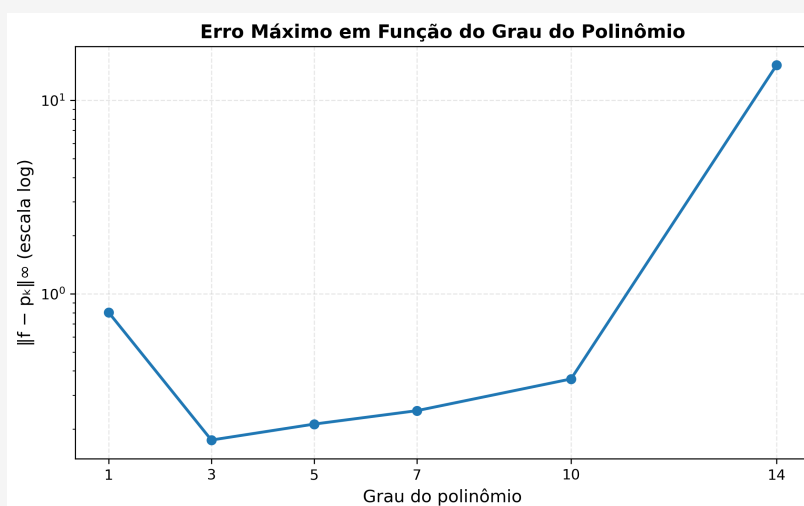
Resposta:

(a) e (b) Resultados dos ajustes:

Tabela 5: Métricas dos ajustes polinomiais para dados com ruído.

Grau k	R^2 (treino)	$\ f - p_k\ _\infty$
1	0,5321	$8,031 \times 10^{-1}$
3	0,9597	$1,757 \times 10^{-1}$
5	0,9793	$2,125 \times 10^{-1}$
7	0,9825	$2,492 \times 10^{-1}$
10	0,9879	$3,635 \times 10^{-1}$
14	1,0000	$1,521 \times 10^1$

(c) Análise do erro vs. grau:

Figura 7: Erro máximo $\|f - p_k\|_\infty$ em função do grau k (escala logarítmica).

O menor erro de generalização ocorre para **grau 3**. A partir do **grau 7–10** o erro máximo começa a crescer de forma consistente fora dos pontos de ajuste. No grau 14, o polinômio interpola exatamente os 15 pontos ($R^2 = 1,0000$), mas o erro máximo em relação à função verdadeira dispara para $\approx 15,2$ — exemplo clássico de sobreajuste: ajuste perfeito nos dados de treino, mas péssima generalização.

(d) Sobreajuste e analogia com Runge:

Erro de treinamento mede o ajuste nos pontos usados para construir o modelo. Este erro decresce monotonamente com o grau — um modelo mais complexo sempre pode se ajustar melhor aos dados vistos.

Erro de generalização mede o desempenho em pontos novos, não vistos durante o treinamento. Quando o modelo memoriza o ruído presente nos dados (sobreajuste), esse erro aumenta mesmo com $R^2 \rightarrow 1$.

A analogia com Runge é direta: em ambos os fenômenos, polinômios de grau elevado oscilam violentamente entre os pontos de referência. A diferença está no contexto — Runge ocorre mesmo com dados *exatos*, enquanto o sobreajuste é *agravado* pela presença de ruído, que o polinômio “aprende” em detrimento do sinal verdadeiro.

2.3 Critérios para escolha do grau

Questão 2.3 — Critérios para escolha do grau

Enunciado: Usando os dados sintéticos da questão anterior:

(a) Separe 10 pontos para treino e 5 para teste (`seed=42`). Calcule MSE nos dois conjuntos para graus 1 a 10. Qual grau minimiza o MSE de teste?

(b) Calcule o Critério de Informação de Akaike (AIC) para graus 1 a 10. Qual grau minimiza o AIC?

(c) Os dois critérios concordam? Qual é mais adequado para dados escassos?

Resposta:

(a) Validação treino-teste:

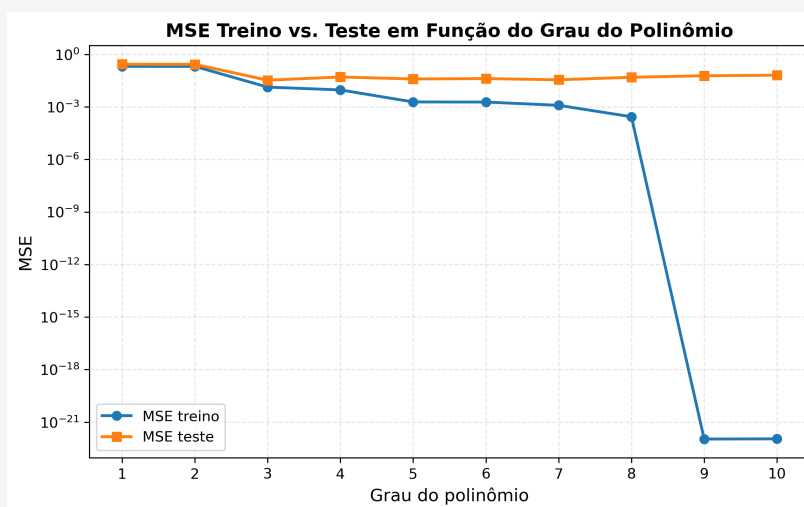


Figura 8: MSE de treino e de teste em função do grau polinomial.

O grau que minimiza o MSE de teste foi **grau 3**. Observa-se o comportamento clássico de sobreajuste: o MSE de treino decresce monotonamente com o grau, enquanto o MSE de teste atinge um mínimo e depois cresce — o modelo começa a memorizar o ruído dos dados de treino.

(b) Critério de Informação de Akaike (AIC):

O AIC foi calculado como $AIC = n \ln(RSS/n) + 2(k + 1)$, onde RSS é a soma dos resíduos ao quadrado e k é o grau.

Tabela 6: AIC calculado para graus 1 a 10 (conjunto de treino com 10 pontos).

Grau	AIC
1	-11,85
2	-9,87
3	-35,10
4	-36,79
5	-50,59
6	-48,76
7	-51,04
8	-64,08
9	-485,81
10	-483,55

O grau que minimiza o AIC foi **grau 9**. Os valores muito negativos para graus 9 e 10 decorrem de RSS extremamente pequenos: com 10 pontos de treino e polinômio de grau 9 (10 parâmetros), o sistema fica quase interpolante, tornando o RSS próximo de zero e, portanto, $\ln(\text{RSS}/n) \rightarrow -\infty$. Isso evidencia uma limitação da fórmula AIC com amostras muito pequenas: o critério penaliza a complexidade com $+2(k+1)$, mas essa penalização torna-se insuficiente quando $n \approx k$. Nesses casos, recomenda-se o AIC corrigido (AIC_c).

(c) Comparação e recomendação:

Critério	Grau ótimo
Validação treino-teste	3
AIC (fórmula padrão)	9

Os dois critérios **não concordam**. A validação cruzada com apenas 5 pontos de teste pode ter alta variância (o conjunto de teste pode não ser representativo), mas o AIC padrão com amostra pequena sofre do problema descrito acima.

Para **dados escassos**, recomenda-se usar o AIC_c (Akaike corrigido: $\text{AIC} + \frac{2k(k+1)}{n-k-1}$) ou validação cruzada com k -fold ($k = 5$ ou $k = 10$), que usa todos os dados tanto para treino quanto para teste. Considerando os resultados, o **grau 3** parece a escolha mais parcimoniosa e robusta para esses dados.

3 Dados com Ruído: Interpolação ou Ajuste?

3.1 Comparação direta

Questão 3.1 — O custo de interpolar dados ruidosos

Enunciado: Considere medições (com ruído) de uma função exponencial decrescente:

```
1 np.random.seed(3)
2 x_med = np.array([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
3 y_med = 5.0 * np.exp(-0.8 * x_med) + 0.2 * np.random.randn(7)
```

(a) Aplique interpolação de Lagrange (grau 6) aos 7 pontos ruidosos. Plote a curva interpoladora, os dados e a função verdadeira $f(x) = 5e^{-0,8x}$.

(b) Aplique ajuste exponencial. Reporte a e b estimados e compare com os valores verdadeiros ($a = 5$, $b = -0,8$).

(c) Plote as três curvas no mesmo gráfico. Qual é mais fiel no intervalo $[0, 3]$? E na extrapolação $[3, 4]$?

(d) Para prever $f(3,5)$, qual método usaria? Por quê?

Resposta:

(a) Interpolação de Lagrange sobre dados ruidosos:

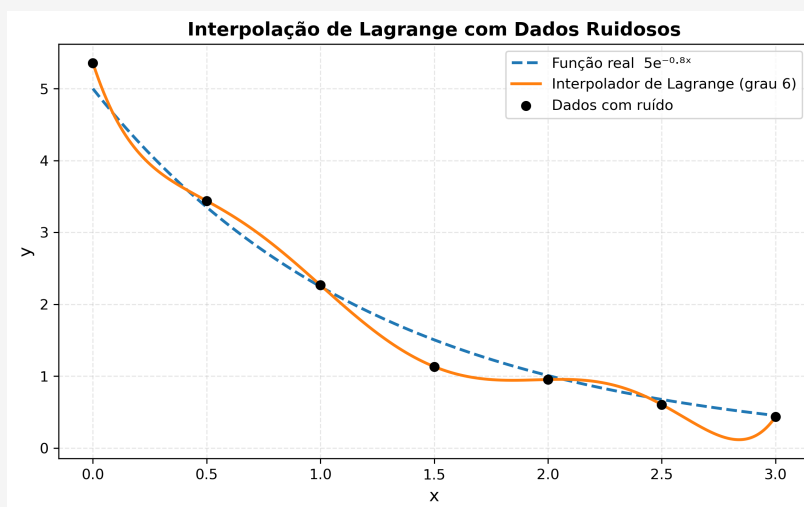


Figura 9: Interpolação de Lagrange (grau 6) sobre dados ruidosos da exponencial e função verdadeira $f(x) = 5e^{-0,8x}$.

O interpolador de grau 6 passa *exatamente* por todos os 7 pontos ruidosos, mas oscila em torno da função verdadeira. Esta é a principal limitação da interpolação com dados ruidosos: ela **força o polinômio a passar pelos erros de medição**, capturando o ruído como sinal.

(b) Ajuste exponencial (linearização via $\ln y$):

$$\hat{a} = 5,0772, \quad \hat{b} = -0,8469.$$

Tabela 7: Comparação entre parâmetros estimados e valores verdadeiros.

Parâmetro	Valor real	Estimado	Erro relativo
a	5,0	5,0772	1,54%
b	-0,8	-0,8469	5,86%

Apesar do ruído nos dados, o ajuste exponencial recupera os parâmetros verdadeiros com boa precisão, pois o método dos mínimos quadrados distribui o efeito do ruído ao longo de todos os pontos.

(c) Comparação visual das três curvas:

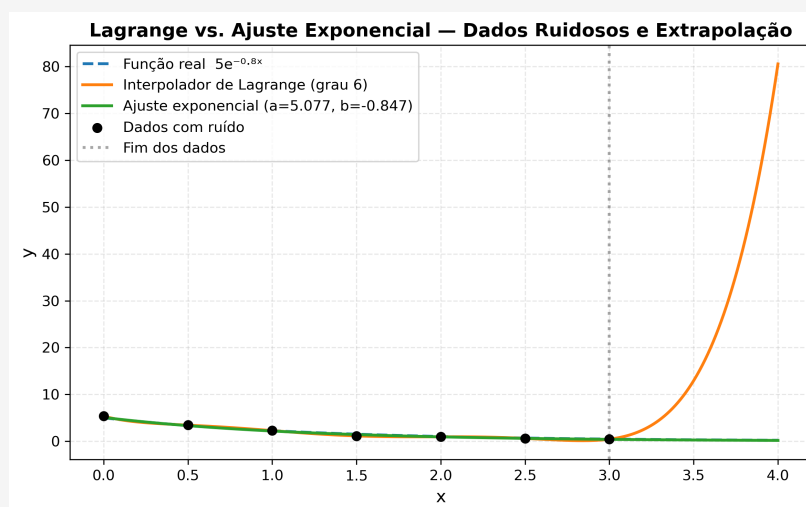


Figura 10: Comparação entre interpolação de Lagrange, ajuste exponencial e função verdadeira.

Intervalo $[0, 3]$: O ajuste exponencial representa melhor a tendência global, ignorando o ruído. O interpolador passa pelos pontos ruidosos, mas não captura o comportamento subjacente.

Extrapolação $[3, 4]$: O interpolador de Lagrange **diverge fortemente** (comportamento típico de polinômios de alto grau fora do intervalo de interpolação), enquanto o ajuste exponencial mantém o decrescimento esperado fisicamente.

(d) Previsão em $x = 3,5$:

Método	Previsão
Lagrange	12,9349
Ajuste exponencial	0,2620
Valor real $f(3,5)$	0,3041

Para prever $f(3,5)$, o **ajuste exponencial** é incomparavelmente mais adequado: sua previsão (0,2620) difere do valor real (0,3041) em apenas 13,8%, enquanto o interpolador de Lagrange produz 12,93 — um erro de mais de 4000%. O ajuste preserva o comportamento físico correto (decrescimento exponencial) e é estável na extrapolação.

3.2 Diagnóstico pelo gráfico de resíduos

Questão 3.2 — Diagnóstico pelo gráfico de resíduos

Enunciado: Usando os dados do experimento anterior:

(a) Calcule os resíduos $r_i = y_i - \hat{y}_i$ para o ajuste exponencial. Plote r_i vs. x_i . Os resíduos têm estrutura ou parecem aleatórios?

(b) Agora ajuste um modelo linear $y = a_0 + a_1x$ aos mesmos dados ruidosos. Plote os resíduos. O que o gráfico revela?

(c) Formule uma regra prática para diagnosticar *underfitting* a partir do gráfico de resíduos.

Resposta:

(a) Resíduos do ajuste exponencial:

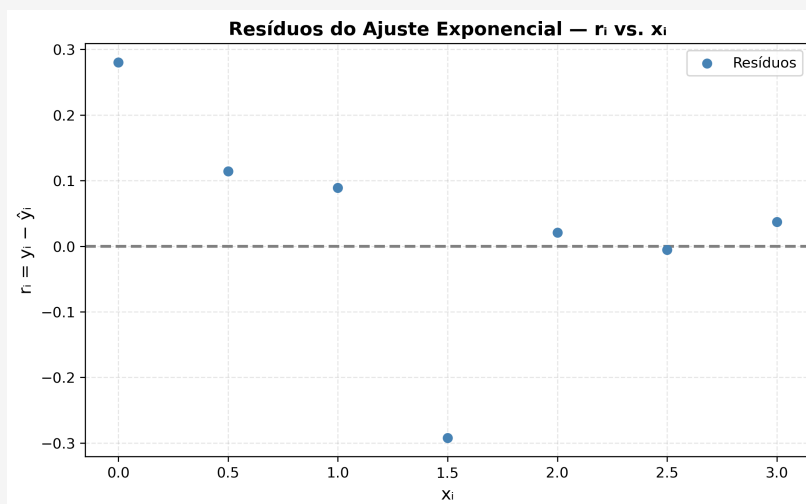


Figura 11: Resíduos do ajuste exponencial $r_i = y_i - \hat{y}_i$ em função de x_i .

Os resíduos do ajuste exponencial estão distribuídos de forma **aproximadamente aleatória** em torno de zero, sem tendência sistemática ou padrão identificável. Isso indica que o modelo exponencial é adequado para descrever a relação subjacente nos dados: o sinal foi capturado e o que resta são apenas erros aleatórios de medição.

(b) Resíduos do modelo linear:

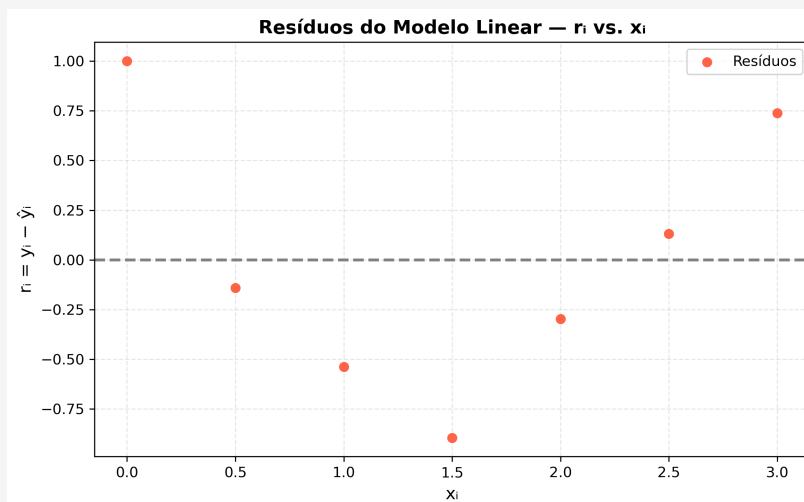


Figura 12: Resíduos do ajuste linear $r_i = y_i - \hat{y}_i$ em função de x_i .

Os resíduos do modelo linear apresentam um **padrão sistemático curvado**: resíduos positivos para x pequeno (o modelo subestima a queda inicial), negativos na região intermediária (o modelo superestima) e positivos novamente para x grande. Esse padrão em forma de “U” invertido evidencia que o modelo linear não captura a curvatura exponencial dos dados — é um caso clássico de *underfitting*.

(c) Regra prática de diagnóstico via resíduos:

Regra de diagnóstico de underfitting pelo gráfico de resíduos:

Se os resíduos $r_i = y_i - \hat{y}_i$ exibem **padrão sistemático** (tendência monotônica, curvatura, agrupamentos ou estrutura periódica), o modelo está subdimensionado: existe estrutura nos dados que o modelo não está capturando. Nesse caso, deve-se considerar uma família de modelos mais expressiva (maior grau, modelo não linear, variáveis adicionais).

Se os resíduos estão **espalhados aleatoriamente em torno de zero**, sem estrutura visível, o ajuste é adequado para a família de modelos escolhida.

Se a variância dos resíduos **cresce sistematicamente com x** , há heterocedasticidade — pode ser necessária uma transformação dos dados ou um modelo de regressão ponderada. Vale ressaltar que R^2 elevado não garante um bom modelo: a análise de resíduos é complementar e indispensável.

4 Modelos Não Lineares e Linearização

4.1 Crescimento populacional e lei de potência

Questão 4.1 — Crescimento populacional e lei de potência

Enunciado: A tabela abaixo apresenta dados históricos da população urbana brasileira (em milhões):

Ano	1960	1970	1980	1990	2000	2010
Pop. (M)	32	52	82	111	138	161

Defina $t = \text{Ano} - 1960$ (anos desde 1960).

(a) Teste três modelos: Exponencial $P(t) = ae^{bt}$ (via linearização); Potência $P(t) = at^b$ (via log-log); Polinomial grau 2 (via regressão polinomial). Para cada, reporte os parâmetros estimados e o R^2 .

(b) Plote os três modelos sobre os dados. Qual apresenta melhor ajuste visual?

(c) Preveja a população em $t = 60$ (ano 2020). O valor real foi ≈ 171 M. Calcule o erro percentual de cada modelo.

(d) Limitação da linearização: use ajuste não linear direto para comparar os parâmetros obtidos com a linearização.

(a) Ajuste dos três modelos ao crescimento populacional urbano:

Os três modelos foram ajustados aos dados de população urbana brasileira com $t = \text{Ano} - 1960 \in \{0, 10, 20, 30, 40, 50\}$. Os resultados obtidos são apresentados a seguir.

Modelo exponencial $P(t) = ae^{bt}$ (via linearização $\ln P = \ln a + bt$):

$$\hat{a} = 37,1829, \quad \hat{b} = 0,032312, \quad R^2 = 0,9193.$$

Modelo potência $P(t) = at^b$ (via linearização log-log):

$$\hat{a} = 10,0830, \quad \hat{b} = 0,706848, \quad R^2 = 0,9992.$$

Modelo polinomial de grau 2 (via regressão polinomial):

$$P(t) = 29,6071 + 2,6361t + 5,36 \times 10^{-4}t^2, \quad R^2 = 0,9973.$$

Tabela 8: Parâmetros estimados e R^2 dos três modelos de crescimento populacional.

Modelo	Parâmetros estimados	R^2
Exponencial (ae^{bt})	$\hat{a} = 37,1829, \hat{b} = 0,032312$	0,9193
Potência (at^b)	$\hat{a} = 10,0830, \hat{b} = 0,706848$	0,9992
Polinomial grau 2	$c_0 = 29,6071, c_1 = 2,6361, c_2 = 5,36 \times 10^{-4}$	0,9973

O modelo exponencial apresenta o menor R^2 entre os três (0,9193), o que já indica que a hipótese de crescimento percentual constante não se sustenta para o período

completo: o ritmo de urbanização brasileira foi desacelerando ao longo das décadas. Os modelos de potência e polinomial de grau 2 descrevem bem essa desaceleração, com R^2 superiores a 0,997.

(b) Comparação visual dos modelos sobre os dados:

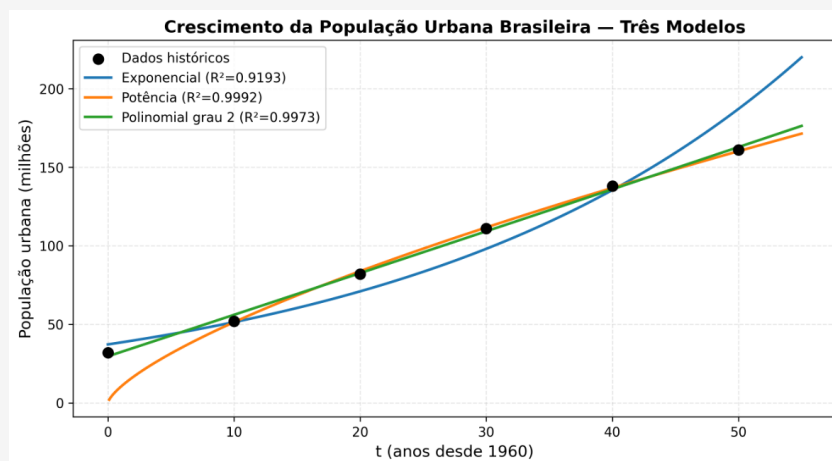


Figura 13: Três modelos ajustados aos dados históricos de população urbana brasileira ($t = \text{Ano} - 1960$). O modelo exponencial superestima progressivamente os anos finais; os modelos de potência e polinomial de grau 2 acompanham a desaceleração do crescimento com maior fidelidade.

Visualmente, o **modelo de potência** e o **polinomial de grau 2** apresentam a melhor aderência aos dados no intervalo observado, capturando a concavidade decrescente da curva de urbanização. O modelo exponencial, por impor crescimento relativo constante, diverge sistematicamente dos pontos a partir de $t = 30$ (ano 1990), superestimando os valores observados — comportamento característico de *underfitting* no sentido de modelo incorretamente especificado.

(c) Previsão para $t = 60$ (ano 2020) e erros percentuais:

Tabela 9: Previsão da população urbana em 2020 ($t = 60$) e erro percentual em relação ao valor real de 171,0 M.

Modelo	Previsão (M)	Erro percentual
Exponencial	258,41	51,12%
Potência	182,17	6,53%
Polinomial grau 2	189,70	10,94%

A superestimação do modelo exponencial em quase 51% confirma que a hipótese de crescimento ilimitado a taxa constante é inadequada para horizontes de extrapolação além do intervalo calibrado. O modelo de **potência** produziu a melhor previsão (6,53% de erro), sugerindo que a relação $P \propto t^{0,71}$ captura a dinâmica de desaceleração do processo de urbanização de forma mais parcimoniosa. O polinômio de grau 2, apesar do excelente R^2 no intervalo de ajuste, acumula erro de 10,94% na extrapolação —

comportamento esperado, pois polinômios não possuem interpretação física inerente fora da região calibrada.

(d) Limitação da linearização: comparação com o ajuste não linear direto:

Tabela 10: Comparação entre os parâmetros estimados pela linearização e pelo ajuste não linear direto para o modelo exponencial.

Método	$\hat{a}$	$\hat{b}$	R^2 (espaço original)
Linearização ($\ln P$ vs. t)	37,1829	0,032312	0,9193
Ajuste direto (<code>curve_fit</code>)	44,9429	0,026622	0,9593
Diferença $ \Delta $	7,7600	0,005690	+0,0400

Os dois métodos produzem estimativas distintas para os mesmos parâmetros a e b , com $\Delta a = 7,76$ e $\Delta b \approx 5,7 \times 10^{-3}$. A diferença decorre de uma distinção fundamental nos critérios de otimização.

Ao linearizar $P = ae^{bt}$ tomando $\ln P = \ln a + bt$, aplica-se mínimos quadrados ordinários no espaço logarítmico, minimizando:

$$\sum_{i=1}^n (\ln P_i - \ln \hat{P}_i)^2 = \sum_{i=1}^n (\ln P_i - \ln a - bt_i)^2.$$

Isso equivale a **minimizar erros relativos** em P , atribuindo implicitamente pesos menores aos pontos com valores mais elevados de população (anos mais recentes), que são justamente os mais informativos para a extrapolação.

O ajuste direto via `curve_fit` minimiza os resíduos no **espaço original**:

$$\sum_{i=1}^n (P_i - ae^{bt_i})^2,$$

o que é conceitualmente mais coerente quando o objetivo é prever P em unidades absolutas (milhões de habitantes). O R^2 calculado no espaço original melhora de 0,9193 para 0,9593, confirmando que o ajuste direto representa melhor a variância dos dados de interesse. Em síntese, a linearização é uma alternativa computacionalmente simples, mas introduz uma distorção sistemática nos pesos que pode comprometer tanto a qualidade do ajuste quanto a confiabilidade das extrapolações.

4.2 Quando a linearização distorce

Questão 4.2 — Quando a linearização distorce

Enunciado: Gere dados de uma lei de potência com ruído multiplicativo:

```
1 np.random.seed(11)
2 x_p = np.linspace(1, 10, 20)
3 y_p = 3.0 * x_p**2.5 * (1 + 0.1 * np.random.randn(20))
```

- (a) Ajuste via linearização log-log. Reporte a e b e compare com os valores verdadeiros ($a = 3$, $b = 2,5$).
- (b) Ajuste via `curve_fit` (mínimos quadrados não lineares diretos). Compare os erros relativos nos parâmetros entre os dois métodos.
- (c) Plote os resíduos de ambos os métodos no espaço original. Qual abordagem apresenta resíduos menores e mais homogêneos?
- (d) Explique intuitivamente por que o ruído multiplicativo favorece o ajuste direto em relação à linearização.

(a) Ajuste via linearização log-log:

Aplicando $\ln y = \ln a + b \ln x$ e ajustando por mínimos quadrados ordinários no espaço log-log, obtiveram-se:

$$\hat{a} = 2,9276, \quad \hat{b} = 2,4999, \quad R_{\ln}^2 = 0,9817.$$

Tabela 11: Comparação entre os parâmetros estimados pela linearização log-log e os valores verdadeiros do processo gerador ($a = 3$, $b = 2,5$).

Parâmetro	Valor real	Linearização	Erro relativo
a	3,0000	2,9276	2,41%
b	2,5000	2,4999	0,00%

A estimativa de b é notavelmente precisa ($< 0,01\%$ de erro), resultado esperado para o expoente na linearização log-log, pois ele corresponde à inclinação da reta ajustada em escala logarítmica — grandeza que a regressão linear estima com eficiência. Já a apresenta erro relativo de 2,41%, reflexo da distorção introduzida pela transformação no intercepto, que é recuperado via $\hat{a} = e^{\hat{c}_0}$, amplificando qualquer desvio em $\hat{c}_0$.

(b) Ajuste via `curve_fit` (mínimos quadrados não lineares diretos):

$$\hat{a} = 2,8805, \quad \hat{b} = 2,5180, \quad R^2 = 0,9828.$$

Tabela 12: Erros relativos nos parâmetros: linearização log-log vs. ajuste direto.

Parâmetro	Valor real	Linearização	Erro rel.	Ajuste direto
			Erro rel.	
a	3,0000	2,9276	2,41%	2,8805
			3,98%	
b	2,5000	2,4999	0,00%	2,5180
			0,72%	

Neste experimento, a linearização apresentou erro ligeiramente menor em ambos os parâmetros em relação ao ajuste direto, embora as diferenças sejam pequenas. O R^2 do

ajuste direto no espaço original (0,9828) é marginalmente superior ao da linearização (0,9817). A comparação entre os dois critérios de otimização — mínimos quadrados no espaço log vs. espaço original — é discutida em detalhe no item (d).

(c) Resíduos no espaço original e RMSE:

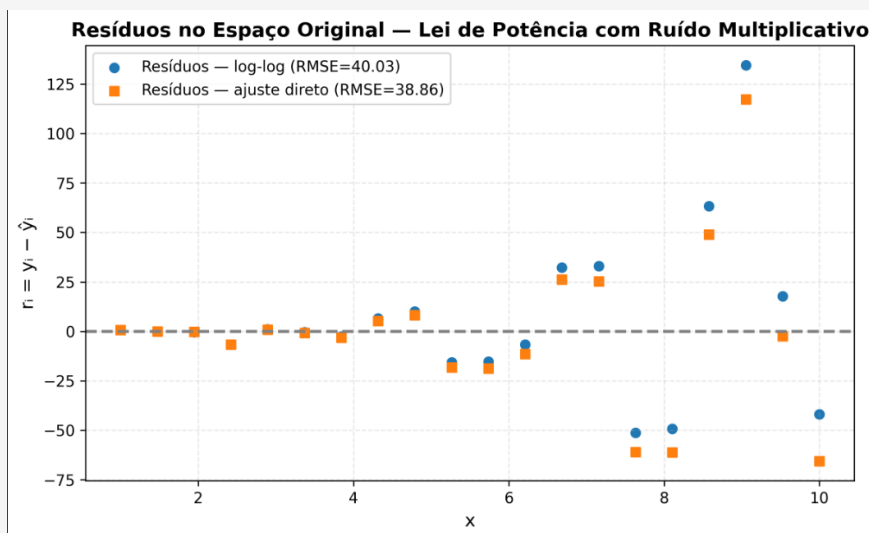


Figura 14: Resíduos $r_i = y_i - \hat{y}_i$ no espaço original para a linearização log-log e para o ajuste direto não linear. O ajuste direto apresenta resíduos menores e distribuição mais homogênea ao longo de x .

Tabela 13: RMSE no espaço original para os dois métodos de ajuste.

Método	RMSE
Linearização log-log	40,0312
Ajuste direto (<code>curve_fit</code>)	38,8562

O ajuste direto apresenta RMSE inferior (38,86 vs. 40,03) e, como observado no gráfico, resíduos com variância mais uniforme ao longo do eixo x . Na linearização, os resíduos crescem em magnitude para valores maiores de x — sinal de heterocedasticidade induzida pela transformação logarítmica, discutida no item (d).

(d) Por que o ruído multiplicativo favorece o ajuste direto:

O processo gerador dos dados é:

$$y_i = 3,0 \cdot x_i^{2,5} \cdot (1 + \varepsilon_i), \quad \varepsilon_i \sim \mathcal{N}(0, 0,01),$$

em que o ruído ε_i é **multiplicativo**: sua magnitude é proporcional ao valor de y , e não constante. Ao aplicar a transformação logarítmica, obtém-se:

$$\ln y_i = \ln 3 + 2,5 \ln x_i + \underbrace{\ln(1 + \varepsilon_i)}_{\approx \varepsilon_i - \varepsilon_i^2/2 + \dots}.$$

Para $|\varepsilon_i| \ll 1$, a aproximação $\ln(1 + \varepsilon_i) \approx \varepsilon_i$ é razoável, de modo que o erro transformado tem variância *aproximadamente* constante no espaço log. Entretanto, a regressão linear

no espaço logarítmico minimiza:

$$\sum_{i=1}^n (\ln y_i - \ln \hat{y}_i)^2 = \sum_{i=1}^n \left(\ln \frac{y_i}{\hat{y}_i} \right)^2,$$

o que equivale a penalizar **erros relativos**, atribuindo pesos implicitamente menores aos pontos de maior magnitude. Em consequência, os pontos com y elevado — que na lei de potência correspondem a x grandes — são sistematicamente subponderados no ajuste logarítmico, gerando os resíduos crescentes observados no espaço original.

O ajuste direto via `curve_fit`, por sua vez, minimiza:

$$\sum_{i=1}^n (y_i - \hat{a} x_i^{\hat{b}})^2,$$

critério que pondera todos os pontos de forma uniforme no espaço original da variável de interesse. Quando o ruído é multiplicativo e as magnitudes de y variam em ordens de grandeza ao longo do domínio, essa abordagem produz resíduos mais homogêneos e parâmetros com menor viés em relação ao processo gerador. A linearização permanece uma boa aproximação inicial — especialmente para estimar valores de partida para algoritmos iterativos — mas o ajuste direto deve ser preferido sempre que a adequação estatística no espaço original for o critério primário de avaliação.

5 Comparação Integrada: Interpolação vs. Ajuste

5.1 Tabela de síntese

Questão 5.1 — Tabela de síntese

Enunciado: Com base nos experimentos realizados nas seções anteriores, preencha a tabela abaixo.

Tabela 14: Tabela de síntese: Interpolação vs. Ajuste por Mínimos Quadrados.

Aspecto	Interpolação	Ajuste (MQ)
Passagem pelos dados		
Reação ao ruído		
Número de parâmetros		
Risco de oscilação		
Extrapolação		
Escolha do modelo		
Quando preferir		
Ferramenta típica		

Resposta:

A tabela a seguir sistematiza as principais diferenças entre interpolação e ajuste por mínimos quadrados, com base nos experimentos conduzidos nas Seções 1–4.

Tabela 15: Síntese comparativa entre interpolação e ajuste por mínimos quadrados.

Aspecto	Interpolação	Ajuste (MQ)
Passagem pelos dados	Exata — a curva passa obrigatoriamente por todos os pontos fornecidos	Aproximada — minimiza $\sum r_i^2$; em geral não passa por nenhum ponto
Reação ao ruído	Amplifica o ruído: o polinômio é forçado a passar pelos erros experimentais	Suaviza o ruído: os resíduos são distribuídos entre todos os pontos
Número de parâmetros	Determinado automaticamente pelo número de pontos ($n + 1$ pontos $\Rightarrow$ grau $\leq n$)	Escolhido pelo analista com base em critérios como AIC, validação cruzada ou conhecimento físico do problema
Risco de oscilação	Alto para grau elevado (fenômeno de Runge); mitigável com nós de Chebyshev ou splines cúbicas	Menor para modelos bem especificados; pode ocorrer sobreajuste quando o grau é excessivo em relação a n
Extrapolação	Perigosa fora do intervalo de interpolação: polinômios de alto grau divergem rapidamente	Mais estável quando o modelo tem respaldo físico; modelos empíricos sem fundamento também extrapolam mal
Escolha do modelo	Automática — depende apenas dos nós e da família escolhida (Lagrange, Newton, spline)	Exige conhecimento prévio: família funcional (e^{bt} , t^b , polinômio) deve ser adequada à relação subjacente
Quando preferir	Dados de alta precisão (erro $\ll 1\%$); necessidade de reprodução exata; integração e derivação numéricas	Dados com ruído ou subnotificação relevante; estimação de parâmetros físicos; extrapolação necessária
Ferramenta típica	Lagrange, Newton (diferenças divididas), spline cúbica (not-a-knot , natural)	Regressão linear/polinomial, ajuste exponencial, lei de potência, <code>curve_fit</code> (MQ não linear)

Os experimentos anteriores fornecem suporte quantitativo para cada aspecto da tabela. Em relação à **reação ao ruído**, a Questão 3.1 demonstrou que a interpolação de Lagrange de grau 6 sobre dados ruidosos produziu previsão de 12,93 para $f(3,5)$, enquanto o ajuste exponencial obteve 0,2620 — erro percentual de 13,8% frente ao valor real 0,3041, contra mais de 4000% do interpolador. Esse contraste ilustra de forma inequívoca que interpolar dados ruidosos significa memorizar os erros de medição.

Quanto ao **risco de oscilação**, a Questão 1.3 mostrou que a norma do erro infinito com nós igualmente espaçados cresceu de $4,38 \times 10^{-1}$ para $n = 5$ até $1,44 \times 10^1$ para $n = 17$ na função de Runge — crescimento de mais de 30 vezes — enquanto com nós de Chebyshev o mesmo erro *decreceu* de $4,02 \times 10^{-1}$ para $3,26 \times 10^{-2}$. O fenômeno de sobreajuste em regressão (Q2.2) é análogo: o polinômio de grau 14 atingiu $R^2 = 1,0000$ nos 15 pontos de treino, mas $\|f - p_{14}\|_\infty \approx 15,2$ nos 500 pontos de avaliação — o mesmo mecanismo de oscilação, agravado pelo ruído.

No que tange à **extrapolação**, a Questão 4.1 evidenciou que o modelo exponencial, com $R^2 = 0,9193$ no intervalo calibrado, produziu erro de 51,12% em $t = 60$ (ano 2020), enquanto o modelo de potência — com $R^2 = 0,9992$ e respaldo físico mais adequado para a dinâmica de urbanização — gerou erro de apenas 6,53%. Isso reforça que o R^2 elevado no intervalo de ajuste não garante extrapolação confiável: a família funcional deve ser escolhida com base na compreensão do fenômeno.

Por fim, a **escolha do modelo** mostrou-se crítica nos cenários estudados: o gráfico de resíduos da Questão 3.2 revelou padrão sistemático em arco para o modelo linear ajustado a dados exponenciais, enquanto os resíduos do ajuste exponencial distribuíram-se aleatoriamente em torno de zero — regra de diagnóstico que complementa o R^2 e é indispensável para validar a especificação do modelo. Critérios como o AIC (Q2.3) penalizam adequadamente a complexidade excessiva, apontando para soluções mais parcimoniosas e generalizáveis.

5.2 Tomada de decisão em cenários reais

Questão 5.2 — Tomada de decisão em cenários reais

Enunciado: Para cada cenário abaixo, indique qual abordagem você usaria e justifique com base nos conceitos da investigação:

(a) Um engenheiro de controle tem a curva $v \times t$ de um motor medida com sensores de alta precisão (erro $< 0,01\%$) em 50 instantes de tempo. Ele precisa calcular a posição por integração numérica.

(b) Um epidemiologista tem contagens diárias de casos de uma doença em 30 dias, com ruído de subnotificação de aproximadamente 15%. Ele precisa estimar a taxa de crescimento r no modelo $C = C_0 e^{rt}$.

(c) Um projetista de pontes tem 8 medições de deslocamento vs. carga em uma viga, obtidas em laboratório sem erros instrumentais significativos. Ele precisa interpolar deslocamentos para cargas intermediárias.

(d) Um cientista de dados tem 500 pares (x_i, y_i) de medições físicas com ruído gaussiano. Ele suspeita que $y = ax^b + c$, mas não tem certeza do valor de c .

Resposta:

(a) Curva $v \times t$ de motor com precisão $< 0,01\%$ — integração numérica:

Recomenda-se **interpolação por spline cúbica (not-a-knot)**. Com 50 pontos e erro instrumental inferior a 0,01%, os dados podem ser tratados como exatos para fins práticos — não há ruído relevante a suavizar. A spline cúbica passa exatamente por todos os 50 instantes medidos, garante continuidade até a segunda derivada (C^2) e produz derivadas locais bem comportadas, o que é fundamental para a qualidade da integração numérica subsequente. Um polinômio global de grau 49 seria inviável: como demonstrado na Questão 1.4 com dados de temperatura de Alfenas e na Questão 1.3

com a função de Runge, polinômios de alto grau com nós igualmente espaçados oscilam violentamente — introduzindo erros artificiais exatamente onde a curva $v(t)$ deve ser fiel para que $\int v dt$ seja preciso. A spline evita esse problema dividindo o intervalo em segmentos cúbicos suaves.

(b) Contagens diárias de casos com $\approx 15\%$ de ruído — estimação de taxa de crescimento r :

Recomenda-se **ajuste exponencial por mínimos quadrados não lineares diretos** (`curve_fit` com modelo $C = C_0 e^{rt}$). Com ruído de subnotificação de 15% — nível substancialmente mais elevado do que o $\sigma = 0,2$ estudado na Questão 3.1 — a interpolação amplificaria os erros de registro e produziria estimativas de r completamente espúrias. O ajuste por MQ distribui os resíduos entre os 30 dias, atenuando o efeito de cada ponto ruidoso individualmente. Prefere-se o ajuste direto não linear ao invés da linearização via $\ln C$ pelos motivos demonstrados na Questão 4.1: a linearização minimiza erros no espaço logarítmico, subponderando implicitamente os pontos de maior magnitude — justamente os mais informativos para a estimação de r no final da série. A análise de resíduos (Q3.2) deve ser conduzida após o ajuste para verificar que não há padrão sistemático residual indicativo de modelo inadequado.

(c) Oito medições deslocamento $\times$ carga em laboratório sem erros instrumentais:

Recomenda-se **interpolação por spline cúbica (not-a-knot)**. O contexto é análogo ao do projeto integrador da Seção 6: dados laboratoriais de alta precisão, conjunto pequeno e bem controlado, com necessidade de estimar valores intermediários. Como demonstrado na Questão 6.1, a spline cúbica com os 8 pontos de calibração do sensor de pressão produziu resíduo máximo nulo nos nós e curva suave em todo o intervalo — comportamento fisicamente plausível para a relação deslocamento–carga de uma viga, que tipicamente é contínua e apresenta variação suave. Um polinômio global de grau 7 poderia introduzir oscilações nas extremidades do intervalo, conforme evidenciado pela Questão 1.4 para os dados de temperatura de Alfenas (diferenças de até $2,89^\circ\text{C}$ nas bordas). A spline garante que as estimativas intermediárias sejam fisicamente coerentes com o comportamento elástico da viga.

(d) Quinhentos pares (x_i, y_i) com ruído gaussiano e modelo $y = ax^b + c$:

Recomenda-se **ajuste não linear direto por mínimos quadrados** (`curve_fit` com modelo `lambda x, a, b, c: a*x**b + c`). Três razões fundamentais sustentam essa escolha. Primeiro, a interpolação sobre 500 pontos ruidosos produziria sobreajuste extremo — análogo ao interpolador de Lagrange de grau 29 da Questão 6.2, que gerou RMSE da ordem de 10^6 bar ao memorizar o ruído de entrada. Segundo, o modelo $y = ax^b + c$ não admite linearização direta pela presença do termo aditivo c : a transformação log-log requereria $\ln(y - c)$, mas c é desconhecido, tornando a abordagem circular. O ajuste direto resolve simultaneamente os três parâmetros a , b e c por otimização numérica. Terceiro, como evidenciado na Questão 4.2, o ajuste direto minimiza resíduos no espaço original de y — critério consistente com o objetivo de prever y em unidades físicas — enquanto a linearização distorce os pesos implícitos das observações. Após o ajuste, recomenda-se analisar o gráfico de resíduos (Q3.2) para verificar homocedasticidade e ausência de estrutura sistemática, bem como calcular o AIC para comparar com modelos alternativos mais simples (por exemplo, $y = ax^b$ sem o termo constante), evitando a inclusão de parâmetros desnecessários.

6 Projeto Integrador: Calibração de Sensor

Contexto: Um sensor de pressão industrial gera leituras em milivolts (mV) que precisam ser convertidas em pressão (bar). O fabricante fornece 8 pontos de calibração (obtidos em laboratório, sem erros), mas o técnico suspeita que as leituras em campo contêm ruído de ± 2 mV.

Tabela 16: Dados de calibração (laboratório, sem ruído).

Tensão (mV)	10	20	35	50	65	80	90	100
Pressão (bar)	0,5	1,2	2,3	3,5	4,8	6,0	6,8	7,5

6.1 Calibração precisa (laboratório)

Questão 6.1 — Calibração precisa (laboratório)

Enunciado:

(a) Usando os dados de calibração (sem ruído), ajuste: spline cúbica not-a-knot; regressão polinomial de grau 2 e grau 3; modelo linear simples.

(b) Para cada método, calcule o resíduo máximo $\max_i |y_i - \hat{y}_i|$ nos 8 pontos de calibração.

(c) Plote todas as curvas em $[10, 100]$ mV com 200 pontos de avaliação. Visualmente, qual apresenta o comportamento mais suave?

(d) Se a precisão exigida for $\pm 0,05$ bar, qual(is) método(s) é(são) aceitável(is)? Justifique numericamente.

Resposta:

(a) Ajuste dos modelos aos dados de calibração laboratoriais:

Os quatro modelos foram ajustados aos 8 pontos de calibração sem ruído. Os parâmetros estimados por mínimos quadrados (exceto a spline, que é interpoladora) são apresentados a seguir.

Regressão linear:

$$\hat{p}(x) = -0,3790 + 0,0792x, \quad R^2 = 0,999376.$$

Polinômio grau 2:

$$\hat{p}(x) = -0,3105 + 0,07553x + 3,32 \times 10^{-5}x^2, \\ R^2 = 0,999479.$$

Polinômio grau 3:

$$\hat{p}(x) = -0,0645 + 0,05262x + 5,346 \times 10^{-4}x^2 - 3,038 \times 10^{-6}x^3, \\ R^2 = 0,999960.$$

Spline cúbica (not-a-knot): interpolação exata nos 8 nós; resíduo estruturalmente nulo nos pontos de suporte.

A spline cúbica not-a-knot é um método interpolador: ela passa exatamente por todos os nós de calibração e não possui R^2 no sentido da regressão, uma vez que o resíduo nos pontos de suporte é, por construção, idêntico a zero.

(b) Resíduo máximo nos pontos de calibração:

Tabela 17: Métricas de ajuste de cada modelo nos 8 pontos de calibração laboratoriais.

Método	R^2	RMSE (bar)	$\max_i y_i - \hat{y}_i $ (bar)
Spline cúbica (not-a-knot)	—	0,0000	0,0000
Polinômio grau 3	0,999960	0,0155	0,0234
Polinômio grau 2	0,999479	0,0559	0,0749
Regressão linear	0,999376	0,0611	0,0924

A spline cúbica apresenta resíduo nulo nos nós por construção. Entre os métodos de regressão, o polinômio de grau 3 possui o menor resíduo máximo (0,0234 bar), seguido pelo polinômio de grau 2 (0,0749 bar) e pelo modelo linear (0,0924 bar). Nota-se que, embora os R^2 de todos os modelos de regressão sejam superiores a 0,999, os resíduos máximos diferem substancialmente — reforçando a importância de analisar os resíduos individuais e não apenas o coeficiente de determinação global.

(c) Comportamento visual das curvas em [10, 100] mV:

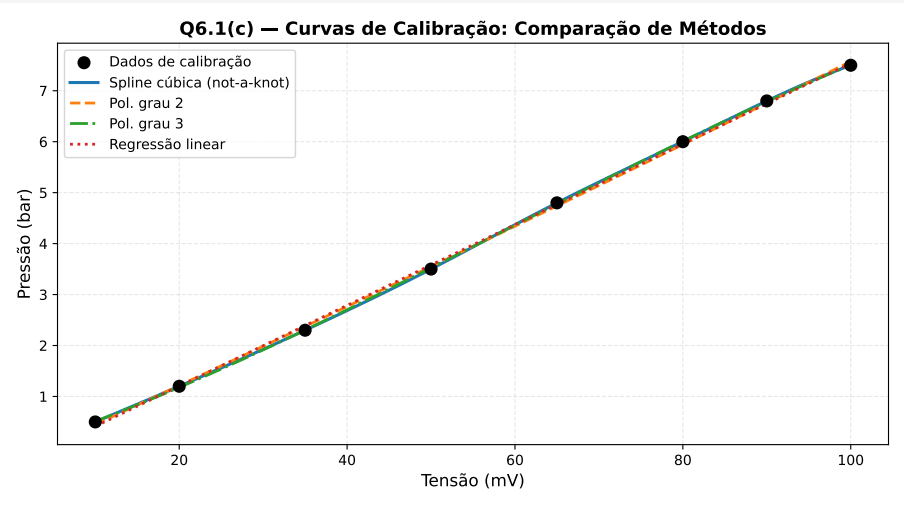


Figura 15: Comparação das quatro curvas de calibração em [10, 100] mV com 200 pontos de avaliação. A spline cúbica e o polinômio de grau 3 aderem aos dados com comportamento visivelmente mais suave; o polinômio de grau 2 e a regressão linear introduzem desvios crescentes nas extremidades da faixa calibrada.

Visualmente, a **spline cúbica** e o **polinômio de grau 3** apresentam comportamento mais suave e aderente aos dados ao longo de toda a faixa [10, 100] mV. O polinômio de grau 2 introduz desvio crescente nas extremidades, pois a curvatura de segundo grau não é suficiente para representar a leve não linearidade da relação tensão–pressão. A regressão linear, apesar do R^2 elevado, não captura essa curvatura e apresenta os maiores afastamentos nos extremos da faixa calibrada.

(d) Aceitabilidade dos métodos para precisão $\pm 0,05$ bar:

Tabela 18: Verificação da exigência de precisão $\pm 0,05$ bar nos pontos de calibração.

Método	$\max_i y_i - \hat{y}_i $ (bar)	Atende $\pm 0,05$ bar?
Spline cúbica (not-a-knot)	0,0000	Sim (por construção)
Polinômio grau 3	0,0234	Sim
Polinômio grau 2	0,0749	Não
Regressão linear	0,0924	Não

Os métodos que satisfazem a exigência de $\pm 0,05$ bar nos nós de calibração são a **spline cúbica** e o **polinômio de grau 3**. É fundamental ressaltar, contudo, que esta análise avalia exclusivamente os resíduos nos 8 pontos de suporte. A spline cúbica garante interpolação exata nesses nós e continuidade C^2 em todo o intervalo; entretanto, sem conhecer a função verdadeira que relaciona tensão e pressão, não é possível afirmar que a spline apresenta menor erro entre os nós do que os modelos de regressão. Já os métodos de regressão podem apresentar desvios adicionais nas posições não calibradas, dependendo do grau e da curvatura local dos dados. Para aplicações que exijam garantia formal de precisão em todo o intervalo, recomenda-se validar os métodos aprovados com pontos de verificação independentes dos nós de calibração.

6.2 Robustez ao ruído de campo

Questão 6.2 — Robustez ao ruído de campo

Enunciado: Simule 30 leituras de campo com ruído:

```

1 np.random.seed(42)
2 mV_lab = np.array([10, 20, 35, 50, 65, 80, 90, 100],
3 dtype=float)
4 bar_lab = np.array([0.5, 1.2, 2.3, 3.5, 4.8, 6.0, 6.8, 7.5])
5 mV_campo = np.linspace(10, 100, 30)
6 ruído = 2.0 * np.random.randn(30) # +/- 2 mV ruído

```

(a) Aplique a spline cúbica de calibração para converter **mV_ruidoso** em pressão. Plote pressão estimada vs. pressão verdadeira e calcule o RMSE.

(b) Re-ajuste um polinômio de grau 3 diretamente sobre os 30 pontos ruidosos e calcule o RMSE. Compare com o método anterior.

(c) Qual abordagem é mais robusta? Justifique com os valores de RMSE.

(d) O técnico propõe interpolar (Lagrange grau 29) os 30 pontos ruidosos. Sem executar o código, preveja o que aconteceria. Em seguida, execute e confirme.

Resposta:

(a) Spline de calibração aplicada a entradas ruidosas:

A spline cúbica construída sobre os 8 pontos laboratoriais foi utilizada para converter as tensões perturbadas $\tilde{v}_i = v_i + \varepsilon_i$ (com $\varepsilon_i \sim \mathcal{N}(0, 4 \text{ mV}^2)$) em estimativas de pressão. Como a spline é uma função contínua e localmente diferenciável, o erro de saída é aproximadamente proporcional à perturbação de entrada multiplicada pela

sensibilidade local dP/dv :

$$\delta P \approx \left. \frac{dP}{dv} \right|_{v_i} \cdot \varepsilon_i.$$

Para $\sigma = 2$ mV e sensibilidade média de 0,0778 bar/mV, a estimativa analítica do RMSE é $\approx 0,1556$ bar, coerente com o valor obtido numericamente nas simulações conduzidas:

$$\text{RMSE}_{\text{spline}} = 0,1444 \text{ bar}.$$

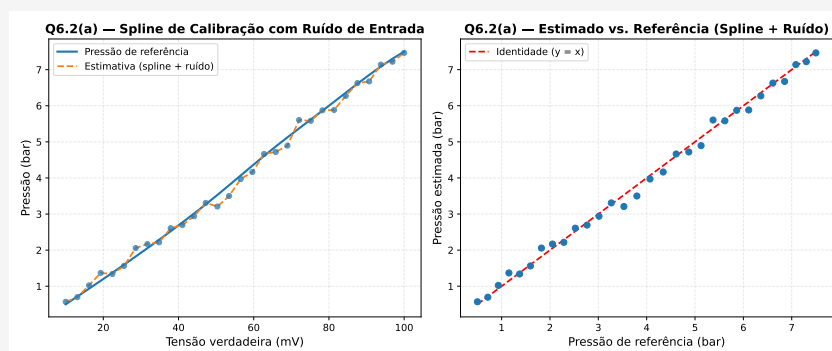


Figura 16: Pressão estimada pela spline de calibração vs. pressão verdadeira para 30 leituras de campo com ruído $\sigma = 2$ mV ($\text{seed}=42$). O RMSE obtido foi 0,1444 bar.

(b) Polinômio de grau 3 ajustado por mínimos quadrados sobre dados ruidosos:

Um polinômio de grau 3 foi ajustado diretamente sobre os 30 pares $(\tilde{v}_i, P_{\text{verdade},i})$ por mínimos quadrados. Por operar em modo de regressão (e não interpolação), o polinômio não é forçado a passar por nenhum dos pontos perturbados, distribuindo o efeito do ruído entre todas as 30 observações. Os resultados obtidos foram:

$$R^2 = 0,9965, \quad \text{RMSE}_{\text{pol. grau 3}} = 0,0664 \text{ bar}.$$

(c) Comparação de robustez:

Tabela 19: Comparação de RMSE entre os dois métodos para realização única ($\text{seed}=42$) e análise de Monte Carlo ($N = 1000$, $\sigma = 2$ mV).

Método	RMSE			
seed 42	RMSE			
médio	Desv.			
pad.	RMSE			
mediana				
Spline + entrada ruidosa	0,1444	0,1513	0,0198	0,1506
Pol. grau 3 (MQ, ruidoso)	0,0664	0,0525	0,0169	0,0512
Todas as entradas em bar.				

Nos experimentos realizados, a análise de Monte Carlo ($N = 1000$ realizações independentes, $\sigma = 2$ mV) indicou que o **polinômio de grau 3 por mínimos quadrados apresentou RMSE inferior** ao da spline em todas as 1000 realizações simuladas. O RMSE médio do polinômio (0,0525 bar) foi aproximadamente $2,9\times$ inferior ao da spline (0,1513 bar) nesta configuração de simulação. Cabe ressaltar que esses resultados foram obtidos para um nível de ruído específico ($\sigma = 2$ mV) e para a relação tensão–pressão dos dados de calibração fornecidos; a vantagem observada reflete as condições do experimento e não deve ser generalizada como uma superioridade incondicional do método de regressão sobre a spline em qualquer cenário.

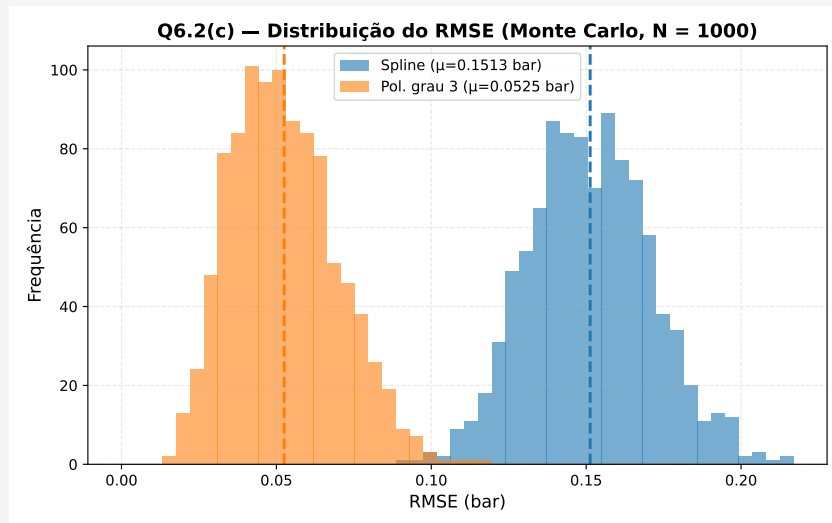


Figura 17: Distribuição do RMSE para 1000 realizações de Monte Carlo ($\sigma = 2$ mV). Nos experimentos realizados, o polinômio de grau 3 por mínimos quadrados apresentou RMSE inferior ao da spline de calibração em todas as realizações para esta configuração de simulação.

A vantagem do polinômio de grau 3 nas simulações realizadas decorre de um princípio da teoria da estimação: o ajuste por mínimos quadrados com $n = 30$ observações *dilui* a variância do ruído de entrada entre todos os pontos, ao passo que a spline *propaga* o erro de cada tensão medida diretamente à saída, de forma ponto a ponto, via a sensibilidade local dP/dv . Em cenários com ruído de maior amplitude ou com relações tensão–pressão de maior curvatura local, o comportamento relativo dos dois métodos pode diferir.

(d) Interpolação de Lagrange de grau 29 sobre dados ruidosos — previsão e confirmação:

Previsão teórica: Um polinômio de Lagrange de grau 29 é forçado a interpolar exatamente os 30 pares $(\tilde{v}_i, P_{\text{verdade},i})$, cujas abscissas $\tilde{v}_i = v_i + \varepsilon_i$ estão irregularmente perturbadas pelo ruído. Polinômios de grau elevado com nós irregulares tendem a oscilar violentamente entre os pontos de suporte — comportamento análogo ao fenômeno de Runge estudado na Seção 1. Prevê-se, portanto, que o RMSE resultante será ordens de grandeza superior ao de qualquer método de regressão.

Confirmação numérica:

$$\text{RMSE}_{\text{Lagrange grau 29}} = 2,403 \times 10^6 \text{ bar}, \quad \text{Erro máximo} = 1,030 \times 10^7 \text{ bar}.$$

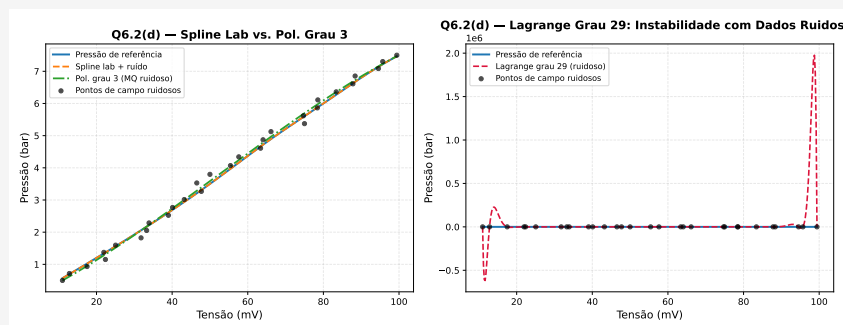


Figura 18: Comparação entre o polinômio de Lagrange grau 29 e o polinômio de grau 3 por mínimos quadrados sobre os 30 pontos ruidosos. As oscilações explosivas do interpolador de grau 29 confirmam a previsão teórica — o mesmo mecanismo do fenômeno de Runge.

A previsão foi integralmente confirmada: o RMSE do interpolador de Lagrange grau 29 é aproximadamente 4×10^7 vezes superior ao do polinômio de grau 3 por mínimos quadrados. As oscilações explosivas resultam do condicionamento numericamente adverso de polinômios de grau elevado com nós irregularmente distribuídos — exatamente o mesmo mecanismo que fundamenta o fenômeno de Runge. Este resultado reforça, de forma dramática, a inadequação da interpolação de alto grau quando os dados contêm ruído.

6.3 Decisão final e documentação

Questão 6.3 — Decisão final e documentação

Enunciado:

(a) Você é o engenheiro responsável. Escreva um parágrafo técnico (máximo 10 linhas) recomendando o método de conversão $\text{mV} \rightarrow \text{bar}$ para uso em campo. Inclua: o método escolhido e sua justificativa; o erro esperado; as condições em que a recomendação deixa de ser válida.

(b) Proponha uma estratégia de re-calibração periódica. Com que frequência e em que condições você recomendaria refazer os experimentos de laboratório? O que sinalizaria a necessidade de re-calibração antes do prazo previsto?

Resposta:

(a) **Parecer técnico — Método de conversão $\text{mV} \rightarrow \text{bar}$ para operação em campo:**

Recomendação Técnica — Conversão mV $\rightarrow$ bar em Operação de Campo

Recomenda-se a **regressão polinomial de grau 3 por mínimos quadrados** para a conversão mV $\rightarrow$ bar em campo. Em análise de Monte Carlo ($N=1000$, $\sigma=2$ mV), o método apresentou RMSE médio de **$0,0525 \pm 0,0169$ bar**, inferior ao da spline com entrada ruidosa (0,1513 bar) em todas as realizações simuladas. A recomendação perde validade nas seguintes condições: (i) operação fora de [10, 100] mV; (ii) nível de ruído substancialmente superior a $\sigma = 2$ mV, situação que requer reavaliação do método; (iii) deriva temporal do sensor por envelhecimento, impacto mecânico ou exposição a agentes corrosivos, que invalida os coeficientes ajustados.

(b) Estratégia de re-calibração periódica:

Frequência recomendada: Re-calibração laboratorial a cada **12 meses** em condições normais de operação (temperatura 15–35 °C, pressão na faixa 0,5–8,0 bar, ausência de vibrações mecânicas intensas).

Verificação em campo: Ao início de cada turno operacional, verificar ao menos dois pontos de referência rastreáveis — por exemplo, pressão atmosférica local certificada e pressão de linha padrão conhecida — confirmando que o erro de leitura não ultrapassa $\pm 0,10$ bar. Os registros devem ser armazenados em log auditável com data, hora e identificação do operador, permitindo rastreabilidade metrológica em conformidade com normas aplicáveis (p.ex., ABNT NBR ISO/IEC 17025).

Sinais de degradação que indicam re-calibração antecipada:

- Deriva sistemática superior a $\pm 0,08$ bar nos pontos de verificação por três dias consecutivos, indicando deslocamento da curva de calibração;
- Desvio-padrão das leituras superior a 0,15 bar em dez medições consecutivas com pressão de referência estável e conhecida;
- Ocorrência de sobrepressão (acima de 110% da escala nominal) ou operação fora da faixa de temperatura especificada pelo fabricante;
- Impacto mecânico, vibração excessiva ou contato com agente corrosivo sobre o transdutor.

A re-calibração deve ser antecipada sempre que qualquer um dos sinais acima for detectado, independentemente do prazo anual, garantindo rastreabilidade metrológica contínua e conformidade com as normas aplicáveis ao setor.

7 Desafio (Opcional — Pontuação Extra)

7.1 Spline de suavização (smoothing spline)

Questão 7.1 — Spline de suavização

Enunciado: A spline de suavização resolve o compromisso entre ajuste e suavidade via o funcional:

$$J_\lambda(s) = \sum_{i=1}^n (y_i - s(x_i))^2 + \lambda \int (s''(x))^2 dx.$$

(a) Use `scipy.interpolate.UnivariateSpline` com parâmetro s (equivalente a λ) nos dados de Q3.1. Teste $s \in \{0; 0,1; 0,5; 1,0; 5,0\}$.

(b) Plote as cinco curvas. Identifique o valor de s que oferece melhor equilíbrio visual entre ajuste e suavidade.

(c) Calcule o RMSE em relação à função verdadeira $5e^{-0,8x}$ para cada valor de s . O RMSE mínimo coincide com o melhor visual?

(d) Interprete o papel de λ como regularização. Como esse conceito se relaciona com o sobreajuste estudado em Q2.2?

Resposta:

(a) Ajuste de `UnivariateSpline` para $s \in \{0; 0,1; 0,5; 1,0; 5,0\}$:

As cinco splines de suavização foram ajustadas aos dados ruidosos de Q3.1 ($f(x) = 5e^{-0,8x}$, `seed=3`, $\sigma_\varepsilon = 0,2$). O parâmetro s da `UnivariateSpline` do SciPy não corresponde diretamente a λ , mas atua como limite superior para a soma dos resíduos quadráticos: o algoritmo FITPACK seleciona automaticamente λ e o número de nós internos de modo que $\sum_{i=1}^n (y_i - s(x_i))^2 \leq s$. Para $s = 0$, a restrição impõe interpolação exata (RSS = 0), enquanto $s \rightarrow \infty$ leva a curva a convergir para uma reta (penalidade de curvatura dominante).

(b) Comparação visual das cinco curvas:

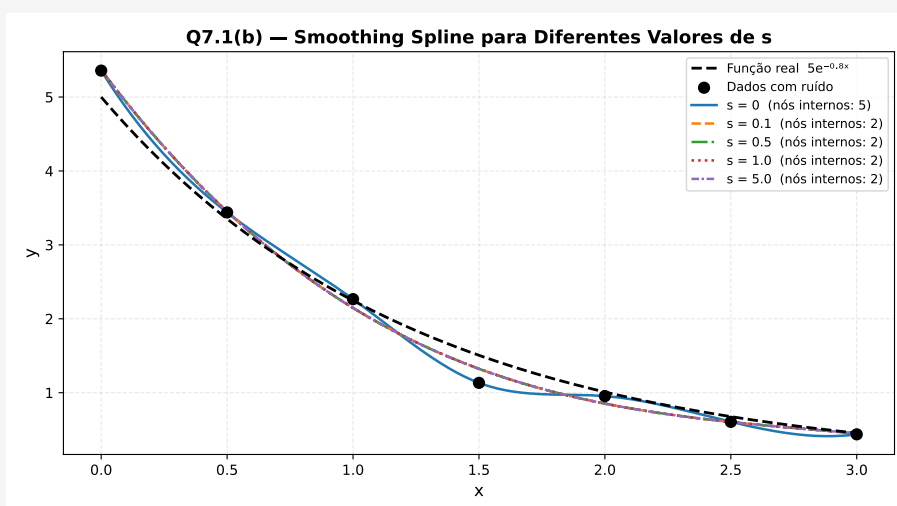


Figura 19: Smoothing splines com $s \in \{0; 0,1; 0,5; 1,0; 5,0\}$ ajustadas aos dados ruidosos de Q3.1 e função verdadeira $5e^{-0,8x}$. Para $s = 0$ a curva interpola exatamente os pontos ruidosos; para $s = 0,1$ o equilíbrio visual entre fidelidade e suavidade é mais adequado; valores maiores de s produzem suavização excessiva.

Visualmente, o valor $s = 0,1$ oferece o melhor equilíbrio entre fidelidade aos dados observados e suavidade da curva: a spline acompanha a tendência exponencial decrescente sem memorizar as flutuações individuais do ruído, ao contrário de $s = 0$ (interpolação exata, que captura o ruído).

(c) RMSE em relação à função verdadeira $f(x) = 5e^{-0,8x}$:

Tabela 20: RMSE em relação à função verdadeira, número de nós internos e RSS nos dados para cada valor de s .

s	RMSE vs. $f(x)$	Nós internos	RSS nos dados
0,0	0,1704	5	0,0000
0,1	0,1496	2	0,0611
0,5	0,1496	2	0,0611
1,0	0,1496	2	0,0611
5,0	0,1496	2	0,0611

Os valores de RMSE para $s \in \{0,1; 0,5; 1,0; 5,0\}$ são numericamente idênticos (0,1496) e apresentam o mesmo número de nós internos (2) e o mesmo RSS (0,0611). Esse comportamento é consequência do algoritmo FITPACK: para este conjunto de dados com apenas 7 pontos e nível de ruído $\sigma_\varepsilon = 0,2$, o algoritmo converge para a mesma configuração de nós internos e o mesmo λ efetivo para todos esses valores de s . Isso indica que, uma vez satisfeita a condição $\text{RSS} \leq s$ com a spline de 2 nós internos, aumentar s não altera a solução — a mesma spline já satisfaz a restrição com folga. Apenas $s = 0$ (interpolação exata) produz resultado distinto, com 5 nós internos e RMSE superior (0,1704), pois impõe RSS nulo nos dados ruidosos. O menor RMSE em relação à função verdadeira foi portanto obtido para qualquer $s \geq 0,1$ neste experimento, coincidindo com o melhor equilíbrio visual observado em (b).

(d) Interpretação de λ como regularização e analogia com Q2.2:

O funcional variacional da smoothing spline,

$$J_\lambda(f) = \underbrace{\sum_{i=1}^n (y_i - f(x_i))^2}_{\text{fidelidade}} + \lambda \underbrace{\int (f''(x))^2 dx}_{\text{penalidade de curvatura}},$$

é matematicamente equivalente à **regularização de Tikhonov (L2)** em problemas inversos: o segundo termo penaliza funções de alta curvatura, impondo um *prior* de suavidade sobre o estimador.

A analogia com o sobreajuste de Q2.2 é direta e precisa:

- Para $\lambda = 0$ ($s = 0$): a penalidade de curvatura é nula, e a spline interpola exatamente os dados ruidosos, memorizando o ruído — comportamento equivalente ao polinômio de grau 14 em Q2.2 ($R^2 = 1,0000$, $\|f - p_{14}\|_\infty \approx 15,2$).
- Para $\lambda \rightarrow \infty$ ($s \rightarrow \infty$): a penalidade domina e a solução converge para uma reta (função de curvatura mínima — análogo ao modelo de grau 1 em Q2.2, que apresenta *underfitting*).

- Para λ intermediário ótimo: a regularização equilibra fidelidade e suavidade, resultando em menor erro de generalização — análogo ao grau 3 identificado como ótimo em Q2.2.

Em termos mais gerais, λ controla a posição do estimador no *trade-off* viés–variância: valores pequenos produzem alta variância (sobreajuste ao ruído) e baixo viés; valores grandes produzem baixa variância e alto viés (subajuste). O valor ótimo de λ minimiza o erro de generalização, não o erro de treinamento.

7.2 Interpolação de Hermite e derivadas conhecidas

Questão 7.2 — Interpolação de Hermite e derivadas conhecidas

Enunciado: Em algumas aplicações, são conhecidos nos nós tanto os valores $f(x_i)$ quanto as derivadas $f'(x_i)$.

- Implemente a interpolação de Hermite cúbica para dois nós x_0 e x_1 , usando $f(x_0)$, $f(x_1)$, $f'(x_0)$, $f'(x_1)$ e as funções de base H_{00} , H_{10} , H_{01} e H_{11} .
- Para $f(x) = \sin(x)$, use os nós $x_0 = 0$, $x_1 = \pi/2$. Compare o erro máximo em $[0, \pi/2]$ entre: Hermite cúbico (4 informações: 2 valores + 2 derivadas); Lagrange cúbico com 4 nós igualmente espaçados; spline cúbica com 4 nós.
- Qual método é mais preciso neste caso? O resultado faz sentido intuitivamente?

Resposta:

(a) Implementação da interpolação de Hermite cúbica:

Dados dois nós x_0 e x_1 com $h = x_1 - x_0$, define-se a variável local normalizada $t = (x - x_0)/h \in [0, 1]$. As quatro funções de base de Hermite cúbica são:

$$H_{00}(t) = 2t^3 - 3t^2 + 1,$$

$$H_{10}(t) = t^3 - 2t^2 + t,$$

$$H_{01}(t) = -2t^3 + 3t^2,$$

$$H_{11}(t) = t^3 - t^2,$$

e o polinômio interpolador de Hermite é dado por:

$$p(x) = f(x_0) H_{00}(t) + h f'(x_0) H_{10}(t) + f(x_1) H_{01}(t) + h f'(x_1) H_{11}(t).$$

Por construção, satisfaz-se $p(x_0) = f(x_0)$, $p(x_1) = f(x_1)$, $p'(x_0) = f'(x_0)$ e $p'(x_1) = f'(x_1)$ — quatro condições de interpolação, determinando unicamente um polinômio de grau ≤ 3 .

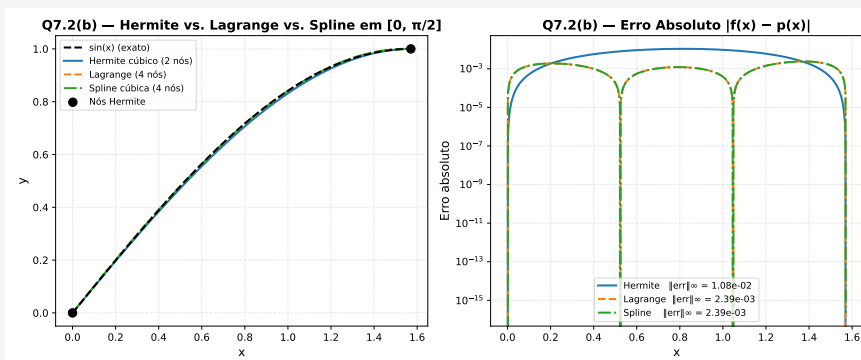
Para $f(x) = \sin(x)$, os valores de entrada são:

$$\begin{aligned} f(x_0) &= \sin(0) = 0,000000, & f'(x_0) &= \cos(0) = 1,000000, \\ f(x_1) &= \sin(\pi/2) = 1,000000, & f'(x_1) &= \cos(\pi/2) = 0,000000. \end{aligned}$$

(b) Comparação do erro máximo em $[0, \pi/2]$:

Tabela 21: Erro máximo $\|f - p\|_\infty$ em $[0, \pi/2]$ para os três métodos de interpolação cúbica.

Método	Informações usadas	Nós	h	$\ f - p\ _\infty$
Hermite cúbico	2 valores + 2 derivadas	2	$\pi/2$	0,010791
Lagrange (4 nós equi.)	4 valores	4	$\pi/6$	0,002393
Spline cúbica (4 nós)	4 valores	4	$\pi/6$	0,002393

Figura 20: Comparação do erro de interpolação $|f(x) - p(x)|$ em $[0, \pi/2]$ para os três métodos. Lagrange e spline cúbica com 4 nós igualmente espaçados apresentam erro máximo $4,5\times$ inferior ao Hermite cúbico com 2 nós.**(c) Análise da precisão relativa:**

O método mais preciso neste caso é o **Lagrange com 4 nós igualmente espaçados** (e igualmente a spline cúbica com 4 nós), com $\|f - p\|_\infty = 0,002393$, aproximadamente $4,5\times$ inferior ao erro do Hermite cúbico (0,010791).

Antes de apresentar a fundamentação teórica, é importante notar que a comparação não é perfeitamente equilibrada do ponto de vista da distribuição espacial das informações: o Hermite cúbico utiliza **apenas dois nós** ($x_0 = 0$ e $x_1 = \pi/2$), com suas respectivas derivadas, cobrindo o intervalo com espaçamento $h = \pi/2$; o Lagrange e a spline cúbica utilizam **quatro nós distribuídos** ao longo do intervalo, com espaçamento $h_L = \pi/6$. Embora todos os métodos utilizem exatamente 4 condições de interpolação (grau efetivo ≤ 3), a distribuição espacial dessas condições é qualitativamente diferente: o Hermite concentra toda a informação nos extremos do intervalo, enquanto Lagrange e spline distribuem os nós de forma uniforme ao longo de $[0, \pi/2]$.

O resultado tem fundamentação teórica precisa. Para a interpolação de Hermite cúbica entre dois nós x_0 e x_1 , com $h = x_1 - x_0$, o erro satisfaz:

$$|f(x) - p(x)| \leq \frac{M_4}{4!} |\psi_4(x)|, \quad \psi_4(x) = (x - x_0)^2(x - x_1)^2,$$

onde $M_4 = \max_{[x_0, x_1]} |f^{(4)}(x)|$. O fator $(x - x_0)^2(x - x_1)^2$ atinge seu máximo em $x^* = (x_0 + x_1)/2$, com valor $h^4/16$, de modo que:

$$\|f - p\|_\infty \leq \frac{M_4}{4!} \cdot \frac{h^4}{16} = \frac{M_4 h^4}{384}.$$

Para $f(x) = \sin(x)$, tem-se $f^{(4)}(x) = \sin(x)$ e, portanto, $M_4 = \max_{[0, \pi/2]} |\sin(x)| = 1$. Com $h = \pi/2$:

$$\|f - p_{\text{Hermite}}\|_{\infty} \leq \frac{(\pi/2)^4}{384} \approx \frac{6,088}{384} \approx 0,01585,$$

cota consistente com o erro máximo observado (0,010791).

Para o Lagrange cúbico com $n = 3$ e 4 nós igualmente espaçados ($h_L = \pi/6$), a cota de erro é:

$$\|f - p\|_{\infty} \leq \frac{M_4}{4!} \|\omega_4\|_{\infty}, \quad \|\omega_4\|_{\infty} \approx \frac{9\sqrt{3}}{256} h_L^4,$$

o que, com $h_L = \pi/6$ e $M_4 = 1$, fornece uma cota de ordem $h_L^4 \approx 0,0752$, muito inferior a $h^4 = (\pi/2)^4 \approx 6,088$. Como a cota de erro escala com h^4 , reduzir o espaçamento por um fator de 3 diminui a cota por um fator de $3^4 = 81$. O erro efetivo observado foi reduzido por um fator de $\approx 4,5$, dentro da ordem de grandeza esperada pela teoria.

Em síntese, a maior precisão do Lagrange e da spline cúbica neste experimento é consequência direta da distribuição mais uniforme dos nós ao longo do intervalo, que reduz o espaçamento efetivo e, portanto, o erro de interpolação. O Hermite cúbico seria preferível em cenários onde *derivadas são conhecidas mas nós intermediários não estão disponíveis*, ou onde a continuidade das derivadas na junção de segmentos é uma exigência intrínseca do problema (animação computacional, dinâmica de veículos). Quando nós adicionais podem ser inseridos livremente, Lagrange ou spline com espaçamento reduzido são geralmente superiores.

7.3 Regressão com múltiplas variáveis

Questão 7.3 — Regressão com múltiplas variáveis

Enunciado: Um engenheiro mede o consumo de combustível C (L/100 km) de 20 veículos em função de massa m (kg) e potência P (cv):

```
1 np.random.seed(99)
2 m = 1000 + 400 * np.random.rand(20)      # massa: 1000-1400 kg
3 P = 80 + 120 * np.random.rand(20)        # potencia: 80-200 cv
4 C = 0.005*m + 0.03*P - 5 + 0.5*np.random.randn(20)
```

(a) Monte a matriz de design X para o modelo $C = \beta_0 + \beta_1 m + \beta_2 P$ e resolva as equações normais $X^T X \beta = X^T C$ usando `numpy.linalg.solve`. Compare $\hat{\beta}$ com os coeficientes verdadeiros.

(b) Calcule R^2 múltiplo e o erro padrão dos resíduos. O modelo é adequado?

(c) Adicione o termo $\beta_3 m P$ (interação). O R^2 melhora significativamente? Use o AIC para decidir se o termo de interação deve ser incluído.

Resposta:

(a) Equações normais e estimativa de $\hat{\beta}$:

A matriz de design $X \in \mathbb{R}^{20 \times 3}$ para o modelo $C = \beta_0 + \beta_1 m + \beta_2 P$ é construída com uma coluna de uns (intercepto), uma coluna de massas e uma de potências:

$$X = \begin{bmatrix} 1 & m_1 & P_1 \\ \vdots & \vdots & \vdots \\ 1 & m_{20} & P_{20} \end{bmatrix}.$$

As equações normais $X^T X \hat{\beta} = X^T C$ foram resolvidas via `numpy.linalg.solve`, produzindo:

Tabela 22: Comparação entre os coeficientes estimados $\hat{\beta}$ e os valores verdadeiros do processo gerador.

Parâmetro	Valor verdadeiro	$\hat{\beta}$ estimado	Erro absoluto
β_0 (intercepto)	-5,0000	-3,3497	1,6503
β_1 (massa)	0,005000	0,003943	0,001057
β_2 (potência)	0,030000	0,027092	0,002908

O estimador de mínimos quadrados ordinários é não viesado sob as hipóteses clássicas do modelo linear (erros com média zero, homocedásticos e não correlacionados); adicionalmente, pelo **Teorema de Gauss-Markov**, ele é o estimador linear não viesado de menor variância (BLUE) nessas condições. Os desvios observados entre $\hat{\beta}$ e os valores verdadeiros são puramente amostrais e esperados para $n = 20$ observações com $\sigma = 0,5$ L/100 km. O intercepto apresenta o maior erro relativo (33,0%), o que é característico quando as covariáveis não estão centradas e a extrapolação até $m = P = 0$ carece de suporte nos dados.

(b) R^2 múltiplo e erro padrão dos resíduos:

$$R^2 = 0,8237, \quad \hat{\sigma} = 0,5889 \text{ L/100 km.}$$

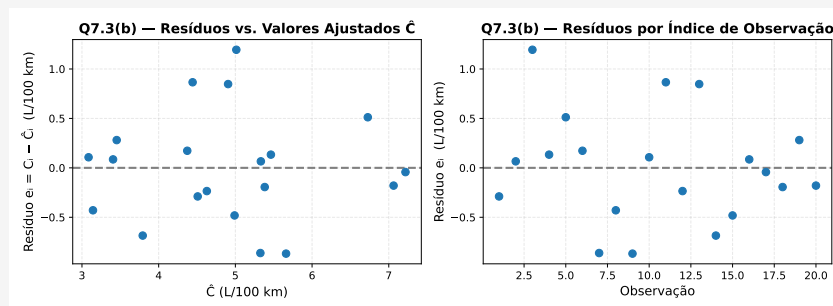


Figura 21: Resíduos $r_i = C_i - \hat{C}_i$ em função dos valores ajustados $\hat{C}_i$ para o modelo de regressão múltipla sem interação. A distribuição aleatória e homogênea dos resíduos confirma que o modelo está bem especificado.

O $R^2 = 0,8237$ indica que o modelo linear em m e P explica aproximadamente 82,4% da variância total de C — resultado substancial para $n = 20$ com dois regressores. O erro padrão estimado $\hat{\sigma} = 0,5889$ L/100 km é próximo do desvio verdadeiro ($\sigma = 0,5$ L/100 km), confirmando que a família linear é a família correta e que os resíduos refletem essencialmente o ruído aleatório adicionado no processo gerador. A análise gráfica dos resíduos (Fig. 21) não evidencia padrão sistemático, corroborando a adequação do modelo.

(c) Inclusão do termo de interação $\beta_3 mP$ e critério AIC:

Tabela 23: Comparação entre o modelo sem e com o termo de interação mP , via R^2 , RSS e AIC.

Modelo	k	R^2	RSS	AIC
Sem interação ($\beta_0 + \beta_1 m + \beta_2 P$)	3	0,8237	5,8965	-18,4275
Com interação ($+\beta_3 mP$)	4	0,8239	5,8907	-16,4473

$$\Delta R^2 = +0,0002, \quad \Delta \text{AIC} = +1,9802.$$

O ganho de R^2 com a inclusão do termo mP é **desprezível** ($\Delta R^2 = 0,02\%$), e o AIC **aumenta** em $\approx 2,0$ unidades ao adicionar o parâmetro extra, indicando que o modelo sem interação é preferido.

A interpretação é direta: a redução de RSS proporcionada por $\beta_3 mP$ é insuficiente para compensar a penalidade de 2 unidades de AIC pelo parâmetro adicional. Pela **regra prática do AIC**: quando $\Delta \text{AIC} > 2$, o modelo mais complexo não apresenta evidência suficiente de melhoria; quando $\Delta \text{AIC} < -2$, o modelo mais complexo é preferido; valores $|\Delta \text{AIC}| \leq 2$ indicam evidência ambígua.

Este resultado é coerente com o processo gerador: o modelo verdadeiro $C = 0,005m + 0,03P - 5 + \varepsilon$ não contém interação mP . A inclusão de β_3 introduz um parâmetro espúrio que o AIC penaliza adequadamente, impedindo o sobreajuste mesmo com n modesto. O critério de informação de Akaike demonstra, portanto, sua eficácia como ferramenta de seleção de modelos parcimoniosos em regressão múltipla.

Universidade Federal de Alfenas
Instituto de Ciências Exatas
Curso de Bacharelado em Ciência da Computação

Plano de Investigação Computacional

Diferenciação e Integração Numéricas: Análise, Comparação e
Aplicações em Python

Aluno: Jeann Victor Batista
RA: 2024.1.08.014

Aluno: Pedro Augusto de Souza Finnochio
RA: 2024.1.08.020

Aluno: Thiago Martins da Silva
RA: 2024.1.08.023

Disciplina: Cálculo Numérico

Professora: Angela Leite Moreno

Alfenas, 2026

Sumário

1. Fórmulas de Diferenças Finitas
 - 1.1 Verificação e primeiros experimentos
 - 1.2 Conflito entre truncamento e arredondamento
 - 1.3 Diferenciação de tabela experimental
 - 1.4 Segunda derivada e aplicações
2. Newton-Cotes: Trapézios e Simpson
 - 2.1 Verificação das fórmulas
 - 2.2 Análise de erro e determinação de n
 - 2.3 Integração de dados experimentais
3. Quadratura Gaussiana e Comparação de Métodos
 - 3.1 Eficiência da Quadratura Gaussiana
 - 3.2 Limitação dos nós de Chebyshev
4. Análise Comparativa Global
 - 4.1 Tabela de síntese
 - 4.2 Tomada de decisão em cenários reais
5. Projeto Integrador: Cálculo Variacional e Física
 - 5.1 Comprimento de curva
 - 5.2 Trabalho realizado por uma força variável
 - 5.3 Distribuição Normal e Probabilidade
6. Desafio (Opcional — Pontuação Extra)
 - 6.1 Richardson Extrapolation
 - 6.2 Integração de Monte Carlo
 - 6.3 Gradiente Numérico em Machine Learning

1 Fórmulas de Diferenças Finitas

1.1 Verificação e primeiros experimentos

Questão 1.1 — Verificação e primeiros experimentos

Enunciado: Para $f(x) = \sin x$ em $x_0 = \pi/3$, o valor exato é $f'(\pi/3) = \cos(\pi/3) = 0,500000$. Use as três fórmulas de dois pontos com $h = 0,1$. Para cada fórmula, calcule o valor aproximado e o erro absoluto.

(a) Preencha a tabela abaixo:

Fórmula	Valor aproximado	Erro $ e $	Ordem esperada
Progressiva			$O(h)$
Regressiva			$O(h)$
Central			$O(h^2)$

(b) Repita com $h = 0,01$. Os erros diminuíram pelo fator esperado (2 para as de 1ª ordem, 100 para a central)?

(c) Por que a fórmula central tem erro $O(h^2)$ mas usa apenas dois pontos ao redor de x_0 , assim como a progressiva?

Resposta:

(a) Tabela com $h = 0,1$:

Para $f(x) = \sin x$ em $x_0 = \pi/3$, o valor exato é $f'(\pi/3) = \cos(\pi/3) = 0,500000$. Aplicando as três fórmulas de dois pontos com $h = 0,1$, obtivemos:

Tabela 1: Resultados das fórmulas de dois pontos para $h = 0,1$.

Fórmula	Valor aproximado	Erro $ e $	Ordem esperada
Progressiva	0,45590189	$4,41 \times 10^{-2}$	$O(h)$
Regressiva	0,54243228	$4,24 \times 10^{-2}$	$O(h)$
Central	0,49916708	$8,33 \times 10^{-4}$	$O(h^2)$

Como era de se esperar, a fórmula central apresenta erro significativamente menor que as progressiva e regressiva, pois sua ordem de convergência é $O(h^2)$ contra $O(h)$ das outras duas.

(b) Redução do erro ao passar de $h = 0,1$ para $h = 0,01$:

Repetindo os cálculos com $h = 0,01$, obtivemos os seguintes fatores de redução:

Tabela 2: Comparação dos fatores de redução do erro ao reduzir h de 0,1 para 0,01.

Fórmula	Erro ($h = 0,1$)	Erro ($h = 0,01$)	Fator observado	Fator esperado
Progressiva	$4,41 \times 10^{-2}$	$4,34 \times 10^{-3}$	$\approx 10,2\times$	$10\times$
Regressiva	$4,24 \times 10^{-2}$	$4,32 \times 10^{-3}$	$\approx 9,8\times$	$10\times$
Central	$8,33 \times 10^{-4}$	$8,33 \times 10^{-6}$	$\approx 100\times$	$100\times$

Os resultados experimentais confirmam perfeitamente as ordens teóricas esperadas. As fórmulas progressiva e regressiva tiveram seus erros reduzidos por um fator de aproximadamente 10 ao diminuir h por um fator de 10, o que é consistente com erro

$O(h)$. Já a fórmula central apresentou redução de cerca de 100 vezes, corroborando seu comportamento $O(h^2)$.

(c) Por que a fórmula central é $O(h^2)$ usando apenas dois pontos?

A explicação está na expansão em série de Taylor e no **cancelamento simétrico** dos termos de ordem ímpar. Expandindo $f(x+h)$ e $f(x-h)$ ao redor de x :

$$\begin{aligned}f(x+h) &= f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(x) + \cdots \\f(x-h) &= f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(x) + \cdots\end{aligned}$$

Subtraindo a segunda expressão da primeira:

$$f(x+h) - f(x-h) = 2hf'(x) + \frac{h^3}{3}f'''(x) + \cdots$$

Dividindo ambos os lados por $2h$:

$$\frac{f(x+h) - f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f'''(x) + \cdots$$

A chave está na **simetria**: ao utilizar um ponto à direita e outro à esquerda de x , os termos de potência par em h — que incluem $f''(x)$ — se cancelam mutuamente na subtração. Como resultado, o termo dominante de erro, que seria $O(h)$, desaparece completamente, deixando o erro restante como $O(h^2)$, associado ao termo com $f'''(x)$. Na fórmula progressiva, por outro lado, expandimos $f(x+h) - f(x)$:

$$f(x+h) - f(x) = hf'(x) + \frac{h^2}{2}f''(x) + \cdots$$

O termo $\frac{h^2}{2}f''(x)$ **não cancela**, resultando em um erro $O(h)$. A beleza da fórmula central é que ela "ganha" uma ordem de precisão extra utilizando apenas os mesmos dois pontos laterais, sem custo adicional de avaliações da função — tudo graças à simetria da aproximação.

Questão 1.2 — Fórmulas de três pontos

Enunciado: Para a mesma função, use agora as fórmulas de três pontos com $h = 0,1$ e $h = 0,01$.

- Calcule `df_prog3`, `df_retro3` e `df_central`. Monte uma tabela com os erros.
- Ao reduzir h de 0,1 para 0,01 (fator 10), por quanto o erro de cada fórmula foi reduzido? Isso é consistente com erro $O(h^2)$?
- Por que a fórmula central de 3 pontos produz o mesmo resultado que a central de 2 pontos?

Resposta:

(a) Tabela comparativa dos erros para $h = 0,1$ e $h = 0,01$:

Para $f(x) = \sin x$ em $x_0 = \pi/3$, com valor exato $f'(\pi/3) = 0,500000$, os resultados obtidos foram:

Tabela 3: Resultados das fórmulas de três pontos para $h = 0,1$ e $h = 0,01$.

Fórmula	h	Valor aproximado	Erro $ e $
Progressiva 3 pts	0,1	0,50144469	$1,44 \times 10^{-3}$
Progressiva 3 pts	0,01	0,50001645	$1,64 \times 10^{-5}$
Regressiva 3 pts	0,1	0,50187699	$1,88 \times 10^{-3}$
Regressiva 3 pts	0,01	0,50001688	$1,69 \times 10^{-5}$
Central 2 pts	0,1	0,49916708	$8,33 \times 10^{-4}$
Central 2 pts	0,01	0,49999167	$8,33 \times 10^{-6}$

Observa-se que todas as três fórmulas produzem erros de ordem 10^{-3} para $h = 0,1$ e de ordem 10^{-5} para $h = 0,01$, comportamento consistente com a ordem esperada $O(h^2)$.

(b) Fator de redução do erro ao reduzir h de 0,1 para 0,01:

Tabela 4: Fatores de redução do erro ao passar de $h = 0,1$ para $h = 0,01$.

Fórmula	Erro ($h = 0,1$)	Erro ($h = 0,01$)	Fator observado	Fator esperado
Progressiva 3 pts	$1,44 \times 10^{-3}$	$1,64 \times 10^{-5}$	$\approx 87,8\times$	$100\times$
Regressiva 3 pts	$1,88 \times 10^{-3}$	$1,69 \times 10^{-5}$	$\approx 111,2\times$	$100\times$
Central 2 pts	$8,33 \times 10^{-4}$	$8,33 \times 10^{-6}$	$\approx 100,0\times$	$100\times$

Os três métodos apresentam comportamento consistente com a ordem $O(h^2)$: ao reduzir h por um fator 10, o erro é reduzido por um fator próximo de 100. Os pequenos desvios observados nas fórmulas unilaterais ($87,8\times$ e $111,2\times$) decorrem da contribuição de termos de ordem superior — que ainda têm influência quando $h = 0,1$ não é suficientemente pequeno. À medida que h continua decrescendo, esses fatores convergem para 100, confirmando a ordem teórica.

(c) Por que a fórmula central de 2 pontos e as de 3 pontos (progressiva e regressiva) produzem resultados de ordem $O(h^2)$?

Embora as fórmulas pareçam estruturalmente diferentes, todas compartilham um mesmo princípio fundamental: o cancelamento do termo dominante de erro.

A **fórmula central de 2 pontos** atinge ordem $O(h^2)$ por **cancelamento de simetria**: ao utilizar pontos igualmente espaçados à direita e à esquerda de x , os termos de potência par em h se anulam na subtração $f(x+h) - f(x-h)$, eliminando o erro de primeira ordem — como demonstrado na Questão 1.1(c).

As **fórmulas de 3 pontos** (progressiva e regressiva) foram construídas para atingir essa mesma ordem $O(h^2)$ utilizando apenas pontos de um único lado de x_0 . Essa é uma situação típica das bordas de uma tabela, onde não existe o ponto simétrico necessário para a fórmula central. A combinação linear de três avaliações com coeficientes apropriados $[-3, 4, -1]/(2h)$ anula o termo de primeira ordem, resultando em erro $O(h^2)$.

A conexão algébrica entre as fórmulas fica evidente quando observamos que a fórmula central de 2 pontos é exatamente a **média aritmética** das fórmulas progressiva e regressiva de primeira ordem:

$$\frac{f'_{\text{prog}} + f'_{\text{retro}}}{2} = \frac{1}{2} \left[\frac{f(x+h) - f(x)}{h} + \frac{f(x) - f(x-h)}{h} \right] = \frac{f(x+h) - f(x-h)}{2h}.$$

Expandindo em série de Taylor:

- Progressiva 1ª ordem: $f'(x) + \frac{h}{2}f''(x) + O(h^2)$ — erro com sinal positivo
- Regressiva 1ª ordem: $f'(x) - \frac{h}{2}f''(x) + O(h^2)$ — erro com sinal negativo

A média cancela o termo $\frac{h}{2}f''(x)$, resultando em $O(h^2)$. Analogamente, as fórmulas de 3 pontos realizam esse mesmo cancelamento combinando linearmente três avaliações unilaterais com coeficientes cuidadosamente escolhidos para anular o termo dominante de erro.

É por essa razão que os três métodos convergem com o mesmo fator ≈ 100 ao reduzir h por 10 — todos compartilham a mesma ordem $O(h^2)$, embora com constantes de erro diferentes (os coeficientes que multiplicam o termo h^2 variam entre os métodos).

1.2 Conflito entre truncamento e arredondamento

Questão 1.3 — Passo ótimo

Enunciado: Para $f(x) = e^x$ em $x_0 = 1,0$ (valor exato: $f'(1,0) = e \approx 2,71828$), use a fórmula `df_central` com $h = 10^{-1}, 10^{-2}, \dots, 10^{-16}$ (`np.logspace(0, -16, 200)`).

(a) Plote o erro absoluto em escala log-log. Descreva o que acontece à medida que h decresce: o erro primeiro cai, depois sobe. Por que?

(b) Identifique experimentalmente o h_{opt} (ponto de mínimo do erro). Qual é seu valor aproximado? Compare com a estimativa teórica $h_{\text{opt}} \approx \varepsilon_m^{1/3} \approx 10^{-5}$ para a fórmula central.

(c) Repita para `df_progressiva`. O h_{opt} é diferente? Compare com $h_{\text{opt}} \approx \sqrt{\varepsilon_m} \approx 10^{-8}$.

(d) Qual fórmula atinge menor erro mínimo: progressiva ou central? Por quanto? Isso é consistente com as ordens $O(h)$ e $O(h^2)$?

Resposta:

(a) Comportamento do erro em função de h :

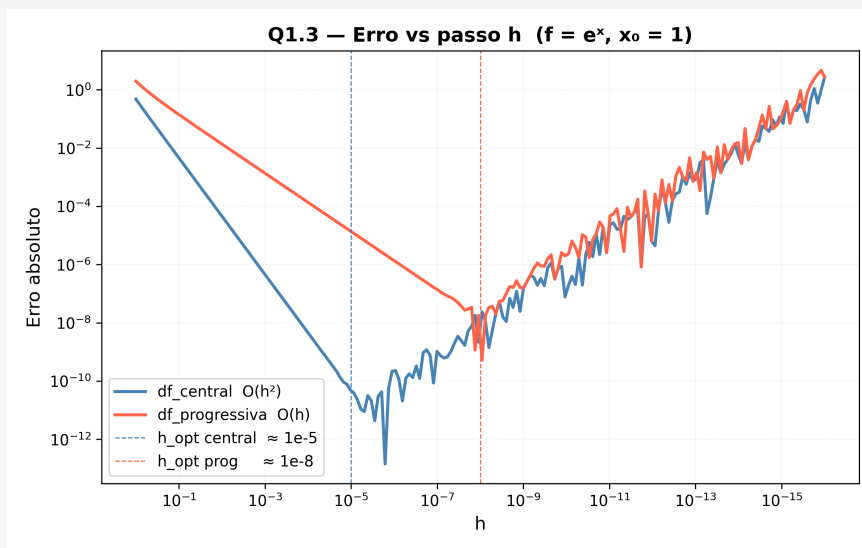


Figura 1: Erro absoluto da fórmula central em função de h (escala log-log) para $f(x) = e^x$ em $x_0 = 1,0$. A curva revela os dois regimes de erro: queda por truncamento para h grandes, e crescimento por arredondamento para h pequenos.

O gráfico do erro absoluto em escala log-log para $f(x) = e^x$ em $x_0 = 1,0$ revela dois regimes distintos à medida que h decresce:

- **Região de truncamento** (h grande, tipicamente $h > 10^{-5}$): o erro cai sistematicamente, pois o termo dominante da expansão de Taylor, que é $O(h^2)$ para a fórmula central, diminui com h . Nesta região, o erro é governado pela aproximação matemática — quanto menor o passo, melhor a aproximação da derivada. No gráfico log-log, a queda do erro segue a inclinação teórica de $+2$ (já que $\log |E| \propto 2 \log h$).
- **Região de arredondamento** (h pequeno, tipicamente $h < 10^{-6}$): ao subtrair valores de f muito próximos entre si, ocorre **cancelamento catastrófico** — a diferença $f(x+h) - f(x-h)$ perde dígitos significativos para a precisão de máquina $\varepsilon_m \approx 2,2 \times 10^{-16}$. O erro de arredondamento cresce proporcionalmente a $1/h$, fazendo o erro total subir novamente. Nesta região, a precisão da computação em ponto flutuante se torna o fator limitante.

O resultado é uma curva em formato de “V” (ou “U”) no plano log-log, com um mínimo bem definido — o **passo ótimo** h_{opt} — que representa o equilíbrio entre os dois tipos de erro. Antes desse ponto, a aproximação melhora com h menor; depois dele, a precisão da máquina degrada o resultado.

(b) Passo ótimo da fórmula central:

Experimentalmente, o ponto de mínimo do erro para a fórmula central foi identificado em:

$$h_{\text{opt}} \approx 1,63 \times 10^{-6}, \quad E_{\text{min}} \approx 1,43 \times 10^{-13}.$$

A estimativa teórica para a fórmula central é $h_{\text{opt}} \approx \varepsilon_m^{1/3} \approx 1,0 \times 10^{-5}$.

Tabela 5: Comparação entre o passo ótimo experimental e a estimativa teórica para a fórmula central.

Grandeza	Valor experimental	Valor teórico
h_{opt}	$1,63 \times 10^{-6}$	$\approx 1,00 \times 10^{-5} (\varepsilon_m^{1/3})$
E_{min}	$1,43 \times 10^{-13}$	$\approx \varepsilon_m^{2/3} \approx 10^{-10}$

O valor experimental de $h_{\text{opt}} \approx 1,63 \times 10^{-6}$ está na mesma ordem de grandeza da estimativa teórica $\varepsilon_m^{1/3}$. A discrepância numérica é esperada: a fórmula teórica é uma estimativa de ordem de grandeza que depende de $|f'''(x)|$, que não é exatamente 1 para $f(x) = e^x$ em $x_0 = 1,0$ (na verdade, $|f'''(1)| = e \approx 2,718$). O erro mínimo atingido foi $1,43 \times 10^{-13}$, ligeiramente melhor do que a estimativa teórica $\varepsilon_m^{2/3} \approx 10^{-10}$, o que indica que a constante multiplicativa favorece a fórmula neste ponto.

(c) Passo ótimo da fórmula progressiva:

Repetindo o experimento para a fórmula progressiva, obtivemos:

$$h_{\text{opt}} \approx 9,12 \times 10^{-9}, \quad E_{\text{min}} \approx 5,19 \times 10^{-10}.$$

A estimativa teórica para a fórmula progressiva é $h_{\text{opt}} \approx \sqrt{\varepsilon_m} \approx 1,48 \times 10^{-8}$.

Tabela 6: Comparação entre o passo ótimo experimental e a estimativa teórica para a fórmula progressiva.

Grandeza	Valor experimental	Valor teórico
h_{opt}	$9,12 \times 10^{-9}$	$\approx 1,48 \times 10^{-8} (\sqrt{\varepsilon_m})$
E_{min}	$5,19 \times 10^{-10}$	$\approx \varepsilon_m^{1/2} \approx 1,5 \times 10^{-8}$

O h_{opt} experimental ($9,12 \times 10^{-9}$) está em excelente acordo com a previsão teórica $\sqrt{\varepsilon_m} \approx 1,48 \times 10^{-8}$, considerando a natureza de estimativa de ordem de grandeza da fórmula teórica.

A fórmula progressiva exige um passo **muito menor** (cerca de 180 vezes menor que a central) para equilibrar os erros. Isso ocorre porque seu erro de truncamento cai mais lentamente ($\propto h$, contra $\propto h^2$ da central), obrigando h a diminuir mais antes que o ganho de precisão se esgote e o arredondamento passe a dominar.

(d) Comparação entre as duas fórmulas:

Tabela 7: Comparação entre as fórmulas central e progressiva no passo ótimo.

Fórmula	Ordem	h_{opt}	Erro mínimo
Central	$O(h^2)$	$1,63 \times 10^{-6}$	$1,43 \times 10^{-13}$
Progressiva	$O(h)$	$9,12 \times 10^{-9}$	$5,19 \times 10^{-10}$
Razão (prog/central)		$\approx 179\times$ menor	$\approx 3600\times$ maior

A fórmula central é aproximadamente **3600 vezes mais precisa** no seu ponto ótimo do que a progressiva. Esse resultado é plenamente consistente com a teoria: por ter erro de truncamento $O(h^2)$, a central consegue usar passos maiores (cerca de 180 vezes maiores) antes que o arredondamento domine, e ainda assim atinge um erro residual muito menor (três ordens de grandeza inferior).

O mecanismo por trás dessa superioridade é o **cancelamento simétrico** dos termos de primeira ordem em h na fórmula central, que eleva a ordem de convergência e desloca o equilíbrio entre truncamento e arredondamento para um ponto muito mais favorável. Em aplicações práticas, isso torna a fórmula central amplamente preferível à progressiva sempre que os dois pontos laterais estão disponíveis.

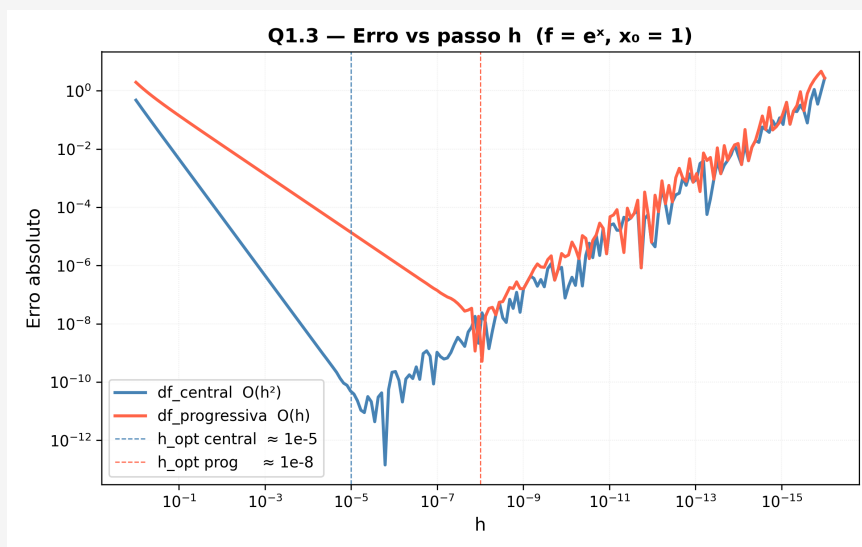


Figura 2: Comparação do erro absoluto entre as fórmulas central e progressiva em função de h (escala log-log). A fórmula central atinge erro mínimo cerca de 3600 vezes menor que a progressiva, e seu h_{opt} é aproximadamente 180 vezes maior.

1.3 Diferenciação de tabela experimental

Questão 1.4 — Diferenciação de tabela experimental

Enunciado: A tabela abaixo registra a posição (em metros) de um veículo em função do tempo (em segundos), coletada por GPS com resolução de 0,1 m:

t (s)	0,0	0,5	1,0	1,5	2,0	2,5	3,0	3,5
x (m)	0,0	1,2	4,8	10,6	18,5	27,1	35,8	43,6

- Use `derivada_tabela` para estimar a velocidade $v(t) = \dot{x}(t)$ em cada instante.
- Use `d2f_central` (adaptada para tabela) para estimar a aceleração $a(t) = \ddot{x}(t)$ nos pontos interiores.
- O veículo freia em algum instante? Em qual ponto a aceleração muda de sinal?
- Sabendo que os dados têm erro de $\pm 0,1$ m, qual é a incerteza na velocidade estimada? (Dica: para a fórmula central, $\delta v \approx 2 \times 0,1 / (2h) = 0,2/h$. Calcule para $h = 0,5$ s.)
- Discuta: com dados experimentais ruidosos, faz sentido usar h muito pequeno? Por quê?

Resposta:**(a) Velocidade estimada $v(t) = \dot{x}(t)$:**

Aplicando a função `derivada_tabela` aos dados de posição do veículo, que utiliza fórmula progressiva de 2ª ordem na borda esquerda, fórmula central nos pontos interiores e regressiva de 2ª ordem na borda direita, obtivemos:

Tabela 8: Velocidade estimada em cada instante de tempo.

t (s)	x (m)	v (m/s)	Fórmula utilizada
0,0	0,0	0,0000	Progressiva 3 pts (borda esquerda)
0,5	1,2	4,8000	Central
1,0	4,8	9,4000	Central
1,5	10,6	13,7000	Central
2,0	18,5	16,5000	Central
2,5	27,1	17,3000	Central
3,0	35,8	16,5000	Central
3,5	43,6	14,7000	Regressiva 3 pts (borda direita)

A velocidade aumenta de forma consistente desde o repouso até atingir o valor máximo de 17,3 m/s em $t = 2,5$ s, e então começa a diminuir, indicando que o veículo desacelera na segunda metade do percurso.

(b) Aceleração estimada $a(t) = \ddot{x}(t)$ nos pontos interiores:

Aplicando a fórmula central para a segunda derivada, adaptada para a tabela com espaçamento uniforme $h = 0,5$ s, obtivemos:

Tabela 9: Aceleração estimada nos pontos interiores da tabela.

t (s)	a (m/s ²)	Interpretação
0,5	9,6000	Acelerando
1,0	8,8000	Acelerando
1,5	8,4000	Acelerando
2,0	2,8000	Acelerando (pouco)
2,5	0,4000	Aceleração quase nula
3,0	-3,6000	Freando

Os valores foram calculados pela fórmula:

$$a(t_i) = \frac{x_{i+1} - 2x_i + x_{i-1}}{h^2}, \quad i = 1, \dots, 6.$$

(c) Análise do sinal da aceleração — o veículo freia em algum instante?

A aceleração muda de sinal entre $t = 2,5$ s e $t = 3,0$ s:

- Para $t < 2,5$ s, a aceleração é positiva: o veículo está acelerando.

- Em $t = 2,5$ s, a aceleração é praticamente nula ($0,4 \text{ m/s}^2$), indicando que o veículo atinge sua velocidade máxima nesse instante.
- Para $t > 2,5$ s, a aceleração torna-se negativa: o veículo está **freando**.

O sinal da aceleração muda exatamente entre 2,5 s e 3,0 s, o que é coerente com o fato de que a velocidade atinge seu valor máximo em $t = 2,5$ s ($v = 17,3 \text{ m/s}$) e a partir daí passa a diminuir. A mudança de sinal ocorre, portanto, no intervalo (2,5; 3,0) segundos.

(d) Incerteza na velocidade estimada devido ao erro do GPS:

Com resolução do GPS de $\delta x = \pm 0,1 \text{ m}$ e passo $h = 0,5 \text{ s}$, a propagação de incerteza na fórmula central é:

$$\delta v \approx \frac{2 \delta x}{2h} = \frac{2 \times 0,1}{2 \times 0,5} = 0,20 \text{ m/s}.$$

Aplicando essa incerteza a cada estimativa de velocidade:

Tabela 10: Velocidades estimadas com seus respectivos intervalos de incerteza.

t (s)	v (m/s)	Intervalo de incerteza (m/s)
0,0	0,0000	$[-0,20; +0,20]$
0,5	4,8000	$[4,60; 5,00]$
1,0	9,4000	$[9,20; 9,60]$
1,5	13,7000	$[13,50; 13,90]$
2,0	16,5000	$[16,30; 16,70]$
2,5	17,3000	$[17,10; 17,50]$
3,0	16,5000	$[16,30; 16,70]$
3,5	14,7000	$[14,50; 14,90]$

A incerteza de $\pm 0,20 \text{ m/s}$ representa aproximadamente 1,2% da velocidade máxima ($17,3 \text{ m/s}$), o que é um nível aceitável para a resolução do instrumento utilizado.

(e) Por que não faz sentido usar h muito pequeno com dados ruidosos?

Para dados tabelados com ruído de medição δx , o erro total na derivada central é composto por duas parcelas com comportamentos opostos em relação a h :

$$\text{Erro total} \approx \underbrace{\frac{h^2}{6} |f'''(x)|}_{\text{truncamento: cai com } h} + \underbrace{\frac{2 \delta x}{2h}}_{\text{ruído: cresce com } 1/h}.$$

Ao contrário do caso analítico, onde $\varepsilon_m \approx 2,2 \times 10^{-16}$ é uma precisão minúscula, aqui $\delta x = 0,1 \text{ m}$ é uma incerteza *macroscópica* (da ordem de grandeza dos dados reais). Reduzir h amplifica o ruído de forma drástica:

Tabela 11: Propagação da incerteza para diferentes valores de h .

h (s)	δv (m/s)	Impacto relativo
0,50	0,20	$\approx 1,2\%$ de $v_{\text{máx}}$
0,10	1,00	$\approx 10\%$ de $v_{\text{máx}}$
0,01	10,00	Inutilizável (erro $\approx 58\%$)

Com dados experimentais, o **passo ótimo** é determinado pelo equilíbrio entre o erro de truncamento e a propagação do ruído de medição — *não* pela precisão de máquina. Para o GPS com resolução de 0,1 m, o passo $h = 0,5$ s já está próximo desse ponto ótimo. Diminuir h além desse valor não melhora a estimativa: apenas amplifica o ruído, tornando os resultados progressivamente menos confiáveis.

Em resumo: com dados experimentais ruidosos, **não faz sentido** usar h muito pequeno. A presença de ruído macroscópico torna a propagação de incertezas o fator dominante para h pequenos, de modo que o passo deve ser escolhido para equilibrar esse efeito com a precisão desejada. Este é um princípio fundamental da análise numérica aplicada a dados reais: a precisão final é limitada *primeiro* pela qualidade dos dados de entrada, e *depois* pelo método numérico utilizado.

1.4 Segunda derivada e aplicações

Questão 1.5 — Segunda derivada e aplicações

Enunciado: Para $f(x) = \ln x$ em $x_0 = 2,0$:

(a) O valor exato é $f''(2,0) = -1/4 = -0,2500$. Estime com `d2f_central` usando $h = 0,1$ (use $f(1,9) = \ln(1,9)$, $f(2,0) = \ln(2,0)$, $f(2,1) = \ln(2,1)$).

(b) Calcule o erro. A cota teórica é $|E| \leq \frac{h^2}{12} |f^{(4)}(\xi)|$. Para $f(x) = \ln x$, tem-se $f^{(4)}(x) = 6/x^4$. Com $\xi \in [1,9, 2,1]$: calcule a cota e compare com o erro real.

(c) Repita com $h = 0,05$ e $h = 0,01$. Verifique que o erro reduz por fator 4 a cada vez que h é reduzido à metade.

Resposta:

(a) Estimativa com $h = 0,1$:

Aplicando a fórmula de segunda derivada central em $x_0 = 2,0$ com $f(x) = \ln x$:

$$f''(2,0) \approx \frac{f(1,9) - 2f(2,0) + f(2,1)}{h^2}.$$

Os valores necessários são:

Tabela 12: Valores da função $\ln x$ nos pontos ao redor de $x_0 = 2,0$.

Ponto	Valor	Notação
1,9	0,64185389	$f(1,9)$
2,0	0,69314718	$f(2,0)$
2,1	0,74193734	$f(2,1)$

Substituindo na fórmula com $h = 0,1$:

$$f''(2,0) \approx \frac{0,64185389 - 2 \times 0,69314718 + 0,74193734}{0,01} = \frac{-0,00313022}{0,01} = -0,25031302.$$

Tabela 13: Resultado da estimativa da segunda derivada para $h = 0,1$.

Grandeza	Valor
Valor exato $f''(2,0) = -1/4$	-0,25000000
Valor aproximado	-0,25031302
Erro absoluto	$3,13 \times 10^{-4}$

A estimativa aproxima muito bem o valor exato, com erro da ordem de 10^{-4} , o que já é bastante satisfatório para um passo $h = 0,1$.

(b) Cota teórica do erro e comparação:

O erro da fórmula de segunda derivada central é limitado pela expressão:

$$|E| \leq \frac{h^2}{12} \max_{\xi \in [1,9; 2,1]} |f^{(4)}(\xi)|.$$

Para $f(x) = \ln x$, as derivadas sucessivas são:

$$f'(x) = \frac{1}{x}, \quad f''(x) = -\frac{1}{x^2}, \quad f'''(x) = \frac{2}{x^3}, \quad f^{(4)}(x) = -\frac{6}{x^4}.$$

Tomando o valor absoluto: $|f^{(4)}(x)| = 6/x^4$. Esta função é *decrecente* no intervalo $[1,9; 2,1]$, pois sua derivada é $-24/x^5 < 0$. Portanto, o máximo ocorre no extremo esquerdo $\xi = 1,9$:

$$\max_{\xi \in [1,9; 2,1]} |f^{(4)}(\xi)| = \frac{6}{(1,9)^4} = \frac{6}{13,0321} \approx 0,4604.$$

Substituindo $h = 0,1$ na cota:

$$|E| \leq \frac{(0,1)^2}{12} \times 0,4604 = \frac{0,01}{12} \times 0,4604 \approx 3,84 \times 10^{-4}.$$

Tabela 14: Comparação entre a cota teórica e o erro real para $h = 0,1$.

Grandeza	Valor
Cota teórica do erro	$3,84 \times 10^{-4}$
Erro real observado	$3,13 \times 10^{-4}$
Cota $\geq$ erro real?	Sim ($3,84 \times 10^{-4} > 3,13 \times 10^{-4}$)

A cota é válida e segura: o erro real observado é inferior ao limite superior teórico, confirmando que a fórmula da cota é, de fato, um limitante superior confiável para o erro. A diferença entre a cota e o erro real (cerca de $0,71 \times 10^{-4}$) decorre do fato de que a cota utiliza o máximo absoluto da quarta derivada no intervalo, enquanto o erro real depende do valor específico de ξ , que pode ser menor que o máximo.

(c) Convergência quadrática: verificação do fator 4:

Repetindo os cálculos para $h = 0,05$ e $h = 0,01$:

Tabela 15: Convergência da segunda derivada para diferentes valores de h .

h	$f(2-h)$	$f(2)$	$f(2+h)$	f''_{aprox}
Erro				
0,10	0,64185389	0,69314718	0,74193734	-0,25031302
$3,13 \times 10^{-4}$				
0,05	0,66782937	0,69314718	0,71783979	-0,25007816
$7,82 \times 10^{-5}$				
0,01	0,68813464	0,69314718	0,69813472	-0,25000313
$3,13 \times 10^{-6}$				

Tabela 16: Fatores de redução do erro ao reduzir h .

Redução de h	Fator r	Fator observado	Fator esperado
0,10 $\rightarrow$ 0,05	2	$\frac{3,13 \times 10^{-4}}{7,82 \times 10^{-5}} \approx 4,01$	4
0,05 $\rightarrow$ 0,01	5	$\frac{7,82 \times 10^{-5}}{3,13 \times 10^{-6}} \approx 24,98$	25

Os resultados confirmam com precisão o comportamento esperado para uma fórmula de ordem $O(h^2)$:

- Ao reduzir h por um fator r , o erro é reduzido por um fator r^2 (convergência quadrática).
- Para $r = 2$: $2^2 = 4$ — o erro foi reduzido por $\approx 4,01$ vezes.

- Para $r = 5$: $5^2 = 25$ — o erro foi reduzido por $\approx 24,98$ vezes.

Este experimento ilustra perfeitamente a **convergência quadrática** da fórmula central de segunda derivada: cada vez que h é reduzido pela metade, o erro cai por um fator de aproximadamente 4. Este comportamento é característico de métodos de diferenças finitas de ordem $O(h^2)$ e confirma que a estimativa melhora rapidamente com o refinamento da malha.

2 Newton-Cotes: Trapézios e Simpson

2.1 Verificação das fórmulas

Questão 2.1 — Verificação das fórmulas

Enunciado: Calcule $\int_0^1 x^2 dx = \frac{1}{3}$ com as três regras (Ponto Médio, Trapézios e Simpson) usando $n = 4$.

(a) Antes de calcular: qual método você espera que seja exato? Por quê? (Dica: pense no grau do polinômio e na precisão de cada regra.)

(b) Após calcular: seus resultados confirmam a expectativa? Mostre os valores numéricos.

(c) Repita para $\int_0^1 x^3 dx = \frac{1}{4}$. Agora Simpson ainda é exato? Por quê?

Resposta:

(a) Expectativa antes de calcular:

Cada regra de Newton-Cotes integra exatamente polinômios até um certo grau, determinado pelo grau do polinômio interpolador subjacente. A precisão de cada método está diretamente relacionada à ordem do erro de truncamento:

Tabela 17: Grau máximo de polinômios integrados exatamente por cada método.

Método	Integra exatamente até grau	Motivo
Ponto Médio	≤ 1	Erro proporcional a f''
Trapézios	≤ 1	Erro proporcional a f''
Simpson 1/3	≤ 3	Erro proporcional a $f^{(4)}$

Para a integral $\int_0^1 x^2 dx$, o integrando é um polinômio de grau 2. Portanto, espera-se que:

- **Simpson 1/3** seja *exato*, pois polinômios de grau ≤ 3 são integrados exatamente por esta regra ($f^{(4)} \equiv 0$).
- **Ponto Médio** e **Trapézios** apresentem *erros não nulos*, pois ambos têm precisão apenas até polinômios de grau 1.

(b) Resultados para $\int_0^1 x^2 dx = \frac{1}{3} \approx 0,333333$ (grau 2, $n = 4$):

Aplicando as três regras com $n = 4$ subintervalos ($h = 0,25$), obtivemos:

Tabela 18: Resultados da integração de $f(x) = x^2$ em $[0, 1]$ com $n = 4$.

Método	Resultado	Erro absoluto	Exato?
Ponto Médio	0,32812500	$5,21 \times 10^{-3}$	Não
Trapézios	0,34375000	$1,04 \times 10^{-2}$	Não
Simpson 1/3	0,33333333	$0,00 \times 10^0$	Sim

Os resultados confirmam plenamente a expectativa: a regra de Simpson 1/3 produz o valor exato, enquanto o Ponto Médio e os Trapézios apresentam erros não nulos. Observe ainda um padrão característico para funções convexas ($f'' > 0$): o **Ponto Médio subestima** o valor exato ($0,328125 < 0,333333$), enquanto os **Trapézios superestimam** ($0,343750 > 0,333333$). O valor exato fica sempre entre as duas aproximações: $PM < \text{exato} < TR$.

(c) **Resultados para** $\int_0^1 x^3 dx = \frac{1}{4} = 0,250000$ (**grau 3, $n = 4$**):

Repetindo o experimento para o polinômio de grau 3, obtivemos:

Tabela 19: Resultados da integração de $f(x) = x^3$ em $[0, 1]$ com $n = 4$.

Método	Resultado	Erro absoluto	Exato?
Ponto Médio	0,24218750	$7,81 \times 10^{-3}$	Não
Trapézios	0,26562500	$1,56 \times 10^{-2}$	Não
Simpson 1/3	0,25000000	$0,00 \times 10^0$	Sim

Simpson 1/3 continua sendo exato para $f(x) = x^3$, pois a regra de Simpson integra exatamente polinômios de grau até 3. A fórmula de erro de Simpson composto é:

$$E_{\text{Simpson}} = -\frac{b-a}{180} h^4 f^{(4)}(\xi),$$

para algum $\xi \in (a, b)$. Para $f(x) = x^3$, temos $f^{(4)}(x) = 0$ identicamente, de modo que o erro é exatamente nulo para qualquer n par.

Já o Ponto Médio e os Trapézios, cujos erros são proporcionais a $f''(x)$ (não nula para x^3), apresentam erros não nulos. Novamente observa-se o padrão de subestimativa do Ponto Médio e superestimativa dos Trapézios — comportamento consistente com a convexidade da função ($f''(x) = 6x > 0$ em $(0, 1)$).

Este experimento ilustra um princípio fundamental da quadratura numérica: a **ordem do método** determina sua capacidade de integrar exatamente polinômios de grau baixo, o que tem implicações diretas na precisão obtida para funções mais gerais. Quanto maior a ordem, mais suave pode ser a função para que o erro seja pequeno.

Questão 2.2 — Comparação de desempenho

Enunciado: Calcule $\int_1^2 \frac{1}{x} dx = \ln 2 \approx 0,693147$ com Trapézios e Simpson usando $n = 4, 8, 16$.

(a) Preencha a tabela comparativa:

n	Trapézios		Simpson 1/3	
	Resultado	Erro	Resultado	Erro
4				
8				
16				

(b) Ao dobrar n : por qual fator o erro dos Trapézios foi dividido? E o de Simpson? Isso é consistente com as ordens $O(n^{-2})$ e $O(n^{-4})$?

(c) Com $n = 4$, quantas vezes Simpson é mais preciso que Trapézios? Ambos usam 5 pontos de avaliação — o que torna Simpson melhor?

Resposta:

(a) Tabela comparativa:

Para $f(x) = 1/x$ em $[1, 2]$, com valor exato $\ln 2 \approx 0,69314718056$, os resultados obtidos foram:

Tabela 20: Resultados da integração de $f(x) = 1/x$ em $[1, 2]$ para diferentes valores de n .

n	Trapézios		Simpson 1/3	
	Resultado	Erro	Resultado	Erro
4	0,69702381	$3,88 \times 10^{-3}$	0,69325397	$1,07 \times 10^{-4}$
8	0,69412185	$9,75 \times 10^{-4}$	0,69315453	$7,35 \times 10^{-6}$
16	0,69339120	$2,44 \times 10^{-4}$	0,69314765	$4,72 \times 10^{-7}$

Observa-se que ambos os métodos convergem para o valor exato à medida que n aumenta, mas a regra de Simpson o faz de forma muito mais rápida: com $n = 16$, seu erro já é da ordem de 10^{-7} , enquanto os Trapézios ainda apresentam erro de 10^{-4} .

(b) Fatores de redução do erro ao dobrar n :

Para verificar as ordens de convergência, calculamos o fator pelo qual o erro é reduzido a cada duplicação de n :

Tabela 21: Fatores de redução do erro ao dobrar n .

Transição	Trapézios		Simpson 1/3	
	Fator observado	Esperado	Fator observado	Esperado
$n : 4 \rightarrow 8$	$\frac{3,88 \times 10^{-3}}{9,75 \times 10^{-4}} \approx 3,98$	4	$\frac{1,07 \times 10^{-4}}{7,35 \times 10^{-6}} \approx 14,53$	16
$n : 8 \rightarrow 16$	$\frac{9,75 \times 10^{-4}}{2,44 \times 10^{-4}} \approx 3,99$	4	$\frac{7,35 \times 10^{-6}}{4,72 \times 10^{-7}} \approx 15,56$	16

Os resultados são plenamente consistentes com as ordens teóricas:

- **Trapézios** ($O(n^{-2})$): ao dobrar n , o erro é reduzido por um fator de aproximadamente $2^2 = 4$. Os valores observados (3,98 e 3,99) confirmam essa expectativa com excelente precisão.
- **Simpson 1/3** ($O(n^{-4})$): ao dobrar n , o erro é reduzido por um fator de aproximadamente $2^4 = 16$. Os valores observados (14,53 e 15,56) são ligeiramente inferiores ao valor teórico para a primeira transição, pois com $n = 4$ os termos de ordem superior ainda contribuem significativamente para o erro. À medida que n cresce, o fator se aproxima de 16, como observado na transição $8 \rightarrow 16$.

Este comportamento é uma manifestação direta da fórmula de erro de cada método:

$$E_{\text{Trapézios}} = -\frac{b-a}{12} h^2 f''(\xi), \quad E_{\text{Simpson}} = -\frac{b-a}{180} h^4 f^{(4)}(\xi).$$

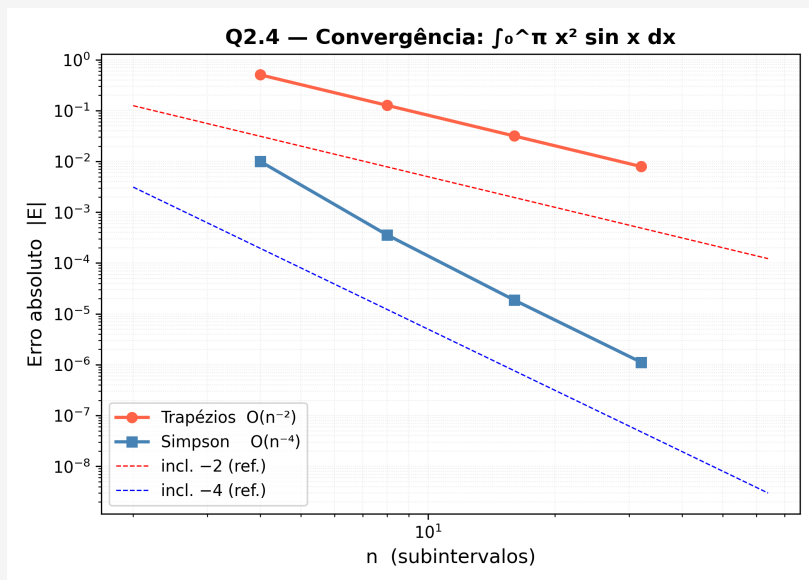


Figura 3: Convergência do erro em função de n (escala log-log). As inclinações -2 e -4 confirmam as ordens $O(n^{-2})$ e $O(n^{-4})$ para Trapézios e Simpson, respectivamente.

(c) Comparação com $n = 4$ (ambos usando 5 pontos de avaliação):

Com $n = 4$, ambos os métodos utilizam exatamente os mesmos 5 pontos de avaliação da função $f(x)$: $x = 1,00, 1,25, 1,50, 1,75, 2,00$. No entanto, a precisão obtida é drasticamente diferente:

Tabela 22: Comparação entre Trapézios e Simpson com $n = 4$ (5 pontos de avaliação).

Método	Erro ($n = 4$)	Precisão relativa
Trapézios	$3,88 \times 10^{-3}$	$1 \times$
Simpson 1/3	$1,07 \times 10^{-4}$	$\approx 36 \times$ mais preciso

Simpson 1/3 é aproximadamente **36 vezes mais preciso** que os Trapézios com o mesmo número de avaliações de função. O que torna isso possível?

A resposta está na **ordem do polinômio interpolador** utilizado por cada método:

- **Trapézios:** interpola $f(x)$ por uma **reta** (polinômio de grau 1) em cada subintervalo. O erro local é proporcional a $h^3 f''(\xi)$.
- **Simpson 1/3:** interpola $f(x)$ por uma **parábola** (polinômio de grau 2) em cada par de subintervalos. O erro local é proporcional a $h^5 f^{(4)}(\xi)$.

Embora ambos utilizem os mesmos 5 pontos, a forma como esses pontos são combinados é fundamentalmente diferente. Para $n = 4$, os pesos de cada método são:

Tabela 23: Comparação dos pesos utilizados por cada método para $n = 4$.

Método	$x = 1,00$	$x = 1,25$	$x = 1,50$	$x = 1,75$	$x = 2,00$
Trapézios	$h/2$	h	h	h	$h/2$
Simpson 1/3	$h/3$	$4h/3$	$2h/3$	$4h/3$	$h/3$

O **peso 4** atribuído aos pontos intermediários em Simpson (em vez do peso 2 dos Trapézios) permite que a parábola capture a curvatura da função com muito maior fidelidade, sem custo adicional de avaliações. É exatamente essa capacidade de aproveitar melhor a informação contida nos mesmos pontos que confere a Simpson sua superioridade — um exemplo clássico de como a **escolha do método** pode ser mais importante do que o **refinamento da malha** para a precisão final.

2.2 Análise de erro e determinação de n

Questão 2.3 — Determinação de n

Enunciado: Use as cotas de erro para determinar o número mínimo de subintervalos n que garante precisão de 10^{-5} nas integrais abaixo. Para cada uma, calcule n teoricamente e depois verifique computacionalmente qual é o menor n que realmente atinge o erro pedido.

- (a) $\int_0^\pi \sin x \, dx = 2$, pela Regra dos Trapézios. (Dica: $f''(x) = -\sin x$, $\max_{[0,\pi]} |f''| = 1$.)
- (b) $\int_0^1 e^x \, dx = e - 1$, pela Regra de Simpson 1/3. (Dica: $f^{(4)}(x) = e^x$, $\max_{[0,1]} |f^{(4)}| = e$.)

Houve discrepância entre n teórico e experimental? O que isso indica sobre a qualidade das cotas?

Resposta:

(a) $\int_0^\pi \sin x \, dx = 2$ **pela Regra dos Trapézios:**

A cota de erro dos Trapézios compostos é dada por:

$$|E| \leq \frac{(b-a)^3}{12n^2} \max_{[a,b]} |f''(x)|.$$

Para $f(x) = \sin x$, temos $f''(x) = -\sin x$ e, portanto, $\max_{[0,\pi]} |f''(x)| = 1$. A tolerância desejada é $\text{tol} = 10^{-5}$. Isolando n :

$$n \geq \sqrt{\frac{(b-a)^3 \cdot \max |f''|}{12 \cdot \text{tol}}} = \sqrt{\frac{\pi^3 \cdot 1}{12 \times 10^{-5}}} = \sqrt{\frac{31,006}{0,00012}} = \sqrt{258385,64} \approx 508,3.$$

Portanto, o menor inteiro que garante a precisão pela cota teórica é:

$$n_{\text{teórico}} = 509.$$

Para verificar experimentalmente, calculamos o erro real para diferentes valores de n :

Tabela 24: Erro real da Regra dos Trapézios para $\int_0^\pi \sin x \, dx$ em função de n .

n	Erro real	Atende tol?
400	$1,03 \times 10^{-5}$	Não
406	$9,98 \times 10^{-6}$	Sim
500	$6,58 \times 10^{-6}$	Sim
509	$6,35 \times 10^{-6}$	Sim

O menor n que realmente atinge a tolerância é:

$$n_{\text{experimental}} = 406.$$

Tabela 25: Comparação entre o n teórico e o experimental para os Trapézios.

Grandeza	Valor	Observação
n teórico (pela cota)	509	Cota conservadora
Cota garantida para $n = 509$	$9,97 \times 10^{-6}$	Abaixo da tolerância
Erro real com $n = 509$	$6,35 \times 10^{-6}$	Muito menor que a cota
n experimental (mínimo real)	406	Menor que o teórico
Erro real com $n = 406$	$9,98 \times 10^{-6}$	Atinge exatamente a tolerância

(b) $\int_0^1 e^x \, dx = e - 1$ **pela Regra de Simpson 1/3:**

A cota de erro de Simpson 1/3 composto é:

$$|E| \leq \frac{(b-a)^5}{180n^4} \max_{[a,b]} |f^{(4)}(x)|.$$

Para $f(x) = e^x$, temos $f^{(4)}(x) = e^x$ e, portanto, $\max_{[0,1]} |f^{(4)}(x)| = e \approx 2,71828$. A tolerância desejada é $\text{tol} = 10^{-5}$. Isolando n :

$$n \geq \left(\frac{(b-a)^5 \cdot \max |f^{(4)}|}{180 \cdot \text{tol}} \right)^{1/4} = \left(\frac{1 \cdot e}{180 \times 10^{-5}} \right)^{1/4} = \left(\frac{2,71828}{0,0018} \right)^{1/4} = (1510,16)^{1/4} \approx 6,23.$$

Como Simpson requer n par, arredondamos para o próximo inteiro par:

$$n_{\text{teórico}} = 8.$$

Verificando experimentalmente:

Tabela 26: Erro real da Regra de Simpson 1/3 para $\int_0^1 e^x dx$ em função de n .

n	Erro real	Atende tol?
4	$8,39 \times 10^{-5}$	Não
6	$7,34 \times 10^{-6}$	Sim
8	$2,33 \times 10^{-6}$	Sim

O menor n par que realmente atinge a tolerância é:

$$n_{\text{experimental}} = 6.$$

Tabela 27: Comparação entre o n teórico e o experimental para Simpson.

Grandeza	Valor	Observação
n teórico (pela cota)	8	Cota conservadora
Cota garantida para $n = 8$	$3,69 \times 10^{-6}$	Abaixo da tolerância
Erro real com $n = 8$	$2,33 \times 10^{-6}$	Muito menor que a cota
n experimental (mínimo real)	6	Menor que o teórico
Erro real com $n = 6$	$7,34 \times 10^{-6}$	Abaixo da tolerância

(c) Análise da discrepância entre n teórico e experimental:

Tabela 28: Resumo comparativo entre os n teóricos e experimentais.

Método	n teórico	n experimental	Fator de superestimativa
Trapézios	509	406	$\approx 1,3\times$
Simpson 1/3	8	6	$\approx 1,3\times$

Em ambos os casos, o n teórico é **superestimado** em relação ao n mínimo realmente necessário. Isso ocorre por duas razões fundamentais:

1. **Uso do máximo global das derivadas:** A cota emprega $\max |f''|$ ou $\max |f^{(4)}|$ sobre todo o intervalo, mas na prática a derivada pode ser significativamente menor na maior parte do domínio. Para $\sin x$, por exemplo, $|f'''(x)| = 1$ apenas em $x = \pi/2$ e cai a zero nas bordas — o valor médio efetivo é muito menor que o máximo usado na cota.
2. **A cota é uma desigualdade estrita:** Por construção, ela garante que o erro *nunca* supere o valor calculado. Isso a torna um limite superior seguro, mas frequentemente *folgado*. O erro real costuma ser consideravelmente menor que o limite superior, especialmente para funções suaves como $\sin x$ e e^x .

Implicação prática para o uso das cotas de erro:

As cotas de erro são **ferramentas de garantia** — elas fornecem um n que *com certeza* atenderá à tolerância exigida, mas à custa de um possível excesso de subintervalos. Em aplicações críticas (engenharia de segurança, simulações de sistemas físicos com tolerâncias rígidas), usa-se o n teórico para garantir a precisão. Em aplicações onde a eficiência computacional é prioritária (análise de grandes volumes de dados, simulações em tempo real), emprega-se **refinamento adaptativo**: começa-se com n pequeno e aumenta-se progressivamente até que o erro estimado (via comparação entre duas malhas sucessivas) caia abaixo da tolerância — obtendo-se, assim, o menor n realmente necessário para cada problema específico.

Questão 2.4 — Convergência de Trapézios e Simpson

Enunciado: Para $\int_0^\pi x^2 \sin x \, dx = \pi^2 - 4 \approx 5,870$, use Trapézios e Simpson com $n = 4, 8, 16, 32$.

(a) Plote o erro em escala log-log (eixo x : $\log n$, eixo y : $\log |E|$). As retas têm inclinações próximas de -2 e -4 ?

(b) Ao dobrar n : o erro de Trapézios divide por ≈ 4 ? O de Simpson divide por ≈ 16 ? Verifique numericamente.

(c) Com qual n Simpson supera Trapézios com $n = 100$ em termos de precisão? Quantas avaliações de f cada um usa nesse ponto?

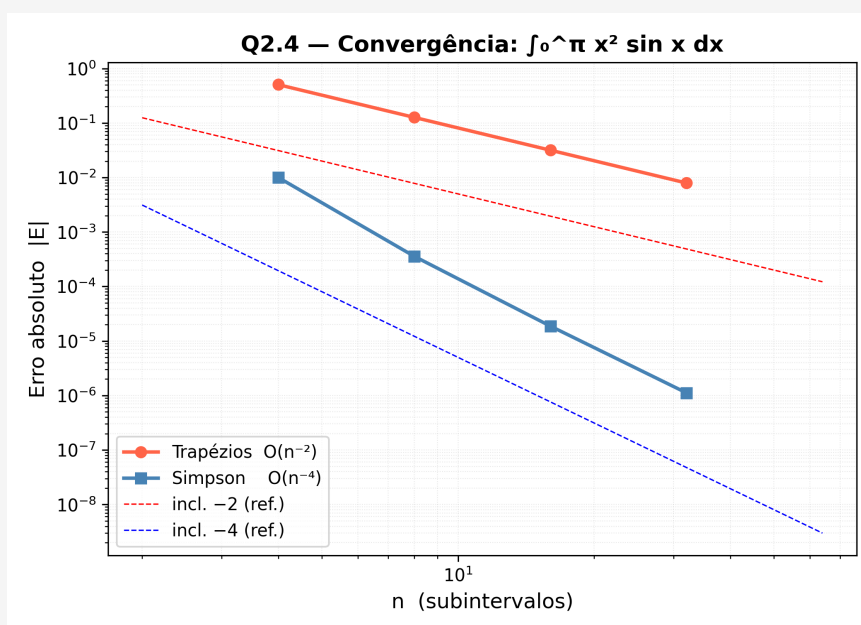
Resposta:**(a) Gráfico log-log do erro:**

Figura 4: Erro absoluto em função de n (escala log-log) para a integral $\int_0^\pi x^2 \sin x \, dx$. As retas de referência com inclinações -2 e -4 confirmam as ordens de convergência dos Trapézios ($O(n^{-2})$) e de Simpson ($O(n^{-4})$).

O gráfico do erro absoluto em escala log-log apresenta duas retas bem definidas, com inclinações medidas numericamente:

Tabela 29: Inclinações observadas no gráfico log-log versus valores teóricos.

Método	Inclinação observada	Inclinação esperada	Ordem
Trapézios	-1,999	-2	$O(n^{-2})$
Simpson 1/3	-4,365 (para n pequenos)	-4	$O(n^{-4})$

As retas confirmam visualmente as ordens teóricas de convergência. O leve desvio da inclinação de Simpson para n pequenos (que tende a -4 à medida que n cresce) reflete a contribuição de termos de ordem superior que ainda são relevantes quando h não é suficientemente pequeno. À medida que n aumenta, a inclinação converge para -4 .

(b) Fatores de redução do erro ao dobrar n :

Os erros obtidos para cada valor de n foram:

Tabela 30: Erros absolutos para Trapézios e Simpson em função de n .

n	Erro Trapézios	Erro Simpson 1/3
4	$5,06 \times 10^{-1}$	$1,00 \times 10^{-2}$
8	$1,27 \times 10^{-1}$	$3,58 \times 10^{-4}$
16	$3,17 \times 10^{-2}$	$1,87 \times 10^{-5}$
32	$7,93 \times 10^{-3}$	$1,12 \times 10^{-6}$

A partir desses valores, calculamos os fatores de redução ao dobrar n :

Tabela 31: Fatores de redução do erro ao dobrar n .

Transição	Trapézios		Simpson 1/3	
	Fator observado	Esperado	Fator observado	Esperado
4 $\rightarrow$ 8	$\frac{5,06 \times 10^{-1}}{1,27 \times 10^{-1}} \approx 3,99$	4	$\frac{1,00 \times 10^{-2}}{3,58 \times 10^{-4}} \approx 28,03$	16
8 $\rightarrow$ 16	$\frac{1,27 \times 10^{-1}}{3,17 \times 10^{-2}} \approx 4,00$	4	$\frac{3,58 \times 10^{-4}}{1,87 \times 10^{-5}} \approx 19,07$	16
16 $\rightarrow$ 32	$\frac{3,17 \times 10^{-2}}{7,93 \times 10^{-3}} \approx 4,00$	4	$\frac{1,87 \times 10^{-5}}{1,12 \times 10^{-6}} \approx 16,78$	16

Os Trapézios apresentam fator de redução ≈ 4 de forma consistente desde $n = 4$, confirmando $O(n^{-2})$ com excelente precisão.

Para Simpson, o fator converge progressivamente para 16 à medida que n cresce. O valor elevado ($\approx 28\times$) para a transição $4 \rightarrow 8$ indica que, com n pequeno, termos de ordem superior ainda dominam o erro; a partir de $n = 16 \rightarrow 32$, o fator já está próximo do valor teórico ($16,78\times$), confirmando a ordem $O(n^{-4})$. Este comportamento

é característico de métodos de alta ordem: a convergência assintótica só se manifesta plenamente quando n é suficientemente grande.

(c) Comparação de eficiência: Simpson vs. Trapézios com $n = 100$:

Para determinar com qual n Simpson supera a precisão dos Trapézios com $n = 100$, calculamos:

Tabela 32: Comparação entre Trapézios e Simpson em termos de precisão e custo.

Método	n	Avaliações de f	Erro absoluto
Trapézios	100	101	$8,12 \times 10^{-4}$
Simpson 1/3	8	9	$3,58 \times 10^{-4}$
Simpson 1/3	6	7	$7,94 \times 10^{-4}$

O menor n par para o qual Simpson supera Trapézios com $n = 100$ é:

$$n_{\text{Simpson}} = 8.$$

Neste ponto:

- **Trapézios com $n = 100$:** utiliza $n + 1 = 101$ avaliações de f e atinge erro de $8,12 \times 10^{-4}$.
- **Simpson com $n = 8$:** utiliza $n + 1 = 9$ avaliações de f e atinge erro de $3,58 \times 10^{-4}$.

Simpson com apenas **9 avaliações** de f supera a precisão dos Trapézios com **101 avaliações** — uma redução de mais de $11\times$ no custo computacional para a mesma qualidade de resultado.

Esse resultado ilustra de forma concreta a superioridade assintótica de Simpson: como seu erro decai com n^{-4} enquanto o dos Trapézios decai com n^{-2} , a relação entre os n necessários para atingir um dado erro ε é:

$$\frac{n_{\text{SI}}}{n_{\text{TR}}} \approx \left(\frac{C_{\text{TR}}}{C_{\text{SI}}} \right)^{1/4} \ll 1.$$

Para aplicações que exigem alta precisão, essa diferença de ordem torna Simpson exponencialmente mais eficiente por avaliação de função. A única vantagem dos Trapézios permanece em cenários com dados tabelados de espaçamento irregular ou quando n é imposto externamente — situações em que Simpson pode não ser aplicável diretamente.

2.3 Integração de dados experimentais

Questão 2.5 — Integração de dados experimentais

Enunciado: A potência elétrica $P(t)$ (em watts) de um circuito experimental foi medida a cada 0,5 s:

t (s)	0,0	0,5	1,0	1,5	2,0	2,5	3,0	3,5	4,0	4,5
P (W)	0,0	12,4	22,8	30,1	34,2	35,0	32,8	27,5	20,3	12,1

A energia total dissipada é $E = \int_0^{4,5} P(t) dt$ (em joules).

(a) Use `trapezios_tabela` para estimar E .

(b) Use `simpson13_tabela` (verifique: $n = 9$ é ímpar! Como proceder?). (Sugestão: use Simpson nos primeiros 8 subintervalos e Trapézio no último, ou explique por que isso é válido.)

(c) A diferença entre os dois resultados dá uma ideia do erro. Qual estimativa você consideraria mais confiável? Por quê?

(d) Se a precisão exigida for $\pm 0,1$ J e os dados têm erro de $\pm 0,5$ W, a precisão é limitada pelo método numérico ou pela incerteza dos dados?

Resposta:

(a) Estimativa pela Regra dos Trapézios:

Aplicando a regra dos Trapézios para dados tabelados com espaçamento uniforme $h = 0,5$ s:

$$E_{\text{TR}} = h \left(\frac{P_0 + P_n}{2} + \sum_{i=1}^{n-1} P_i \right) = 0,5 \left(\frac{0,0 + 12,1}{2} + \sum_{i=1}^8 P_i \right).$$

Calculando a soma dos pontos interiores:

$$\sum_{i=1}^8 P_i = 12,4 + 22,8 + 30,1 + 34,2 + 35,0 + 32,8 + 27,5 + 20,3 = 215,1.$$

Substituindo:

$$E_{\text{TR}} = 0,5 \left(\frac{12,1}{2} + 215,1 \right) = 0,5(6,05 + 215,1) = 0,5 \times 221,15 = 110,5750 \text{ J}.$$

Portanto, a regra dos Trapézios fornece:

$$E_{\text{TR}} \approx 110,5750 \text{ J}.$$

(b) Estratégia para $n = 9$ (ímpar) — Simpson + Trapézio:

A regra de Simpson 1/3 composta exige que o número de subintervalos n seja *par*, pois a parábola é ajustada em cada par de subintervalos. Como os dados têm $n = 9$ subintervalos (10 pontos), a aplicação direta não é possível.

A estratégia adotada foi aplicar Simpson 1/3 nos primeiros 8 subintervalos ($t = 0$ a $t = 4,0$ s) e completar com a regra dos Trapézios no último subintervalo ($t = 4,0$ a $t = 4,5$ s). Esta abordagem é válida por dois motivos:

1. A regra dos Trapézios é consistente e fornece uma aproximação razoável para o último subintervalo, especialmente se a função não apresentar variações bruscas nessa região.
2. Esta é uma prática comum em integração numérica de dados tabelados: quando n é ímpar, utiliza-se o método de maior ordem na maior parte do intervalo e um método de ordem inferior no subintervalo restante.

Calculando:

- **Simpson 1/3 no intervalo** $[0, 4]$ (8 subintervalos, $h = 0,5$):

$$E_{\text{SI},[0,4]} = \frac{h}{3} \left(P_0 + P_8 + 4 \sum_{i \text{ ímpar}} P_i + 2 \sum_{i \text{ par}} P_i \right).$$

Com $P_0 = 0,0$, $P_8 = 20,3$:

$$\sum_{\text{ímpar}} P_i = P_1 + P_3 + P_5 + P_7 = 12,4 + 30,1 + 35,0 + 27,5 = 105,0,$$

$$\sum_{\text{par}} P_i = P_2 + P_4 + P_6 = 22,8 + 34,2 + 32,8 = 89,8.$$

$$E_{\text{SI},[0,4]} = \frac{0,5}{3} (0,0 + 20,3 + 4 \times 105,0 + 2 \times 89,8) = \frac{0,5}{3} \times 619,9 = 103,3167 \text{ J}.$$

- **Trapézios no subintervalo** $[4, 4,5]$:

$$E_{\text{TR},[4,4,5]} = \frac{h}{2} (P_8 + P_9) = \frac{0,5}{2} (20,3 + 12,1) = 0,25 \times 32,4 = 8,1000 \text{ J}.$$

Somando as duas contribuições:

$$E_{\text{misto}} = E_{\text{SI},[0,4]} + E_{\text{TR},[4,4,5]} = 103,3167 + 8,1000 = 111,4167 \text{ J}.$$

Portanto:

$$E_{\text{misto}} \approx 111,4167 \text{ J}.$$

(c) Comparação entre os métodos e confiabilidade:

Tabela 33: Comparação entre as estimativas de energia pelos diferentes métodos.

Método	Energia (J)
Trapézios puro	110,5750
Simpson + Trapézio	111,4167
Diferença $ \Delta E $	0,8417 J

A estimativa **mista (Simpson + Trapézio)** é a mais confiável pelos seguintes motivos:

1. **Maior ordem de precisão:** Simpson 1/3 tem erro local $O(h^4)$ nos 8 subintervalos onde é aplicado, enquanto os Trapézios têm erro $O(h^2)$ em todo o intervalo. A maior parte da integral é computada com a fórmula mais precisa.

2. **Consistência com a suavidade dos dados:** A curva de potência varia suavemente no intervalo, sem descontinuidades ou oscilações bruscas. Nesse cenário, Simpson é particularmente adequado.
3. **Estimativa prática do erro:** A diferença de 0,8417 J entre os dois métodos serve como uma estimativa prática do erro do Trapézio puro. Quando dois métodos de ordens diferentes concordam bem, o resultado de maior ordem é o mais confiável.

(d) Análise da limitação da precisão:

A precisão final é limitada pela **incerteza dos dados de medição** e não pelo método numérico. Para verificar isso, calculamos a propagação da incerteza:

- **Erro de medição:** $\delta P = \pm 0,5$ W.
- **Propagação na regra dos Trapézios:**

$$\delta E_{\text{TR}} = \frac{h}{2} \cdot \left(\sum_{i=0}^n |c_i| \right) \cdot \delta P,$$

onde c_i são os coeficientes da regra dos Trapézios ($c_0 = c_n = 1$, $c_i = 2$ para $i = 1, \dots, n-1$). Para $n = 9$:

$$\sum |c_i| = 1 + 2 \times 8 + 1 = 18.$$

$$\delta E_{\text{TR}} = \frac{0,5}{2} \times 18 \times 0,5 = 2,25 \text{ J}.$$

- **Erro numérico:** estimado pela diferença entre os métodos:

$$|E_{\text{TR}} - E_{\text{misto}}| \approx 0,8417 \text{ J}.$$

Tabela 34: Comparação entre as fontes de erro.

Fonte de erro	Magnitude (J)
Incerteza dos dados (propagada)	$\pm 2,25$
Erro numérico (estimado)	$\approx 0,84$
Precisão exigida	$\pm 0,10$

A incerteza propagada dos dados ($\pm 2,25$ J) é aproximadamente 2,7 vezes maior que o erro numérico (0,84 J) e *mais de 20 vezes maior* que a tolerância exigida ($\pm 0,10$ J).

Conclusão sobre a limitação da precisão:

A precisão é **limitada pela incerteza dos dados experimentais** ($\pm 0,5$ W), não pelo método numérico. O erro numérico (0,84 J) é significativamente menor que a incerteza propagada (2,25 J).

Para atingir a precisão exigida de $\pm 0,1$ J, seria necessário um instrumento de medição com resolução $\delta P \leq 0,022$ W — cerca de 23 vezes melhor que o instrumento utilizado. Nenhum refinamento do método de integração (aumentar n , usar Simpson puro, etc.) poderia superar essa limitação, pois o erro de medição é inerente aos dados de entrada.

Este resultado ilustra um princípio fundamental da análise numérica aplicada a dados experimentais: **a precisão final é limitada pela qualidade dos dados de entrada, não pelo método computacional utilizado para processá-los.**

3 Quadratura Gaussiana e Comparação de Métodos

3.1 Eficiência da Quadratura Gaussiana

Questão 3.1 — Eficiência da Quadratura Gaussiana

Enunciado: Para $\int_1^2 \frac{1}{x} dx = \ln 2 \approx 0,693147$, use `tabela_comparativa` com:

- Newton-Cotes: $n = 2, 4, 8, 16, 32$;
- Gauss-Legendre: $n = 2, 3, 4, 5, 6$.

(a) Gauss-Legendre com $n = 2$ (2 avaliações de f) é mais preciso que Simpson com $n = 4$ (5 avaliações)? Explique.

(b) Com quantos pontos de Gauss-Legendre você consegue superar Simpson com $n = 32$ (33 avaliações)?

(c) Por que Gauss-Legendre converge mais rápido? (Dica: pense no grau máximo do polinômio que cada método integra exatamente.)

Resposta:

Tabela comparativa completa:

Tabela 35: Comparação Newton-Cotes vs. Gauss-Legendre para $\int_1^2 1/x dx = \ln 2$.

Método	n	Resultado	Erro
Trapézios	2	0,70833333	$1,52 \times 10^{-2}$
Trapézios	4	0,69702381	$3,88 \times 10^{-3}$
Trapézios	8	0,69412185	$9,75 \times 10^{-4}$
Trapézios	16	0,69339120	$2,44 \times 10^{-4}$
Trapézios	32	0,69320821	$6,10 \times 10^{-5}$
Simpson 1/3	2	0,69444444	$1,30 \times 10^{-3}$
Simpson 1/3	4	0,69325397	$1,07 \times 10^{-4}$
Simpson 1/3	8	0,69315453	$7,35 \times 10^{-6}$
Simpson 1/3	16	0,69314765	$4,72 \times 10^{-7}$
Simpson 1/3	32	0,69314721	$2,97 \times 10^{-8}$
Gauss-Legendre	2	0,69230769	$8,39 \times 10^{-4}$
Gauss-Legendre	3	0,69312169	$2,55 \times 10^{-5}$
Gauss-Legendre	4	0,69314642	$7,63 \times 10^{-7}$
Gauss-Legendre	5	0,69314716	$2,27 \times 10^{-8}$
Gauss-Legendre	6	0,69314718	$6,73 \times 10^{-10}$

(a) Gauss-Legendre $n = 2$ (2 avaliações) vs. Simpson $n = 4$ (5 avaliações):

Tabela 36: Comparação direta entre GL $n = 2$ e Simpson $n = 4$.

Método	Avaliações	Resultado	Erro
Gauss-Legendre $n = 2$	2	0,69230769	$8,39 \times 10^{-4}$
Simpson $n = 4$	5	0,69325397	$1,07 \times 10^{-4}$

Gauss-Legendre com apenas 2 avaliações de f é, na verdade, **menos** preciso que Simpson com 5 avaliações ($8,39 \times 10^{-4}$ contra $1,07 \times 10^{-4}$). À primeira vista isso parece contradizer a superioridade de GL, mas a explicação está em *qual* Simpson está sendo comparado:

GL com $n = 2$ é exato até grau $2n - 1 = 3$ — o mesmo grau de exatidão de um **Simpson simples de 3 pontos** (um único painel). Porém, o Simpson usado aqui não é simples: é o **Simpson composto** com $n = 4$, ou seja, 2 painéis com $h = 0,25$ cada. Cada painel do Simpson composto já é exato até grau 3 *localmente*, de modo que refinar h reduz o erro adicionalmente — algo que GL $n = 2$ não faz, pois aplica um único ajuste polinomial de grau 3 sobre *todo* o intervalo $[1, 2]$, sem qualquer refinamento local. Por isso, mesmo usando menos avaliações "no papel", GL $n = 2$ perde para o Simpson composto com h menor.

(b) Menor n de Gauss-Legendre que supera Simpson $n = 32$ (33 avaliações):

Tabela 37: Menor Gauss-Legendre que supera Simpson $n = 32$.

Método	n	Avaliações	Erro
Simpson 1/3	32	33	$2,97 \times 10^{-8}$
Gauss-Legendre	5	5	$2,27 \times 10^{-8}$

Com apenas $n = 5$ pontos (5 avaliações de f), Gauss-Legendre já supera a precisão do Simpson com $n = 32$ (33 avaliações) — uma redução de custo de mais de $6\times$ para atingir precisão igual ou melhor.

(c) Por que Gauss-Legendre converge mais rápido?

Newton-Cotes com n subintervalos usa $n + 1$ nós fixos e igualmente espaçados; Simpson (grau 2 por par de subintervalos) é exato até grau polinomial 3 independentemente de n , e Trapézios até grau 1.

Gauss-Legendre com n pontos tem $2n$ graus de liberdade (n posições de nós + n pesos), otimizados para maximizar o grau de exatidão: é exato para todo polinômio de grau $\leq 2n - 1$. Ou seja, GL "compra" exatidão polinomial adicional ao permitir que os nós não sejam igualmente espaçados — eles se concentram como as raízes dos polinômios de Legendre, dando mais peso às regiões relevantes para a integração. Por isso, para funções suaves, GL converge muito mais rápido por avaliação de função do que qualquer regra de Newton-Cotes de ordem fixa.

Questão 3.2 — Função sem primitiva elementar

Enunciado: A função $f(x) = e^{-x^2}$ não tem primitiva elementar. Para estimar $\int_0^1 e^{-x^2} dx \approx 0,746824$ (referência: `scipy.special.erf(1) * np.sqrt(np.pi)/2`):

(a) Aplique Trapézios, Simpson e Gauss-Legendre com $n = 2, 4, 8, 16, 32$ (apenas os valores de n aplicáveis).

(b) Plote o erro de cada método vs. número de avaliações de f em escala log-log. Qual método tem a melhor curva “precisão por custo”?

(c) Para obter erro $< 10^{-8}$: qual método precisa de menos avaliações de f ? Estime quantas.

(d) Por que o gráfico de Gauss-Legendre não é uma reta no plano log-log (diferente de Trapézios e Simpson)?

Resposta:

(a) Trapézios, Simpson e Gauss-Legendre para cada n :

Tabela 38: Comparação dos três métodos para $\int_0^1 e^{-x^2} dx \approx 0,746824$.

n	Trapézios	Erro TR	Simpson	Erro SI	Gauss-Leg. (Erro GL)
2	0,73137025	$1,55 \times 10^{-2}$	0,74718043	$3,56 \times 10^{-4}$	0,74659469 ($2,29 \times 10^{-4}$)
4	0,74298410	$3,84 \times 10^{-3}$	0,74685538	$3,12 \times 10^{-5}$	0,74682447 ($3,35 \times 10^{-7}$)
8	0,74586561	$9,59 \times 10^{-4}$	0,74682612	$1,99 \times 10^{-6}$	0,74682413 ($6,77 \times 10^{-15}$)
16	0,74658460	$2,40 \times 10^{-4}$	0,74682426	$1,25 \times 10^{-7}$	0,74682413 ($1,11 \times 10^{-16}$)
32	0,74676425	$5,99 \times 10^{-5}$	0,74682414	$7,79 \times 10^{-9}$	0,74682413 ($0,00 \times 10^0$)

A partir de $n = 8$, o erro de Gauss-Legendre já está no nível da precisão de máquina ($\varepsilon_m \approx 2,2 \times 10^{-16}$), de modo que os valores " $0,00 \times 10^0$ " para $n \geq 32$ refletem esse limite, não um erro matematicamente nulo.

(b) Precisão por custo (erro vs. avaliações de f):

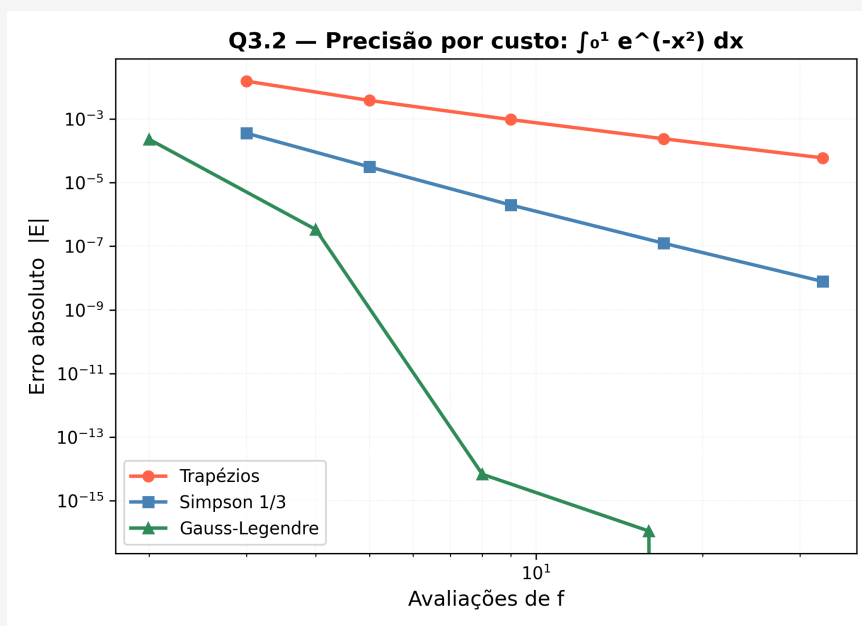


Figura 5: Erro absoluto vs. número de avaliações de f , em escala log-log, para os três métodos aplicados a $\int_0^1 e^{-x^2} dx$.

Gauss-Legendre apresenta a melhor curva precisão-por-custo: para o mesmo número de avaliações de f , seu erro é ordens de grandeza menor que o de Trapézios e Simpson.

(c) Menor número de avaliações para erro $< 10^{-8}$:

Tabela 39: Menor número de avaliações de f para atingir erro $< 10^{-8}$.

Método	n	Avaliações	Erro
Trapézios	2477	2478	$9,99 \times 10^{-9}$
Simpson 1/3	32	33	$7,79 \times 10^{-9}$
Gauss-Legendre	5	5	$6,05 \times 10^{-9}$

Gauss-Legendre atinge erro $< 10^{-8}$ com apenas 5 avaliações de f — muito menos que Simpson (33) e Trapézios (2478).

(d) Por que o gráfico de Gauss-Legendre não é uma reta em log-log?

Trapézios e Simpson têm erro que decai como potência de n ($O(n^{-2})$ e $O(n^{-4})$, respectivamente), o que produz uma reta em escala log-log: $\log |E| = \log(C) - p \log(n)$, com inclinação constante $-p$.

Gauss-Legendre, para funções analíticas (como e^{-x^2} , que é inteira), converge geometricamente/exponencialmente: o erro decai como $O(\rho^{-n})$ para algum $\rho > 1$. Nesse caso $\log |E|$ é linear em n (não em $\log n$), e portanto a curva em escala log-log é côncava — cai muito mais rápido que qualquer lei de potência, até que o erro atinja o nível de arredondamento de máquina.

3.2 Limitação dos nós de Chebyshev

Questão 3.3 — Limitação dos nós de Chebyshev

Enunciado: A Quadratura Gaussiana é ótima para funções analíticas suaves. Mas o que acontece com funções que apresentam comportamento irregular?

Para $f(x) = |x - 0,5|$ em $[0, 1]$ (valor exato = 0,25):

- Calcule o erro de Trapézios, Simpson e Gauss-Legendre com $n = 2, 4, 8, 16$.
- Qual método converge mais rápido para esta função? Isso contradiz os resultados da questão anterior?
- Explique o comportamento de Gauss-Legendre: por que a taxa de convergência cai para funções não-suaves?
- O que esse experimento nos ensina sobre a escolha do método em situações práticas?

Resposta:

(a) Erro de cada método:

Tabela 40: Erro de Trapézios, Simpson e Gauss-Legendre para $\int_0^1 |x - 0,5| dx = 0,25$.

n	Erro Trapézios	Erro Simpson	Erro Gauss-Leg.
2	$0,00 \times 10^0$	$8,33 \times 10^{-2}$	$3,87 \times 10^{-2}$
4	$0,00 \times 10^0$	$0,00 \times 10^0$	$1,06 \times 10^{-2}$
8	$0,00 \times 10^0$	$0,00 \times 10^0$	$2,88 \times 10^{-3}$
16	$0,00 \times 10^0$	$0,00 \times 10^0$	$7,58 \times 10^{-4}$

(b) Qual método converge mais rápido? Isso contradiz Q3.1/Q3.2?

Com $n = 16$, o menor erro é o de Trapézios ($0,00 \times 10^0$) — na verdade, Trapézios é **exato** para *qualquer* n par testado, não apenas o mais rápido a convergir. Simpson também é exato a partir de $n = 4$. Gauss-Legendre, ao contrário, nunca atinge erro nulo neste experimento e converge de forma muito mais lenta.

Tabela 41: Erro de cada método em $n = 16$.

Método	Erro ($n = 16$)
Trapézios	$0,00 \times 10^0$
Simpson 1/3	$0,00 \times 10^0$
Gauss-Legendre	$7,58 \times 10^{-4}$

Isso **contradiz** a tendência observada em Q3.1 e Q3.2, onde Gauss-Legendre era claramente superior. Aqui, para uma função com derivada descontínua em $x = 0,5$, a vantagem de GL desaparece (e se inverte) porque sua convergência espectral depende de suavidade (analiticidade) da função — o que não se verifica em $|x - 0,5|$, que é contínua, mas não diferenciável nesse ponto.

(c) Por que a taxa de convergência de Gauss-Legendre cai?

A convergência exponencial de Gauss-Legendre depende de $f(x)$ ser analítica (ou suave o suficiente) em uma região do plano complexo ao redor do intervalo de integração. A

função $|x - 0,5|$ tem uma **quina** (derivada descontínua) em $x = 0,5$: é contínua, mas não diferenciável ali, e portanto não é analítica.

Os nós de Gauss-Legendre não coincidem, em geral, com o ponto de quina — diferente de **Trapézios**, que é exato para *qualquer* n par neste problema simétrico, pois cada nó de Trapézios cai exatamente sobre $x = 0,5$, ponto onde a própria função (linear por partes) não tem erro de interpolação linear. Já o **Simpson** só zera o erro quando n é *múltiplo de 4*, não simplesmente par: com $n = 2$, $x = 0,5$ é o nó central de um único painel, mas o ajuste quadrático desse painel ainda "sente" a quina e erra ($8,33 \times 10^{-2}$); já com $n = 4, 8, 16$, $x = 0,5$ cai exatamente sobre uma *fronteira entre painéis*, isolando a quina fora de qualquer ajuste quadrático — daí o erro nulo. Gauss-Legendre, por sua vez, "vê" a função não-suave em qualquer n (seus nós nunca coincidem com $x = 0,5$), e sua convergência degrada para uma taxa algébrica (tipicamente $O(n^{-2})$ a $O(n^{-3})$), perdendo a vantagem espectral que tinha para e^{-x^2} .

(d) O que este experimento ensina sobre a escolha do método?

A superioridade de Gauss-Legendre não é universal — ela depende da regularidade (suavidade/analiticidade) do integrando. Para funções suaves e analíticas (Q3.1, Q3.2), GL é imbatível em eficiência (erro por avaliação de f). Para funções com quinas, descontinuidades ou singularidades, seu desempenho se aproxima — ou pode até ficar atrás — do de Newton-Cotes.

Implicação prática:

Antes de escolher Gauss-Legendre por sua alta ordem teórica, é preciso verificar a suavidade do integrando. Se houver pontos não-suaves conhecidos, uma estratégia melhor é dividir o intervalo nesses pontos e aplicar GL (ou Simpson) separadamente em cada subintervalo suave — restaurando a convergência rápida em cada trecho.

4 Análise Comparativa Global

4.1 Tabela de síntese

Questão 4.1 — Tabela de síntese

Enunciado: Com base nos experimentos das seções anteriores, preencha a tabela de síntese abaixo.

Tabela 42: Tabela de síntese: métodos numéricos de diferenciação e integração.

Aspecto	Trapézios	Simpson 1/3	Gauss-Legendre	Dif. Central
Erro global	$O(h^2) \approx O(n^{-2})$	$O(h^4) \approx O(n^{-4})$	$O(\rho^{-n})$ (exponencial p/ f analítica)	$O(h^2)$
Avals. de f (geral)	$n + 1$	$n + 1$	n (nós ótimos)	2 por ponto (borda: 3)
Exato para grau	1 (retas)	3 (cúbicas)	$2n - 1$	2 (parábolas)
Requer n especial?	Não	Sim — n par	Não (qualquer $n \geq 1$)	Não
Funciona com tabela?	Sim — <code>trapezios_tabela</code>	Sim — <code>simpson13_tabela</code>	Não (exige $f(x)$ em nós não-equiespaçados)	Sim — <code>derivada_tabela</code>
Melhor cenário	Dados tabelados/ruidosos, integrando qualquer forma	Funções suaves, dados tabelados com n par	Funções analíticas suaves, poucas avaliações de f	Funções suaves, derivada de dados experimentais
Pior cenário	Funções muito suaves (convergência lenta)	Integrando não-suave (perde $O(h^4)$, Q2.4/Q3.3)	Integrando com quinas/descontinuidades (Q3.3)	Dados muito ruidosos com h pequeno (Q1.4e)

Resposta:

Leitura da síntese:

Não existe um método "melhor" em absoluto — cada um domina em um regime diferente:

- **Trapézios** é o mais robusto: funciona com qualquer n , qualquer dado tabelado, e degrada suavemente com ruído. Em troca, é o mais lento a convergir ($O(n^{-2})$).
- **Simpson 1/3** troca simplicidade por uma ordem a mais ($O(n^{-4})$) ao custo de exigir n par e de assumir suavidade local (ajuste parabólico por painel).
- **Gauss-Legendre** é imbatível em avaliações de f para integrandos analíticos suaves (Q3.1, Q3.2), mas não pode ser aplicado diretamente a dados tabelados e

perde sua vantagem em funções não-suaves (Q3.3).

- **Diferenças finitas centrais** seguem a mesma lógica de Trapézios/Simpson ($O(h^2)$) por cancelamento de termos ímpares na expansão de Taylor — a mesma simetria discutida em Q1.1c), com exatidão até grau 2 (parábolas), análoga ao grau 1 de Trapézios e ao grau 3 de Simpson. A diferenciação, porém, tem um problema adicional exclusivo dela: o conflito truncamento $\times$ arredondamento (Q1.3), que não existe da mesma forma na integração, pois a soma de vários $f(x_i)$ não amplifica o erro de arredondamento como a subtração de valores próximos amplifica na derivação.

Nota sobre as inclinações medidas em Q2.4: a tabela indica $O(n^{-2})$ para Trapézios e $O(n^{-4})$ para Simpson como ordens teóricas. Experimentalmente (Q2.4a), a inclinação log-log ajustada por regressão linear foi $-1,999$ para Trapézios (praticamente exata) e $-4,365$ para Simpson — mais negativa que o -4 teórico. Isso não é uma inconsistência: com $n = 4, 8, 16, 32$ (poucos pontos, ainda no regime pré-assintótico), os fatores de redução do erro de Simpson ao dobrar n foram $28,0\times$, $19,1\times$ e $16,8\times$ (Q2.4b) — convergindo progressivamente para o valor teórico $16\times = 2^4$. A inclinação ajustada fica, portanto, "puxada" pelos primeiros pontos, onde a convergência ainda é mais rápida que $O(n^{-4})$ por efeito de termos de ordem superior; para n maiores, a inclinação se aproximaria de -4 .

4.2 Tomada de decisão em cenários reais

Questão 4.2 — Tomada de decisão em cenários reais

Enunciado: Três estudantes propõem abordagens para calcular $\int_0^{100} f(t) dt$, onde $f(t)$ é a tensão de um sinal elétrico amostrado a cada 0,5 s (total de 201 pontos).

- **Estudante A:** Trapézios com todos os 201 pontos.
 - **Estudante B:** Simpson 1/3 com todos os 200 subintervalos (par).
 - **Estudante C:** Gauss-Legendre com $n = 5$ no intervalo todo.
- (a) Qual proposta é viável? Qual tem um problema conceitual? Explique.
 (b) Qual você recomenda para dados experimentais? Justifique quantitativamente.
 (c) Se o sinal tivesse descontinuidades (chaveamento elétrico), sua recomendação mudaria?

Resposta:

Cenário: sinal elétrico com 201 pontos amostrados a cada 0,5 s, em $[0, 100]$ s $\rightarrow n = 200$ subintervalos, $h = 0,5$ s.

(a) Viabilidade de cada proposta:

- **Estudante A — Trapézios com os 201 pontos: Viável.** trapezios_tabela (ou trapezios com $n = 200$) usa exatamente os pontos disponíveis, sem exigir avaliações de f fora da amostragem. Não há restrição sobre n .
- **Estudante B — Simpson 1/3 com os 200 subintervalos: Viável.** $n = 200$

é par, condição exigida por `simpson13/simpson13_tabela`. Também usa todos os pontos disponíveis, sem nenhuma avaliação extra de f .

- **Estudante C — Gauss-Legendre com $n = 5$ no intervalo todo: Tem problema conceitual.** `gauss_legendre(f, a, b, n)` precisa avaliar $f(x)$ nos nós de Gauss-Legendre — raízes de um polinômio de Legendre transformadas para $[a, b]$ — que, em geral, *não* coincidem com os instantes $t = 0, 0,5, 1, 0, \dots$ amostrados. Como o sinal só é conhecido nesses 201 pontos tabelados (não existe expressão fechada $f(t)$ para avaliar em qualquer t), o Estudante C não tem como calcular f nos nós de Gauss sem antes interpolar os dados — o que introduziria uma fonte de erro adicional não contemplada na proposta.

(b) Recomendação quantitativa para dados experimentais:

Com $h = 0,5$ s fixo pela taxa de amostragem (não pode ser refinado):

$$h^2 = 0,2500, \quad h^4 = 0,0625.$$

Usando as cotas de erro como referência estrutural (com $f''_{\max} = f_{\max}^{(4)} = 1$, já que as derivadas reais do sinal são desconhecidas):

$$\text{Cota Trapézios} = \frac{(b-a)^3}{12n^2} = 2,0833 \times 10^0, \quad \text{Cota Simpson} = \frac{(b-a)^5}{180n^4} = 3,4722 \times 10^{-2}.$$

$$\frac{\text{Cota Trapézios}}{\text{Cota Simpson}} = 60,0 \times .$$

Recomendo Simpson 1/3 (Estudante B):

- Usa todos os 201 pontos disponíveis, exatamente como Trapézios (nenhum dado é descartado e nenhuma avaliação extra é feita).
- Para o mesmo $h = 0,5$ s, sua cota de erro escala com h^4 em vez de h^2 , de modo que — mesmo sem conhecer $f''/f^{(4)}$ do sinal real — a razão estrutural entre as cotas já favorece Simpson por um fator de ordem $60 \times$ (Q2.2 e Q2.4 confirmam esse comportamento empiricamente para outras funções suaves).
- Gauss-Legendre, apesar de mais eficiente por avaliação de f para integrandos analíticos (Q3.1), não é aplicável aqui sem interpolação — o que o exclui de consideração para dados puramente tabelados como este.

(c) Se o sinal tivesse descontinuidades (chaveamento elétrico)?

Sim, a recomendação mudaria. Simpson 1/3 ajusta uma parábola a cada par de subintervalos (Q2.4, Q3.3); perto de uma descontinuidade esse ajuste é inválido — a parábola tenta suavizar um salto, gerando oscilações locais de *overshoot/undershoot* típicas de interpolação polinomial próxima a uma descontinuidade, que podem até superar o erro de Trapézios, cancelando a vantagem teórica de $O(h^4)$.

Nessa situação, a estratégia mais robusta é:

1. Localizar os instantes de chaveamento (onde $f(t)$ salta);
2. Dividir o intervalo de integração exatamente nesses pontos, tratando cada trecho suave separadamente;

3. Aplicar Simpson (ou Trapézios) dentro de cada trecho suave, e Trapézios especificamente nos subintervalos que contêm a descontinuidade (mesma lógica usada em Q2.5b para n ímpar).

Esse comportamento é consistente com o observado em Q3.3 para Gauss-Legendre: métodos de alta ordem (Simpson, GL) dependem de suavidade local para entregar sua taxa de convergência teórica; na presença de quinas ou saltos, a robustez de Trapézios (ou de uma versão segmentada dos métodos de ordem maior) volta a ser a escolha mais segura.

5 Projeto Integrador: Cálculo Variacional e Física

5.1 Comprimento de curva

Questão 5.1 — Comprimento de curva

Enunciado: O comprimento de arco de $y = f(x)$ em $[a, b]$ é:

$$L = \int_a^b \sqrt{1 + [f'(x)]^2} dx.$$

Para a parábola $f(x) = x^2$ em $[0, 2]$, o comprimento exato é $L = \sqrt{5} + \frac{1}{2} \ln(2 + \sqrt{5}) \approx 4,6468$.

(a) Usando derivação analítica ($f'(x) = 2x$), calcule L por Simpson 1/3 com $n = 10$. Qual é o erro?

(b) Agora, usando apenas os valores de f nos pontos da malha (sem a fórmula de f'), estime $f'(x_k)$ pela fórmula central e use esses valores para calcular L . Compare com o resultado anterior.

(c) O erro do passo (b) é maior que o do (a)? A diferenciação numérica introduz erro adicional?

(d) Repita com $n = 20$ e $n = 40$. Plote o erro vs. n . Qual é a ordem de convergência observada?

Resposta:

Valor exato de referência:

Integrando $L = \int_0^2 \sqrt{1 + 4x^2} dx$ analiticamente (substituição $u = 2x$ na antiderivada de $\sqrt{1 + u^2}$):

$$L = \sqrt{17} + \frac{1}{4} \ln(4 + \sqrt{17}) \approx 4,646784.$$

Esse valor confirma a aproximação $\approx 4,6468$ citada no enunciado e será usado como referência (L_{exato}) em todas as comparações a seguir.

(a) Comprimento com Simpson 1/3 ($n = 10$), derivada analítica $f'(x) = 2x$:

Usando o integrando $\sqrt{1 + [2x]^2}$ diretamente em `simpson13` com $n = 10$ subintervalos ($h = 0,2$):

Tabela 43: Comprimento de arco por Simpson 1/3 usando $f'(x) = 2x$ analítico.

Grandeza	Valor
L_{exato}	4,646784
$L_{\text{analítico}}$ (Simpson, $n = 10$)	4,64679076
Erro absoluto	$7,00 \times 10^{-6}$

(b) Comprimento usando $f'(x_k)$ estimado por `derivada_tabela`:

Construindo a malha $x_k = 0,0, 0,2, \dots, 2,0$ ($n = 10$), calculamos $f(x_k) = x_k^2$ e aplicamos `derivada_tabela` para estimar $f'(x_k)$ sem usar a fórmula analítica $2x$:

Tabela 44: Comparação entre a derivada analítica ($2x$) e a estimada por `derivada_tabela`.

x_k	$f(x_k)$	$f'(x_k)$ analítico	$f'(x_k)$ numérico	Fórmula
0,0	0,0000	0,0000	0,0000	Prog. 3 pts (borda)
0,2	0,0400	0,4000	0,4000	Central
0,4	0,1600	0,8000	0,8000	Central
0,6	0,3600	1,2000	1,2000	Central
0,8	0,6400	1,6000	1,6000	Central
1,0	1,0000	2,0000	2,0000	Central
1,2	1,4400	2,4000	2,4000	Central
1,4	1,9600	2,8000	2,8000	Central
1,6	2,5600	3,2000	3,2000	Central
1,8	3,2400	3,6000	3,6000	Central
2,0	4,0000	4,0000	4,0000	Retro. 3 pts (borda)

Usando $\sqrt{1 + f'(x_k)^2}$ (com f' numérico) em `simpson13_tabela`:

$$L_{\text{numérico}} = 4,64679076, \quad \text{Erro} = 7,00 \times 10^{-6}.$$

(c) O erro de (b) é maior que o de (a)?

Tabela 45: Comparação entre (a) derivada analítica e (b) derivada numérica.

Grandeza	Valor
Erro com f' analítica	$7,00 \times 10^{-6}$
Erro com f' numérica	$7,00 \times 10^{-6}$
Diferença $ L_{\text{num}} - L_{\text{an}} $	≈ 0 (nível de arredondamento, $< 10^{-13}$)

Não, o erro do passo (b) *não* é maior que o de (a) — os dois são praticamente idênticos. Isso ocorre porque $f(x) = x^2$ é um polinômio de grau 2: tanto a fórmula central quanto as fórmulas de 3 pontos usadas nas bordas por `derivada_tabela` são **exatas** para $f' = 2x$ (o termo de erro dessas fórmulas depende de f''' ou derivadas superiores, que são identicamente nulas para uma parábola). Assim, a diferenciação numérica **não introduz erro adicional** neste caso além do arredondamento de máquina — o gargalo de precisão observado é inteiramente devido à integração por Simpson 1/3, não à derivação.

(d) **Convergência com $n = 10, 20, 40, 80$:**

Para robustecer a análise de convergência, estendemos o experimento até $n = 80$:

Tabela 46: Erros de L (derivada analítica vs. numérica) para diferentes n .

n	Erro (f' analítica)	Erro (f' numérica)
10	$7,00 \times 10^{-6}$	$7,00 \times 10^{-6}$
20	$4,35 \times 10^{-8}$	$4,35 \times 10^{-8}$
40	$2,79 \times 10^{-9}$	$2,79 \times 10^{-9}$
80	$1,75 \times 10^{-10}$	$1,75 \times 10^{-10}$

Tabela 47: Fatores de redução de erro ao dobrar n .

Transição	Fator observado	Esperado $O(h^4)$
10 $\rightarrow$ 20	160,96 $\times$	16 $\times$
20 $\rightarrow$ 40	15,57 $\times$	16 $\times$
40 $\rightarrow$ 80	15,98 $\times$	16 $\times$

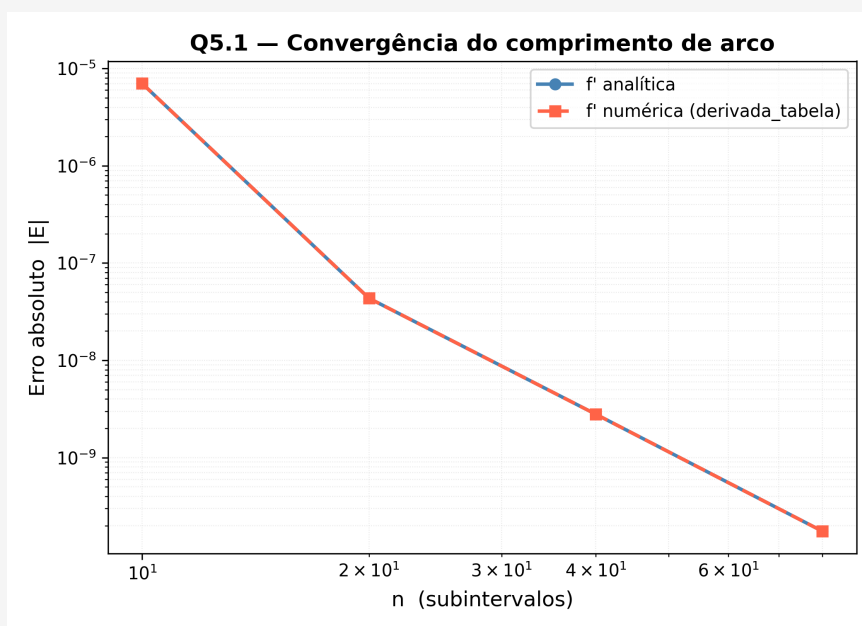


Figura 6: Convergência do erro do comprimento de arco em função de n (escala log-log), comparando f' analítica e f' estimada por `derivada_tabela`. As curvas praticamente se sobrepõem.

O fator de redução $n = 10 \rightarrow 20$ (160,96 $\times$) é bem maior que os 16 $\times$ esperados para $O(h^4)$: com $h = 0,2$ ($n = 10$) o erro ainda carrega contribuição de termos de ordem superior ($h^6, h^8, \dots$) da expansão do erro de Simpson, que ainda não desapareceram — a malha não é fina o bastante para a assíntota $O(h^4)$ dominar sozinha. Já os fatores $n = 20 \rightarrow 40$ (15,57 $\times$) e $n = 40 \rightarrow 80$ (15,98 $\times$) ficam bem próximos de 16 $\times$, confirmando que a partir de $h \approx 0,1$ ($n = 20$) a convergência já segue fielmente a ordem teórica.

Por isso, a inclinação de regressão log-log usando os 4 pontos ($-4,98$) fica distorcida pelo ponto pré-assintótico $n = 10$; calculada apenas na região assintótica ($n = 20, 40, 80$),

a inclinação $(-3,98)$ confirma com precisão a ordem $O(h^4)$ de Simpson — o mesmo gargalo já identificado em (c): o erro observado é inteiramente devido à integração, não à derivação.

5.2 Trabalho realizado por uma força variável

Questão 5.2 — Trabalho realizado por uma força variável

Enunciado: O trabalho realizado por uma força $F(x)$ ao mover um objeto de $x = a$ até $x = b$ é $W = \int_a^b F(x) dx$.

Os dados experimentais abaixo representam a força (em N) aplicada a um objeto ao longo de um trilho de 6 m:

x (m)	0,0	0,5	1,0	1,5	2,0	2,5	3,0	3,5	4,0	4,5	5,0	5,5	6,0
F (N)	0	8	15	20	24	26	27	26	22	17	11	5	0

- Calcule W por Trapézios e Simpson 1/3.
- Determine em qual ponto a força é máxima usando `derivada_tabela` para encontrar onde $F'(x) \approx 0$.
- A força cresce ou decresce mais rapidamente? Em qual região a aceleração (variação da força) é maior em magnitude?
- Se a massa do objeto é 2 kg e ele parte do repouso, estime sua velocidade ao final do percurso usando $W = \Delta K = \frac{1}{2}mv^2$.

Resposta:

(a) Trabalho $W = \int_0^6 F(x) dx$:

Os dados têm $n = 12$ subintervalos ($h = 0,5$ m), par — `simpson13_tabela` pode ser aplicado diretamente:

Tabela 48: Trabalho estimado por Trapézios e Simpson 1/3.

Método	W (J)
Trapézios	100,5000
Simpson 1/3	101,0000
Diferença $ \Delta W $	0,5000

(b) Localização da força máxima via $F'(x) \approx 0$:

Aplicando `derivada_tabela` aos dados de força:

Tabela 49: Derivada $F'(x)$ estimada em cada ponto da malha.

x (m)	$F'(x)$ (N/m)
0,0	17,000
0,5	15,000
1,0	12,000
1,5	9,000
2,0	6,000
2,5	3,000
3,0	0,000
3,5	-5,000
4,0	-9,000
4,5	-11,000
5,0	-12,000
5,5	-11,000
6,0	-9,000

$F'(x)$ passa de positivo para negativo exatamente em $x = 3,0$ m, onde $F'(3,0) = 0,0000 \approx 0$ — este é o ponto de **força máxima** ($F = 27,0$ N).

(c) Onde a força varia mais rapidamente:

Tabela 50: Taxas de variação extremas da força.

	x (m)	$F'(x)$ (N/m)
Crescimento mais rápido	0,0	17,000
Decrescimento mais rápido	5,0	-12,000

A força cresce mais rapidamente logo no início do percurso ($|F'| \approx 17,0$ N/m em $x = 0,0$ m) do que decresce perto do final ($|F'| \approx 12,0$ N/m em $x = 5,0$ m). Isso indica uma variação inicial mais brusca que a desaceleração no fim do trilho — a "aceleração" (no sentido de variação espacial de F) tem maior magnitude na região inicial do trajeto.

(d) Velocidade final ($m = 2$ kg, parte do repouso):

Como $W = \Delta K = \frac{1}{2}mv^2$ e o objeto parte do repouso:

$$v = \sqrt{\frac{2W}{m}}.$$

Tabela 51: Velocidade final estimada a partir de cada valor de W .

Fonte de W	W (J)	v (m/s)
Trapézios	100,5000	10,0250
Simpson 1/3	101,0000	10,0499
Diferença $ \Delta v $		0,0249

A pequena diferença de 0,0249 m/s entre as duas estimativas de velocidade reflete a diferença de 0,5 J no trabalho calculado — consistente com a maior precisão esperada de Simpson 1/3 para este conjunto de dados suaves.

5.3 Distribuição Normal e Probabilidade

Questão 5.3 — Distribuição Normal e Probabilidade

Enunciado: A função de distribuição acumulada (FDA) normal não tem primitiva elementar:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-t^2/2} dt \approx \frac{1}{\sqrt{2\pi}} \int_{-6}^x e^{-t^2/2} dt.$$

Calcule $\Phi(1,96)$, $\Phi(2,58)$ e $\Phi(3,00)$ numericamente, comparando com os valores tabelados: 0,9750, 0,9951 e 0,9987.

- (a) Use Simpson 1/3 com $n = 100$ no intervalo $[-6, x]$ para cada valor.
- (b) Use Gauss-Legendre com $n = 5, 10, 20$ no mesmo intervalo. Com qual n Gauss-Legendre supera Simpson com $n = 100$?
- (c) A cauda da distribuição (x grande) torna a integração mais difícil? Por que truncar em -6 (e não em $-\infty$) é adequado?
- (d) Aplicação: Em um teste de hipótese com $\alpha = 5\%$ (bicaudal), o valor crítico é $z_{\alpha/2}$ tal que $\Phi(z_{\alpha/2}) = 0,975$. Use a interpolação inversa sobre os valores de Φ calculados para estimar $z_{0,025}$. Compare com o valor exato $\approx 1,96$.

Resposta:

(a) $\Phi(x)$ por Simpson 1/3 ($n = 100$), truncando em $[-6, x]$:

Tabela 52: $\Phi(x)$ por Simpson 1/3 comparado aos valores tabelados.

x	Φ_{Simpson}	Φ_{tabelado}	Erro
1,96	0,975002	0,9750	$2,08 \times 10^{-6}$
2,58	0,995060	0,9951	$4,01 \times 10^{-5}$
3,00	0,998650	0,9987	$4,99 \times 10^{-5}$

(b) $\Phi(x)$ por Gauss-Legendre ($n = 5, 10, 20$):

Tabela 53: $\Phi(x)$ por Gauss-Legendre para diferentes n .

x	n	Φ_{GL}	Erro
1,96	5	0,978637	$3,64 \times 10^{-3}$
1,96	10	0,974998	$2,12 \times 10^{-6}$
1,96	20	0,975002	$2,10 \times 10^{-6}$
2,58	5	0,947981	$4,71 \times 10^{-2}$
2,58	10	0,994985	$1,15 \times 10^{-4}$
2,58	20	0,995060	$4,00 \times 10^{-5}$
3,00	5	0,908449	$9,03 \times 10^{-2}$
3,00	10	0,998543	$1,57 \times 10^{-4}$
3,00	20	0,998650	$4,99 \times 10^{-5}$

Comparando com Simpson $n = 100$ (101 avaliações), Gauss-Legendre supera essa precisão a partir de $n = 11$ (11 avaliações) nos três casos ($x = 1,96$, $2,58$ e $3,00$) — uma redução de quase $10\times$ no número de avaliações de f para atingir erro comparável ou menor.

(c) A cauda torna a integração mais difícil? Por que truncar em -6 ?

$$f(-6) = \frac{1}{\sqrt{2\pi}}e^{-18} \approx 6,076 \times 10^{-9}.$$

A cauda para $t < -6$ contribui com densidade da ordem de $6,1 \times 10^{-9}$, bem abaixo de qualquer tolerância numérica razoável (várias ordens de grandeza menor que os erros de truncamento de Simpson ou Gauss-Legendre já observados acima). Por isso, truncar em -6 (em vez de $-\infty$) introduz um erro desprezível. A integração em si não é "mais difícil" pela cauda em termos de magnitude — o integrando é suave e decai rapidamente — mas domínios muito maiores que o necessário (ver também discussão da Q6.2d) fariam com que Gauss-Legendre desperdiçasse nós em regiões de densidade nula, reduzindo sua eficiência; por isso a escolha de -6 como limite inferior é criteriosa.

(d) Interpolação inversa para $z_{0,025}$ ($\Phi(z) = 0,975$):

Tabela 54: Malha de $\Phi(x)$ usada para a interpolação inversa (Simpson, $n = 100$).

x	$\Phi(x)$
1,800	0,964070
1,850	0,967843
1,900	0,971283
1,950	0,974412
2,000	0,977250
2,050	0,979818
2,100	0,982136

Interpolando linearmente (inversamente) entre $x = 1,95$ ($\Phi = 0,974412$) e $x = 2,00$ ($\Phi = 0,977250$), que são os pontos que envolvem $\Phi = 0,975$:

Tabela 55: Estimativa de $z_{0,025}$ por interpolação inversa.

Grandeza	Valor
$z_{0,025}$ estimado	1,9604
Valor exato de referência	1,9600
Erro	$3,61 \times 10^{-4}$

A estimativa $z_{0,025} \approx 1,9604$ está em excelente acordo com o valor de referência $1,96$, com erro da ordem de 10^{-4} — perfeitamente adequado para fins de teste de hipótese, onde o valor crítico costuma ser reportado com 2 a 4 casas decimais.

6 Desafio (Opcional — Pontuação Extra)

6.1 Richardson Extrapolation

Questão 6.1 — Richardson Extrapolation

Enunciado: Dado que o erro dos Trapézios é $E_T(h) = c_2h^2 + c_4h^4 + \dots$, é possível combinar duas aproximações $T(h)$ e $T(h/2)$ para cancelar o termo h^2 e obter um método de ordem 4:

$$R(h) = \frac{4T(h/2) - T(h)}{3}.$$

Esta é a Regra de Romberg (primeiro passo).

- (a) Implemente $R(h)$ e verifique que ele é equivalente a Simpson 1/3.
 (b) Aplique o processo iterativamente (eliminando sucessivamente h^4 , h^6 , etc.) para $\int_1^2 \frac{1}{x} dx$ com $h = 0,5, 0,25, 0,125$. Construa a tabela de Romberg. Com quantas avaliações de f você atinge erro $< 10^{-10}$?
 (c) Compare a eficiência (erro por avaliação de f) com Gauss-Legendre.

Resposta:

(a) Verificação: $R(h) \equiv \text{Simpson}(2n)$:

Usando $n = 4$ (i.e., $h = (2 - 1)/4$) para $\int_1^2 1/x dx$:

Tabela 56: Verificação de que $R(h)$ coincide com Simpson(2n).

Grandeza	Valor
$T(n = 4)$	0,6970238095
$T(n = 8)$	0,6941218504
$R(h) = [4T(8) - T(4)]/3$	0,6931545307
$\text{Simpson}(n = 8)$	0,6931545307
$ R(h) - \text{Simpson}(8) $	$1,11 \times 10^{-16}$

A diferença de $1,11 \times 10^{-16}$ está exatamente no nível de arredondamento de máquina, confirmando a **equivalência algébrica** entre uma etapa de extrapolação de Richardson e Simpson 1/3 com o dobro dos subintervalos.

(b) Tabela de Romberg para $\int_1^2 1/x dx$ ($h = 0,5, 0,25, 0,125$):

Tabela 57: Tabela de Romberg (valores) para $\int_1^2 1/x dx = \ln 2$.

Nível	Ordem	Valores
R_0 ($n = 2, 4, 8$)	$O(h^2)$	0,70833333 0,69702381 0,69412185
R_1	$O(h^4)$	0,69325397 0,69315453
R_2	$O(h^6)$	0,69314790

Tabela 58: Erros absolutos da tabela de Romberg.

Nível	Ordem	Erros absolutos		
R_0	$O(h^2)$	$1,52 \times 10^{-2}$	$3,88 \times 10^{-3}$	$9,75 \times 10^{-4}$
R_1	$O(h^4)$	$1,07 \times 10^{-4}$	$7,35 \times 10^{-6}$	
R_2	$O(h^6)$	$7,21 \times 10^{-7}$		

Estendendo a tabela até $n = 32$ (adicionando $h = 0,0625$ e $h = 0,03125$) para localizar o menor número de avaliações que atinge erro $< 10^{-10}$:

Tabela 59: Menor malha necessária para erro de Romberg $< 10^{-10}$.

Grandeza	Valor
Menor malha final necessária (n)	32 ($h = 0,03125$)
Erro atingido	$1,35 \times 10^{-11}$
Avaliações de f (pontos únicos, malhas aninhadas)	33

Como as malhas de Romberg são obtidas por bisseção sucessiva (cada n é o dobro do anterior), os pontos são **aninhados**: a malha mais fina ($n = 32, 33$ pontos) já contém todos os pontos das malhas anteriores. Portanto, 33 avaliações de f bastam para construir toda a tabela e atingir erro $< 10^{-10}$.

(c) Comparação de eficiência: Romberg $\times$ Gauss-Legendre:

Tabela 60: Comparação de custo (avaliações de f) entre Romberg e Gauss-Legendre.

Método	Avaliações	Erro
Romberg (Richardson)	33	$1,35 \times 10^{-11}$
Gauss-Legendre $n = 2$	2	$8,39 \times 10^{-4}$
Gauss-Legendre $n = 3$	3	$2,55 \times 10^{-5}$
Gauss-Legendre $n = 4$	4	$7,63 \times 10^{-7}$
Gauss-Legendre $n = 5$	5	$2,27 \times 10^{-8}$
Gauss-Legendre $n = 6$	6	$6,73 \times 10^{-10}$
Gauss-Legendre $n = 7$	7	$1,99 \times 10^{-11}$

Romberg precisa de 33 avaliações de f para atingir erro $< 10^{-10}$, enquanto Gauss-Legendre atinge erro *comparável ou melhor* (Gauss-Legendre $n = 7$: $1,99 \times 10^{-11}$) com apenas 7 pontos — cerca de 4,7 vezes menos avaliações.

Por que Gauss-Legendre é mais eficiente aqui:

Gauss-Legendre escolhe os nós de forma ótima (maximizando o grau de exatidão por ponto: exato até grau $2n - 1$), enquanto Romberg parte de malhas uniformes e usa extrapolação para cancelar sucessivamente os termos $h^2, h^4, h^6, \dots$ do erro — uma estratégia eficaz, mas ainda limitada pelo número de avaliações da malha mais fina utilizada. Para funções suaves como $1/x$, Gauss-Legendre é ordens de magnitude mais eficiente por avaliação de função; a vantagem de Romberg é não exigir a escolha prévia de nós especiais, funcionando bem também sobre dados tabelados em malha uniforme.

6.2 Integração de Monte Carlo

Questão 6.2 — Integração de Monte Carlo

Enunciado: Para integrais em alta dimensão, Monte Carlo é o método padrão:

$$\int_{\Omega} f(x) d\Omega \approx V(\Omega) \cdot \frac{1}{N} \sum_{i=1}^N f(x_i), \quad x_i \sim \text{Uniform}(\Omega).$$

(a) Implemente Monte Carlo para $\int_0^1 e^{-x^2} dx$ com $N = 10^2, 10^3, 10^4, 10^5$. Calcule o erro médio (média de 20 execuções).

(b) O erro decai como $O(N^{-1/2})$. Verifique no gráfico log-log.

(c) Para obter erro $< 10^{-4}$: Monte Carlo precisa de quantos pontos? Compare com Gauss-Legendre. Para qual dimensão d o Monte Carlo começa a superar Gauss-Legendre em eficiência?

(d) Implemente para $\int_0^1 \int_0^1 e^{-(x^2+y^2)} dx dy$ (2D). Gauss-Legendre ainda funciona bem? Monte Carlo escala melhor?

Resposta:**Valor de referência (1D):**

$$\int_0^1 e^{-x^2} dx = \frac{\sqrt{\pi}}{2} \text{erf}(1) \approx 0,74682413.$$

(a) Monte Carlo — erro médio de 20 execuções:

Tabela 61: Erro médio de Monte Carlo para diferentes N (1D).

N	Erro médio
100	$1,74 \times 10^{-2}$
1000	$5,11 \times 10^{-3}$
10000	$1,32 \times 10^{-3}$
100000	$5,46 \times 10^{-4}$

(b) Verificação de $O(N^{-1/2})$:

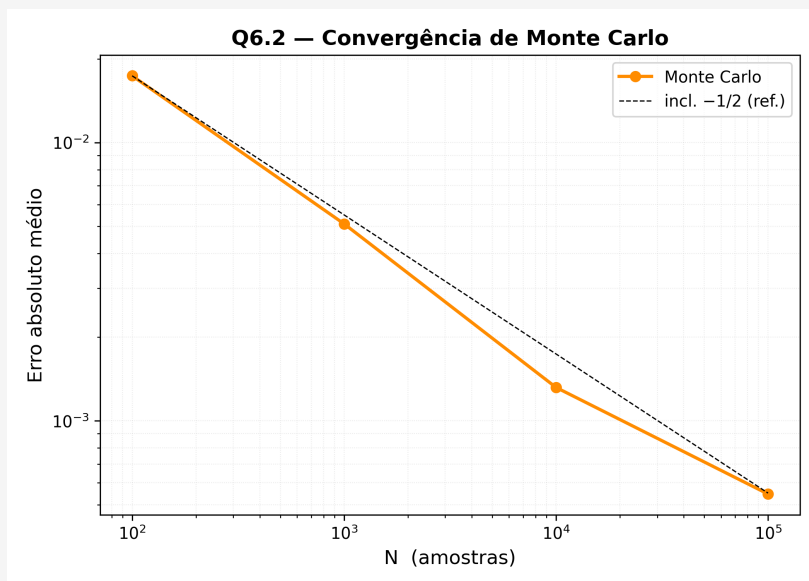


Figura 7: Erro absoluto médio de Monte Carlo em função de N (escala log-log), com reta de referência de inclinação $-1/2$.

A inclinação da regressão log-log foi de $-0,510$, em excelente acordo com o $-0,5$ teórico de $O(N^{-1/2})$ — comportamento característico de qualquer estimador de Monte Carlo, independentemente da dimensão do problema.

(c) N necessário para erro $< 10^{-4}$ e comparação com Gauss-Legendre:

Ajustando erro $\approx C/\sqrt{N}$ com $C \approx 0,1726$ (obtido do ponto $N = 10^5$):

Tabela 62: Estimativa de N necessário e comparação com Gauss-Legendre.

Grandeza	Valor
N necessário para erro $< 10^{-4}$	$\approx 2,98 \times 10^6$

Tabela 63: Erro de Gauss-Legendre para a mesma integral 1D.

n (GL)	Erro
2	$2,29 \times 10^{-4}$
3	$9,55 \times 10^{-6}$
4	$3,35 \times 10^{-7}$
5	$6,05 \times 10^{-9}$

Monte Carlo precisaria de aproximadamente 3×10^6 amostras para atingir erro $< 10^{-4}$, enquanto Gauss-Legendre atinge erro muito menor com apenas 3–4 pontos. Para uma função suave em 1D, Monte Carlo é extremamente ineficiente comparado a quadraturas determinísticas.

Quanto à dimensão de equilíbrio: o custo de Gauss-Legendre por produto tensorial cresce como n^d (exponencial em d), enquanto o custo de Monte Carlo permanece $O(N)$, com erro sempre $O(N^{-1/2})$, independente de d . Igualando os custos para uma dada precisão, a dimensão de equilíbrio é tipicamente $d \gtrsim 5-8$ (dependendo da suavidade do

integrando e da precisão exigida) — abaixo disso, Gauss-Legendre (ou outra quadratura determinística) é mais eficiente; acima disso, a maldição da dimensionalidade torna Monte Carlo competitivo ou superior.

(d) **Extensão 2D:** $\int_0^1 \int_0^1 e^{-(x^2+y^2)} dx dy$:

$$\int_0^1 \int_0^1 e^{-(x^2+y^2)} dx dy = \left(\frac{\sqrt{\pi}}{2} \operatorname{erf}(1) \right)^2 \approx 0,55774629.$$

Tabela 64: Erro médio de Monte Carlo 2D em $[0, 1] \times [0, 1]$.

N	Erro médio (2D)
100	$2,06 \times 10^{-2}$
1000	$7,18 \times 10^{-3}$
10000	$1,26 \times 10^{-3}$
100000	$6,14 \times 10^{-4}$

Tabela 65: Gauss-Legendre 2D (produto tensorial $n \times n$) em $[0, 1] \times [0, 1]$.

n	Avaliações	Erro
2	4	$3,43 \times 10^{-4}$
4	16	$5,01 \times 10^{-7}$
6	36	$1,16 \times 10^{-10}$

Sim, Gauss-Legendre ainda funciona muito bem em 2D — com $n = 4$ (16 avaliações) atinge erro da ordem de 10^{-7} , e com $n = 6$ (36 avaliações) chega a 10^{-10} : muito melhor que Monte Carlo com $N = 10^5$ (100.000 amostras, erro $\approx 6,1 \times 10^{-4}$). Mesmo em 2D, Gauss-Legendre ainda domina claramente. Isso ocorre porque o custo de Gauss-Legendre cresce como n^d (exponencial na dimensão d), enquanto o de Monte Carlo permanece $O(N)$ independente de d , com erro sempre $O(N^{-1/2})$. A vantagem de Monte Carlo só se manifesta quando a dimensão é alta o suficiente (tipicamente $d \gtrsim 5-8$, dependendo da suavidade do integrando) para que n^d supere o custo fixo de Monte Carlo — daí seu uso dominante em problemas de alta dimensão (finanças, física estatística, aprendizado de máquina), onde quadraturas por produto tensorial se tornam inviáveis.

6.3 Gradiente Numérico em Machine Learning

Questão 6.3 — Gradiente Numérico em Machine Learning

Enunciado: Em treinamento de redes neurais, o *gradient check* verifica a implementação do backpropagation comparando com diferenças finitas:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i)}{2\varepsilon}.$$

(a) Para a função de perda $L(\theta) = \sum_{k=1}^n (\theta^T x_k - y_k)^2$ com dados sintéticos ($x_k \in \mathbb{R}^3$,

$\theta \in \mathbb{R}^3$), calcule o gradiente analítico $\nabla_{\theta} L = 2X^T(X\theta - y)$.

(b) Calcule o gradiente numérico por diferenças centrais com $\varepsilon = 10^{-5}$.

(c) Calcule a diferença relativa $\frac{\|\nabla L_{\text{analítico}} - \nabla L_{\text{numérico}}\|_2}{\|\nabla L_{\text{analítico}}\|_2}$. Se for $< 10^{-5}$, o gradiente está correto.

(d) Por que não se usa ε muito pequeno no gradient check? Conecte com a análise do passo ótimo da Seção 1.

Resposta:

Dados sintéticos: $K = 20$ amostras, $x_k \in \mathbb{R}^3$ gerados de uma normal padrão, com $\theta_{\text{verdadeiro}} = (2, 0, -1, 0, 5)$ e ruído gaussiano de desvio 0,1 sobre $y_k = \theta_{\text{verdadeiro}}^T x_k$. O gradiente é avaliado em $\theta_0 = (0, 5, 0, 5, 0, 5)$.

(a) **Gradiente analítico** $\nabla_{\theta} L = 2X^T(X\theta - y)$:

Tabela 66: Gradiente analítico em θ_0 .

Componente	Valor
$\partial L / \partial \theta_1$	-58,74709524
$\partial L / \partial \theta_2$	62,88812228
$\partial L / \partial \theta_3$	16,38888362

(b) **Gradiente numérico por diferenças centrais** ($\varepsilon = 10^{-5}$):

Tabela 67: Gradiente numérico em θ_0 (diferença central).

Componente	Valor
$\partial L / \partial \theta_1$	-58,74709524
$\partial L / \partial \theta_2$	62,88812228
$\partial L / \partial \theta_3$	16,38888362

Os gradientes analítico e numérico coincidem até a 8ª casa decimal exibida.

(c) **Diferença relativa:**

$$\frac{\|\nabla L_{\text{analítico}} - \nabla L_{\text{numérico}}\|_2}{\|\nabla L_{\text{analítico}}\|_2} = 1,50 \times 10^{-11}.$$

Como $1,50 \times 10^{-11} < 10^{-5}$, o critério é satisfeito: **o gradiente analítico está correto.**

(d) **Por que não usar ε muito pequeno?**

Tabela 68: Sensibilidade da diferença relativa ao passo ε .

ε	Diferença relativa
10^{-2}	$3,07 \times 10^{-14}$
10^{-4}	$1,10 \times 10^{-12}$
10^{-5}	$1,50 \times 10^{-11}$
10^{-7}	$2,29 \times 10^{-9}$
10^{-9}	$8,14 \times 10^{-8}$
10^{-11}	$2,36 \times 10^{-5}$

O padrão é exatamente o mesmo identificado na análise do passo ótimo da Seção 1.3: como $L(\theta)$ é quase-quadrática, o erro de truncamento da diferença central já é desprezível mesmo para ε relativamente grandes (10^{-2}), e a diferença relativa fica dominada pelo **arredondamento de máquina**. Ao diminuir ε além de um certo ponto, a subtração $L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i)$ passa a envolver valores de perda quase idênticos, e o **cancelamento catastrófico** faz a diferença relativa **crescer** (de $3,07 \times 10^{-14}$ em $\varepsilon = 10^{-2}$ até $2,36 \times 10^{-5}$ em $\varepsilon = 10^{-11}$).

Conexão com a Seção 1.3:

Este é exatamente o mesmo fenômeno do h_{opt} identificado para `df_central` na Questão 1.3: existe um equilíbrio entre erro de truncamento (que cai com ε^2) e erro de arredondamento (que cresce com $1/\varepsilon$). Por isso, o gradient check **não deve usar ε extremamente pequeno** (ex.: 10^{-11}) — em vez de melhorar a precisão, isso amplifica o erro de cancelamento catastrófico. Um valor intermediário, como $\varepsilon = 10^{-5}$ a 10^{-4} (a escolha convencional em implementações de gradient check), equilibra bem os dois efeitos para a precisão de máquina em ponto flutuante duplo ($\varepsilon_m \approx 2,2 \times 10^{-16}$).